

Aula 04

*DataPrev (Perfil 3: Desenvolvimento de
Software) Banco de Dados - 2026
(Pós-Edital)*

Autor:

Lucas Fargoni Di Ianni

09 de Julho de 2026

Índice

1) 6.0 SQL - Avançado - Consultas Aninhadas - Teoria	3
2) 6.0 SQL - Avançado - Consultas Aninhadas - Questões Comentadas - FGV	23
3) 6.0 SQL - Avançado - Consultas Aninhadas - Lista de Questões - FGV	92
4) 6.1 SQL - Avançado - Objetos no Banco de Dados - Teoria	143
5) 6.1 SQL - Avançado - Objetos no Banco de Dados - Questões Comentadas - MULTIBANCAS	161
6) 6.1 SQL - Avançado - Objetos no Banco de Dados - Lista de Questões - MULTIBANCAS	189

SQL - AVANÇADO

Consultas Aninhadas (Subconsultas)

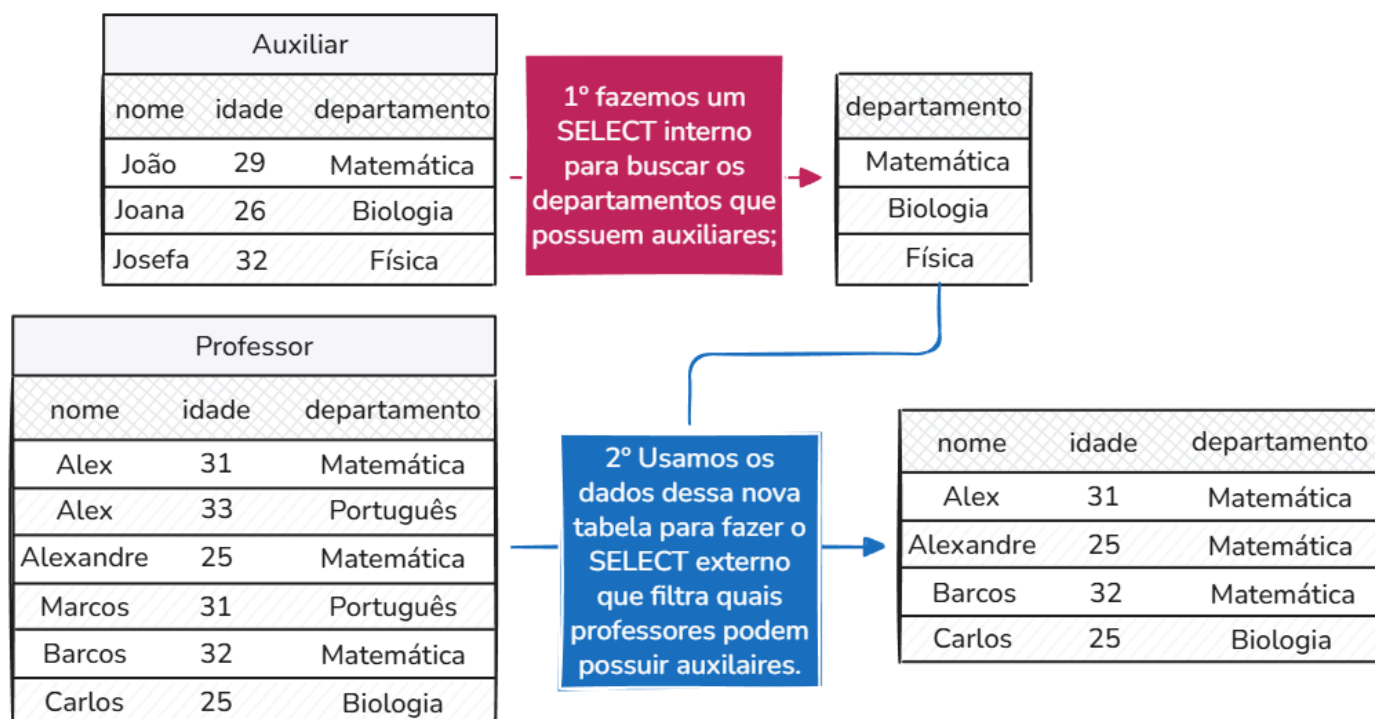
Consultas aninhadas, como o próprio nome diz, é uma consulta (SELECT) que ocorre dentro de outra consulta (tipicamente outro SELECT, mas também pode aparecer em INSERT, UPDATE ou DELETE).

E por que queremos fazer um SELECT para fornecer uma tabela para que outro SELECT, professor? Isso é muito confuso! É verdade, mas, se não fosse assim, alguns problemas seriam impossíveis de serem resolvidos, então somos forçados a escrever esse tipo de SQL complexo.

Nesses casos, em vez de escrevermos uma única consulta que busca todos dados, antes escrevemos uma consulta "secundária" (a consulta aninhada), que irá rodar primeiro, e essa fornecerá uma nova tabela de dados para que a consulta "principal" (a chamada de consulta externa) consiga resolver.

Por exemplo, vamos supor que queremos encontrar, em uma tabela Professor, quais professores podem possuir um auxiliar, ou seja, quais estão alocados em um departamento que há auxiliares alocados também.

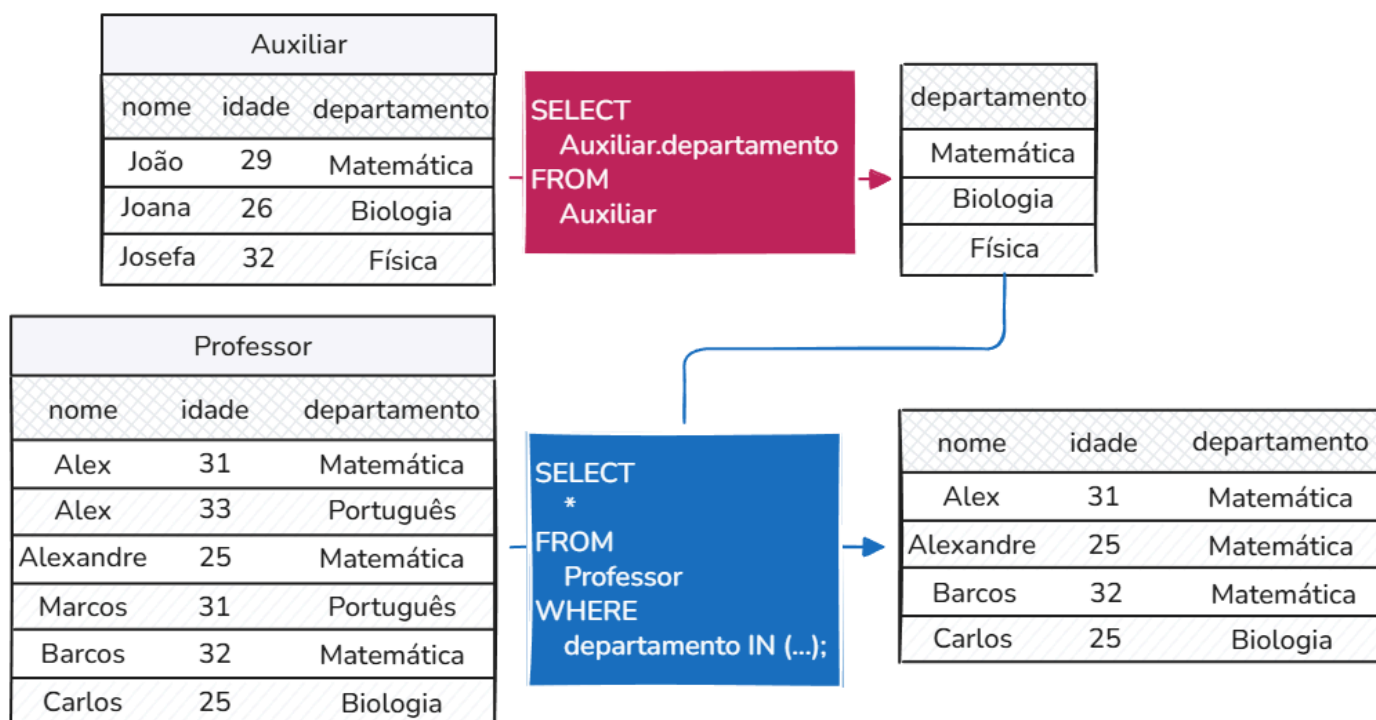
Para isso, precisamos fazer duas coisas: 1. Obter todos departamentos que possuem auxiliares através de um SELECT; e 2. Filtrar para os departamentos encontrados quais professores estão em um deles em um outro SELECT externo.



Bom, sabemos que para obter os departamentos dos auxiliares podemos fazer um SELEC na tabela Auxiliar buscando pelos seus departamentos. Depois disso, podemos fazer um SELECT na tabela Professor filtrando (WHERE) quais departamentos estão (IN) na lista de departamentos de auxiliares.

Na prática, o que fazemos é aninhar a primeira consulta na segunda, porque aquela fornece os dados necessários para esta resolver.

```
SELECT
  *
FROM
  Professor
WHERE
  departamento IN (
    SELECT Auxiliar.departamento FROM Auxiliar -- Consulta aninhada
  );
```



(CEBRASPE (CESPE)/Ana (FINEP)/2024) Em relação a SQL, é correto afirmar que

- a) uma função agregada serve para verificar a integridade de dados.
- b) uma instrução compreende uma série de comandos de programação.

- c) uma subquery é uma consulta aninhada dentro de outra consulta.
- d) uma subquery é uma consulta usada para verificar a integridade de dados.
- e) o comando CASE é usado para transformar texto em números.

Comentários:

- a) **Errada.** Função agregada não tem relação com integridade de dados; esse tema sequer trata de integridade nesta aula.
- b) **Errada.** Uma instrução SQL é um comando único enviado ao banco, e não uma série de comandos de programação.
- c) **Correta.** Subquery (subconsulta) é exatamente uma consulta SELECT que aparece dentro de outra consulta, podendo estar no WHERE, FROM ou até no SELECT.
- d) **Errada.** Subquery serve para fornecer dados a outra consulta (como filtrar valores ou gerar uma tabela intermediária), e não para verificar integridade de dados.
- e) **Errada.** O CASE não é usado para transformar texto em números, e essa função não é descrita assim no conteúdo desta aula.

Gabarito: C

(CEBRASPE (CESPE)/Ana (BACEN)/BACEN/2024) Quando utilizado o comando SELECT, podem-se incluir várias subqueries internas na mesma instrução.

Comentários:

Uma subquery (ou subconsulta) é simplesmente um SELECT que aparece dentro de outro comando, e nada impede que você use várias delas numa mesma instrução. Pode ter subconsulta no WHERE para filtrar, no FROM para servir como tabela temporária e até no próprio SELECT para gerar uma coluna calculada, tudo no mesmo comando. Por isso a afirmação está correta.

Gabarito: Certo

Continuando, desde 1999, o padrão SQL passou a permitir a cláusula WITH, que materializa o conceito de Common Table Expressions (CTEs), ou, em tradução livre, “expressões de tabela comuns”.

Embora esses nomes todos sejam bem confusos, aqui nada mais temos que a possibilidade de atribuir um nome à subconsulta para utilizá-lo depois múltiplas vezes no mesmo SQL.

A seguir, vamos escrever a mesma consulta com WITH, nomeando nossa consulta aninhada de “departamentos_auxiliares”.

```
WITH departamentos_auxiliares AS (  
    SELECT  
        Auxiliar.departamento  
    FROM  
        Auxiliar  
)  
  
SELECT  
    *  
FROM  
    Professor  
WHERE  
    departamento IN (SELECT departamento FROM departamentos_auxiliares);
```

Percebam que não podemos usar a subconsulta "departamentos_auxiliares" depois em outro SQL, porque uma CTE definida com WITH só existe durante a execução daquela consulta específica.

Ou seja, ela não persiste no banco e não pode ser referenciada em outras consultas posteriores, como se "morresse" no ";" ao final do comando.

```
-- Primeira consulta: usa a CTE  
WITH departamentos_auxiliares AS (  
    SELECT departamento FROM Auxiliar  
)  
SELECT * FROM Professor  
WHERE departamento IN (SELECT departamento FROM departamentos_auxiliares);  
-- A CTE "morre" aqui, ao final dessa instrução.  
  
-- Segunda consulta: tenta usar a CTE de novo  
SELECT * FROM departamentos_auxiliares;  
-- ERRO! A CTE não existe mais.
```



(FCC/SEFAZ MT/2026) Deseja-se obter a apuração do total de ICMS por município e, a partir desse resultado intermediário, retornar apenas municípios cujo total esteja acima da média estadual, usando SQL Oracle com legibilidade e reúso do resultado agregado. Para isso, a estrutura adequada deve usar

- Common Table Expression (WITH) para total por município e, na consulta final, filtrar por comparação com a média.
- SUM e AVG na mesma lista SELECT sem separar níveis de agregação.
- HAVING com subconsulta que referencia a própria consulta agregada sem CTE.

- d) subconsulta correlacionada no SELECT para recalculer o total de cada município em cada linha.
- e) GROUP BY e aplicar WHERE comparando diretamente com AVG(total) no mesmo nível.

Comentário:

- a) **Correta.** A cláusula WITH cria uma Common Table Expression (CTE), que é um bloco de consulta nomeado, permitindo calcular o total de ICMS por município uma vez e reutilizar esse resultado na consulta final, comparando-o com a média estadual.
- b) **Errada.** Colocar SUM e AVG na mesma lista SELECT sem separar níveis de agregação misturaria cálculos de granularidades diferentes (por município e estadual), não resolvendo corretamente o problema.
- c) **Errada.** Usar HAVING com subconsulta sem CTE até pode funcionar em alguns casos, mas não atende ao requisito de legibilidade e reúso do resultado agregado, que é justamente o que a CTE proporciona.
- d) **Errada.** Subconsulta correlacionada recalcularia o total para cada linha, sendo ineficiente e contrário à ideia de reúso de um resultado agregado já calculado.
- e) **Errada.** O WHERE não pode ser usado com funções de agregação como AVG diretamente, pois ele filtra antes da agregação; para isso seria necessário HAVING, e ainda assim não haveria o reúso proporcionado pela CTE.

Gabarito: A

(CEBRASPE (CESPE)/AFRE RJ/SEFAZ RJ/2025) A principal função da cláusula WITH do SQL é

- a) agrupar registros com base em valores comuns em uma ou mais colunas.
- b) combinar registros de duas ou mais tabelas com base em colunas relacionadas.
- c) filtrar registros com base em condições específicas.
- d) ordenar o resultado de uma consulta por uma ou mais colunas.
- e) definir blocos de consulta nomeados que podem ser referenciados múltiplas vezes.

Comentários:

- a) **Errada.** Agrupar registros com base em valores comuns é função da cláusula GROUP BY, não do WITH.
- b) **Errada.** Combinar registros de duas ou mais tabelas com base em colunas relacionadas é função do JOIN, não do WITH.
- c) **Errada.** Filtrar registros com base em condições específicas é função do WHERE, não do WITH.
- d) **Errada.** Ordenar o resultado de uma consulta é função do ORDER BY, não do WITH.

e) **Correta.** A cláusula WITH materializa as Common Table Expressions (CTEs), permitindo dar um nome a uma subconsulta para que esse bloco possa ser referenciado múltiplas vezes dentro da mesma instrução SQL, melhorando a legibilidade e o reuso.

Gabarito: E

Onde uma subconsulta pode aparecer?

Infelizmente a subconsulta pode aparecer em diversos locais dentro de um SELECT e, a depender do local em que aparece, tem um propósito distinto.

Se aparecer no WHERE, como vimos na seção anterior, usamos a tabela retornada pela subconsulta para filtrar os valores da tabela da consulta principal.

De outro modo, se aparecer no FROM, consideramos a tabela formada pela subconsulta para obter os atributos buscados pelo SELECT. Nesse caso, ao invés de buscarmos os dados em uma tabela persistida no banco de dados, usamos a tabela em memória formada pelo subselect.



Quando uma subconsulta aparece no FROM, ela recebe o nome técnico de tabela derivada (derived table).

Por exemplo, a seguir vamos criar uma tabela "Professor_e_Auxiliar" e depois retornar apenas o nome de Professor e Auxiliar.

```
SELECT pa.nome_prof, pa.nome_aux
FROM (
    SELECT P.nome AS nome_prof, A.nome AS nome_aux
    FROM Professor P
    JOIN Auxiliar A ON P.departamento = A.departamento
) AS pa;
```

Podemos ter subconsultas no próprio SELECT para produzir colunas calculadas a partir de outras consultas.

```
SELECT nome,
       idade,
       (SELECT AVG(idade) FROM Professor) AS media_idade_geral
FROM Professor;
```

Finalmente, embora muito pouco cobrado, a consulta aninhada pode aparecer no HAVING também.



(Instituto AOCP/TRF 2/Tecnologia da Informação/2024) Considere a tabela PEDIDO com os atributos pedido_id, cliente_id, data_pedido, e total, e a tabela CLIENTE com os atributos cliente_id, nome e cidade. Um técnico necessita encontrar os clientes que têm pedidos com valor total acima da média dos pedidos de todos os clientes da mesma cidade. Para resolver esse problema, o técnico decide utilizar subqueries. Em relação ao tema, analise as assertivas e assinale a alternativa que aponta as corretas.

- I. Uma subquery pode ser utilizada na cláusula SELECT, no contexto apresentado, para calcular a média dos pedidos por cidade.
 - II. Uma subquery pode ser utilizada na cláusula FROM para criar uma tabela derivada contendo a média dos pedidos por cidade.
 - III. Uma subquery pode ser utilizada na cláusula WHERE para filtrar clientes com pedidos acima da média dos pedidos da mesma cidade.
 - IV. Uma subquery pode ser utilizada na cláusula GROUP BY para agrupar os pedidos por cidade e calcular a média dos pedidos.
- a) Apenas I e II.
 - b) Apenas II e III.
 - c) Apenas II e IV.
 - d) Apenas III e IV.
 - e) I, II, III e IV.

Comentários:

I. Errada. Embora seja tecnicamente possível colocar uma subquery dentro do SELECT para gerar uma coluna calculada, no contexto apresentado isso não funcionaria adequadamente para calcular a média por cidade considerando o cliente avaliado, pois a subquery no SELECT produz um valor por linha sem o agrupamento por cidade exigido pelo problema. Ou seja, seria necessário utilizar uma cláusula GROUP BY também.

II. Correta. Uma subquery pode aparecer no FROM para criar uma tabela derivada (uma tabela em memória formada pelo subselect), e essa tabela poderia conter a média dos pedidos por cidade, sendo então usada pela consulta externa.

III. Correta. Esse é o uso clássico de subquery no WHERE: usamos a tabela retornada pela subconsulta para filtrar os valores da consulta principal, neste caso filtrando os clientes cujo total de pedidos esteja acima da média da cidade.

IV. Errada. Subqueries não são utilizadas dentro da cláusula GROUP BY para agrupar registros. O GROUP BY serve para agrupar com base em colunas, e os locais onde uma subconsulta aparece são WHERE, FROM, SELECT e HAVING, conforme estudado.

Gabarito: B

ALL e ANY

Com os operadores ALL e ANY conseguimos comparar cada valor da tabela da consulta principal contra um conjunto inteiro de valores retornados por uma subconsulta.

A diferença entre os dois é que o **ALL exige que a comparação seja válida para todos valores do conjunto**, enquanto que para o ANY basta ser válido para pelo menos um deles.



ANY e ALL só funcionam com operadores de comparação padrão: igual (=), diferente (<> ou !=), maior (>), maior ou igual (>=), menor (<) e menor ou igual (<=).

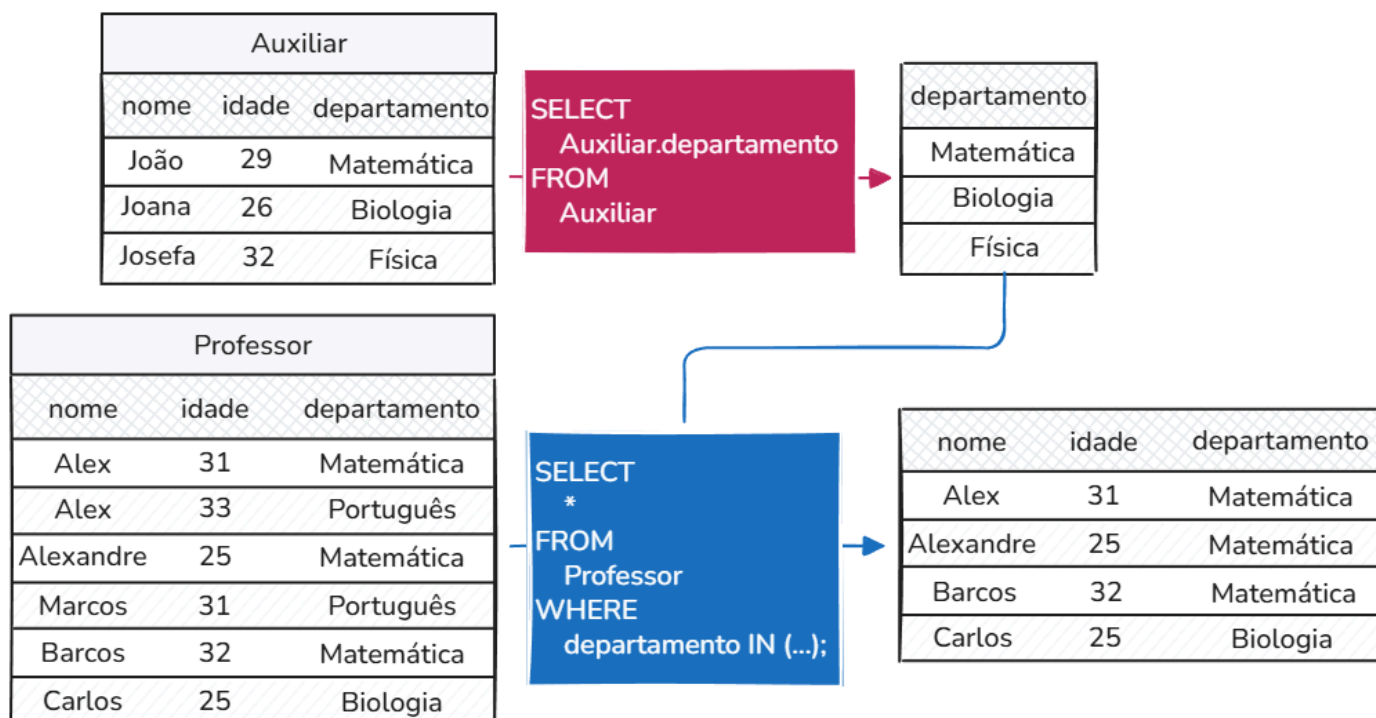
Ou seja, não dá para usar com operadores especiais como LIKE ou IN, tudo bem?

Por exemplo, vamos supor que queremos saber *“qual o nome dos professores que possuem idade maior que a idade de cada um dos auxiliares?”*.

Para conseguirmos isso, basta compararmos, no WHERE, se a idade contida na tabela Professor é maior que todas (ALL) idades contidas na tabela Auxiliar.

```
SELECT nome
FROM Professor
WHERE
    idade > ALL (
        SELECT idade
        FROM Auxiliar
    );
```

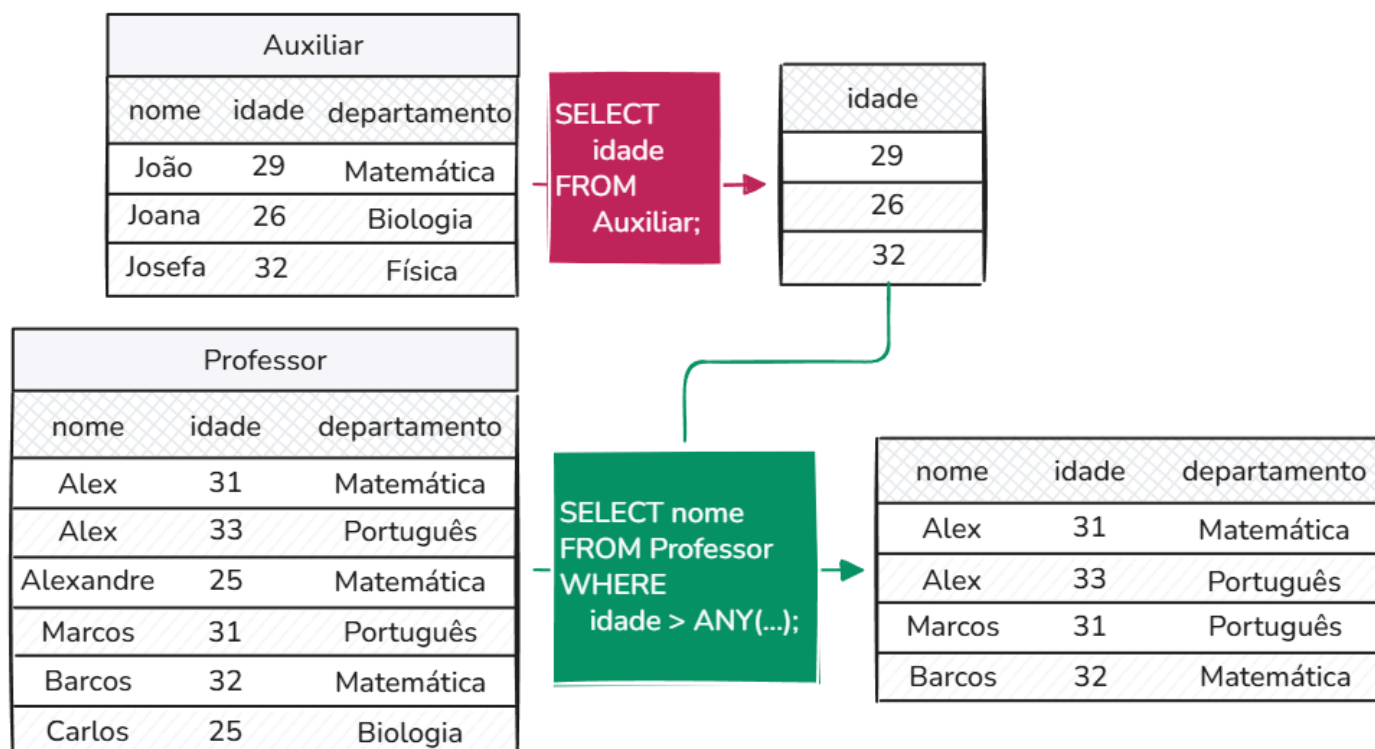
Percebam que, para um professor conseguir passar nesse filtro, é necessário que sua idade seja maior que todas as idades de todos auxiliares.



Do mesmo modo, o operador **ANY** também compara cada valor da tabela da consulta principal contra cada um dos valores retornados pela consulta aninhada, porém com uma lógica mais permissiva: **basta pelo menos um dos valores do conjunto retornado pela subconsulta satisfazer a condição.**

Por exemplo, aqui conseguimos descobrir “qual o nome dos professores que são mais velhos que pelo menos um dos auxiliares?”.

```
SELECT nome
FROM Professor
WHERE
    idade > ANY (
        SELECT idade
        FROM Auxiliar
    );
```



Vale saber que o operador ANY com "=" é logicamente idêntico ao operador IN. Isso porque, com "IN (a, b, c)" procuramos se determinado valor é igual a "a" ou "b" ou "c", que é exatamente o que "= ANY (a, b, c)" faz ("igual a pelo menos um dos valores").

Por isso, as duas consultas a seguir equivalentes:

```
SELECT nome
FROM Professor
WHERE
    idade = ANY (
        SELECT idade
        FROM Auxiliar
    );
```

```
SELECT nome
FROM Professor
WHERE
    idade IN (
        SELECT idade
        FROM Auxiliar
    );
```



O **SOME** é um sinônimo do **ANY** no SQL, ou seja, ao invés de escrever ANY, podemos escrever SOME.



(QUADRIX/CREFITO 4/Tecnologia da Informação/2021) Ao se substituir o operador ANY pelo operador SOME, a consulta produzirá o mesmo resultado, já que os operadores são equivalentes.

Comentários:

O SOME é apenas um sinônimo do ANY no SQL. Ou seja, escrever um ou outro dá exatamente no mesmo: a consulta funciona da mesma forma e retorna o mesmo resultado. É só uma questão de nomenclatura, a linguagem aceita as duas palavras para a mesma operação.

Gabarito: Certa

(Instituto Verbena/IF GOIANO/Tecnologia da Informação/2019) Em um banco de dados de empregados, no qual existem vários empregados com salários distintos entre si, o comando SQL que retorna os empregados cujo salário é inferior ao maior salário existente é:

- a) `select * from empregado where salario > all (select salario from empregado)`
- b) `select * from empregado where salario < all (select salario from empregado)`
- c) `select * from empregado where salario > any (select salario from empregado)`
- d) `select * from empregado where salario < any (select salario from empregado)`

Comentários:

a) Errada. O operador `> ALL` exige que o salário seja maior que todos os salários, ou seja, retornaria apenas o maior salário, mas mesmo assim não funcionaria, pois nenhum salário é maior que ele mesmo.

b) Errada. O operador `< ALL` exige que o salário seja menor que todos os salários, inclusive menor que ele mesmo, o que é impossível, retornando um conjunto vazio.

c) Errada. O operador `> ANY` retorna empregados cujo salário é maior que pelo menos um salário, ou seja, retornaria todos exceto o de menor salário, e não o que se pede.

d) Correta. O operador `< ANY` retorna empregados cujo salário é menor que pelo menos um salário existente, ou seja, todos que não possuem o maior salário, que é exatamente o que a questão pede.

Gabarito: D

(DIRPS UFU - Ana TI (UFU)/UFU/Área 1/2017)

- `Create table Jogadores(numj integer not null, nome varchar(20), ano_n integer, primary key(numj))`
- `create table Lista_torneios(numt integer not null, nome varchar(20) not null, primary key (numt))`

– create table Torneios(numt integer not null, numj integer not null, ano integer not null, premio double not null, primary key(numt, ano), foreign key(numt) references Lista_torneios(numt), foreign key(numj) references Jogadores(numj))

Com o objetivo de retornar o nome e ano de nascimento dos jogadores mais velhos de todos os outros jogadores do grupo, a consulta SQL correta é

- a) select nome, ano_n from jogadores where ano_n <= any (select ano_n from jogadores)
- b) select nome, ano_n from jogadores where ano_n <= all (select ano_n from jogadores)
- c) select nome, ano_n from jogadores where ano_n <= exists (select ano_n from jogadores)
- d) select nome, ano_n from jogadores where ano_n <= union (select ano_n from jogadores)

Comentários:

a) **Errada.** A alternativa traz um texto fora de contexto, sem ser um comando SQL válido para o problema.

b) **Correta.** O operador ALL garante que o ano_n do jogador seja menor ou igual a todos os anos de nascimento dos outros jogadores, retornando assim os mais velhos do grupo.

c) **Errada.** O EXISTS é um operador de teste binário de existência, usado para verificar se uma subconsulta retorna ao menos uma linha. Ele não é usado com operadores de comparação como <=, então essa sintaxe é inválida.

d) **Errada.** UNION não é um operador de comparação; ele serve para combinar resultados de consultas, não para comparar valores no WHERE.

Gabarito: B

Subconsulta Correlacionada

Vimos até agora subconsultas “comuns”, situação em que a consulta aninhada é executada uma única vez e o seu resultado (uma tabela) é usado pela consulta externa.

Entretanto, em uma **subconsultas correlacionada**, a **consulta aninhada precisa ser reexecutada para cada linha da consulta externa**. Ou seja, retorna várias análises, para ser utilizada paulatinamente pela consulta de fora.

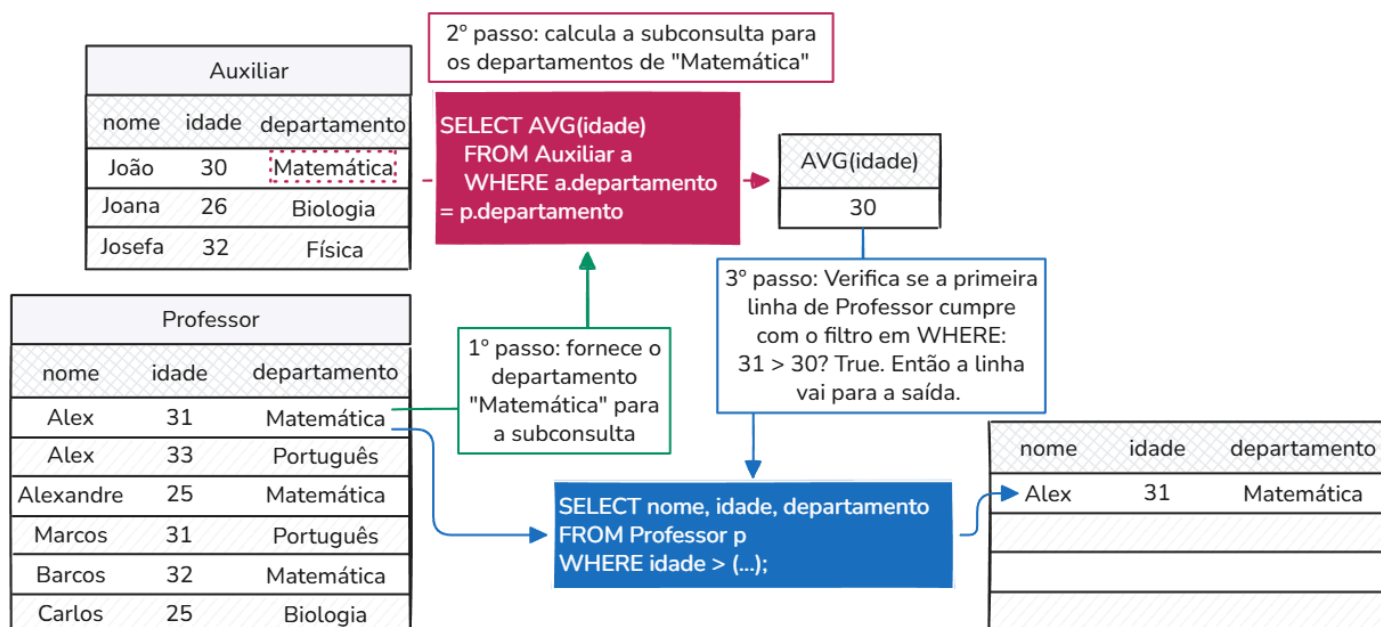
Embora seja um pouco confuso entender a execução da subconsulta correlacionada, identificá-la é bem mais simples: a condição após o WHERE da consulta interna referencia alguma das tabelas da consulta externa.

A seguir, vejamos uma subconsulta correlacionada, para passarmos a estudá-la na sequência.

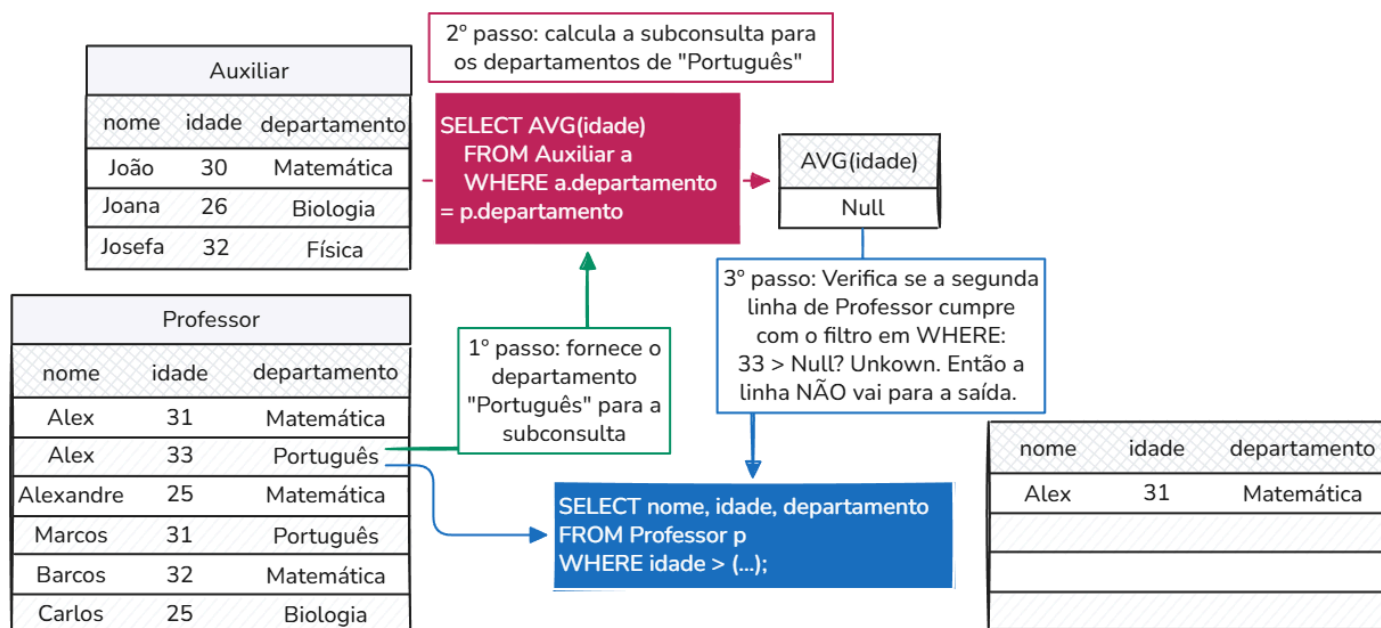
```
SELECT nome, idade, departamento
FROM Professor p
WHERE idade > (
    SELECT AVG(idade)
    FROM Auxiliar a
    WHERE a.departamento = p.departamento -- é subconsulta correlacionada porque
a tabela "p" (alias de Professor) vem de fora da subconsulta!
);
```

Percebam então que, por se tratar de uma subconsulta correlacionada, precisamos avaliar a subconsulta para cada linha a tabela Professor. Isso porque essa subconsulta não consegue “rodar sozinha”, pois precisa de um valor que só a consulta externa fornece: o departamento de cada professor (p.departamento).

Pois bem, considerando então nossa tabela Professor, passemos a avaliar o resultado do SQL para cada uma de suas linhas.



Verificada a análise para a primeira linha, reinicia-se o processo para a segunda.



Ao final do processo, apenas a linha de "Alex, 31 , Matemática" e "Barcos, 32, Matemática" serão selecionadas para a resposta final



(FGV/TJDFT/Apoio Especializado/Análise de Dados/2022)

Tabela TURFE

páreo cavalo tempo

1	Estupenda	52,2
1	Mandrake	52,3
1	Capricho	53,4
2	Estupenda	51,0
2	Mandrake	51,1
2	Muddy River	52,1
2	Uramak	52,3
2	Escorial	52,5
2	Capricho	56,1
3	Muddy River	52,1
3	Estupenda	52,3
3	Uramak	53,0

3 Escorial 55,1

Considere um banco de dados relacional que contém uma única tabela, TURFE, cuja estrutura é exibida com sua instância, a seguir.

Com relação à tabela TURFE, descrita anteriormente, analise o comando SQL a seguir.

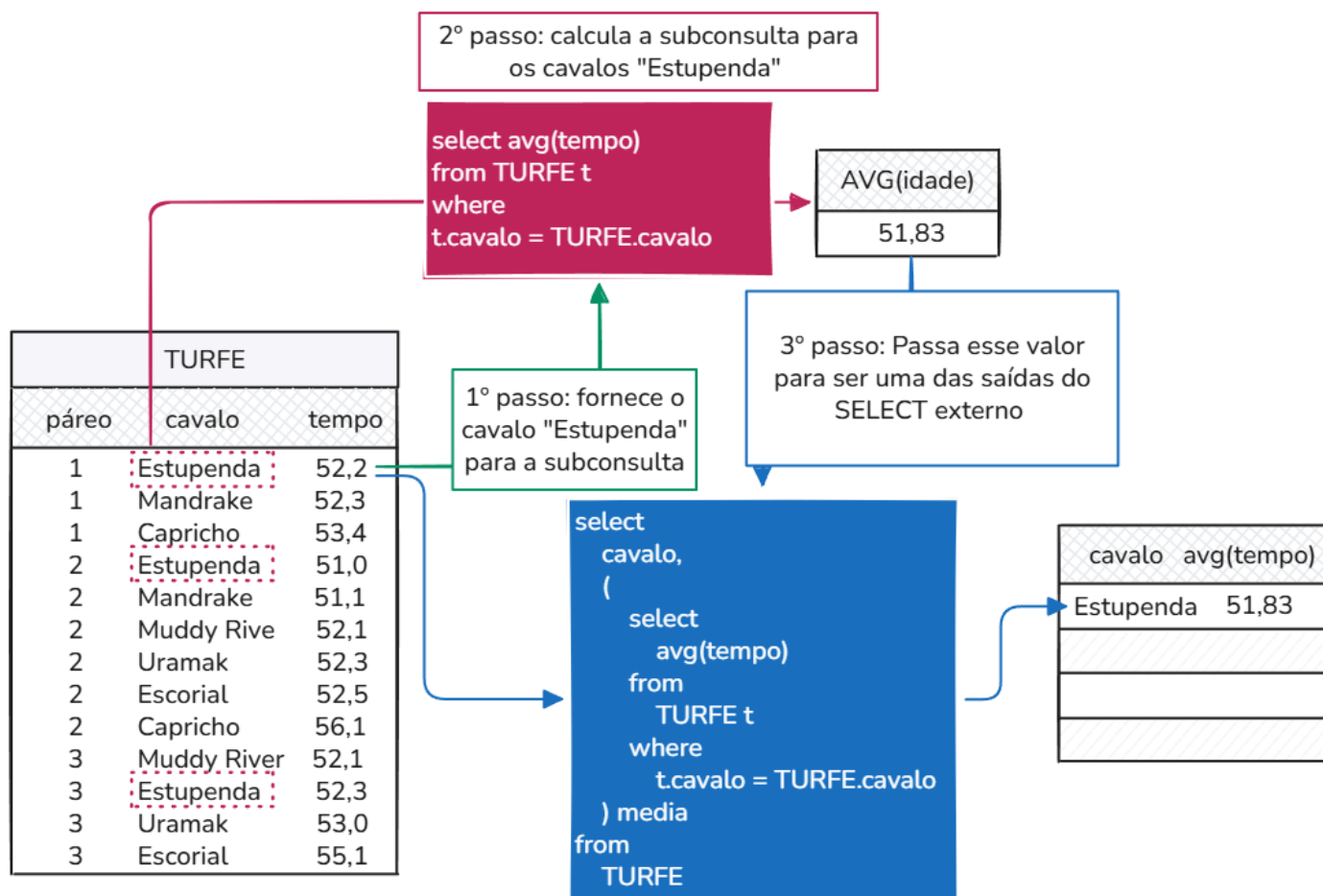
```
select  
cavalo,  
(select avg(tempo)  
from TURFE t  
where t.cavalo = TURFE.cavalo) media  
from TURFE
```

O número de linhas do resultado produzido pela execução do comando acima, excetuada a linha de títulos, é:

- a) 3;
- b) 6;
- c) 9;
- d) 13;
- e) 16.

Comentários:

Percebam que temos uma consulta correlacionada, ou seja, precisamos verificar o SQL linha por linha da tabela original.



Agora vejam só que interessante, esse SQL nada mais faz do que pegar cada linha da tabela TURFE e passar o nome do cavalo e a média de tempo daquele tipo de cavalo para a saída (não há WHERE no SELECT externo para filtrar nada).

Ou seja, a subconsulta relacionada rodará para todas as linha, dando uma saída para cada uma. Assim, a tabela de saída terá exatamente o mesmo número de linhas que a tabela TURFE

Gabarito: D

EXISTS

O EXISTS é um operador usado dentro de uma cláusula WHERE com, regra geral, subconsultas correlacionadas para verificar se essa subconsulta retorna uma tabela de ao menos uma linha.

O interessante aqui é que o EXISTS simplesmente não se importa com quais linhas a consulta aninhada de fato retorna, nem com quantas são, nem com os valores dos seus atributos que há nelas.

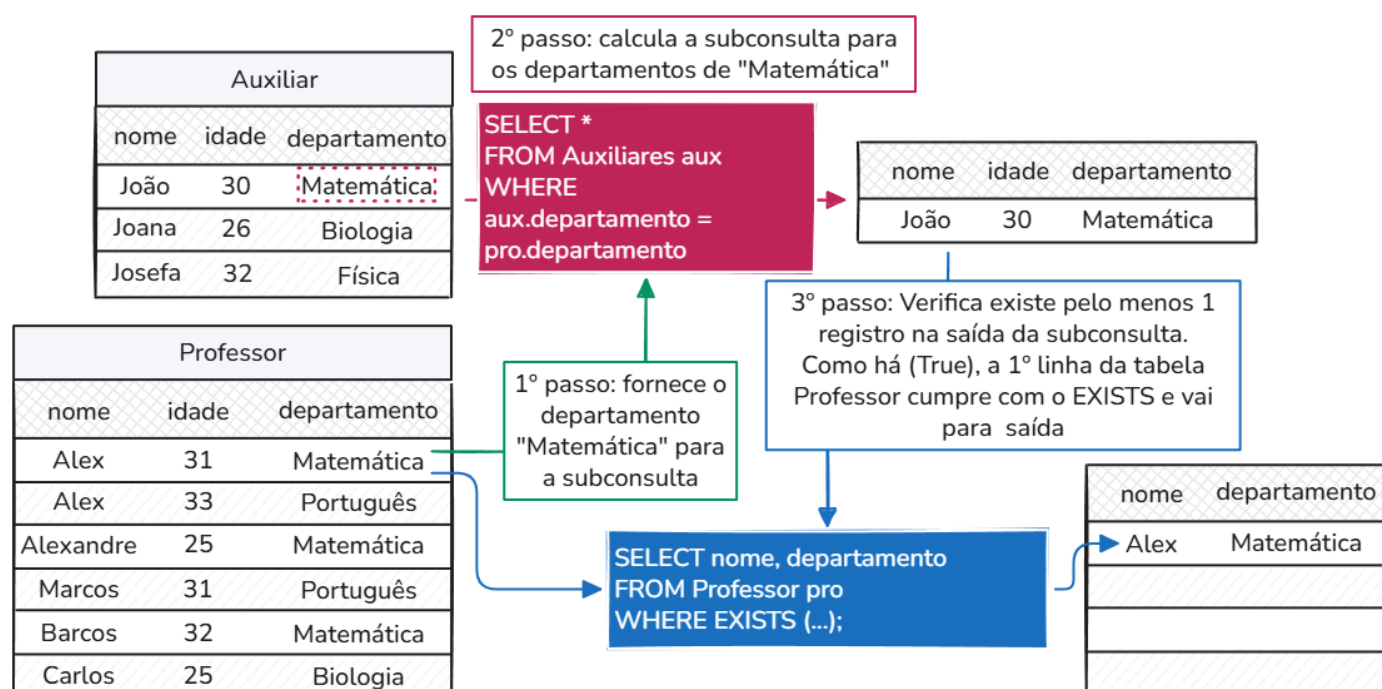
Ou seja, o EXISTS só avalia uma coisa: *existe ao menos uma linha que satisfaça as condições da consulta aninhada?*

Se sim (a subconsulta retorna pelo menos uma linha), o EXISTS avalia para TRUE, e a linha é considerada no SELECT "principal", e para FALSE caso contrário, não sendo considerada no SELECT principal.

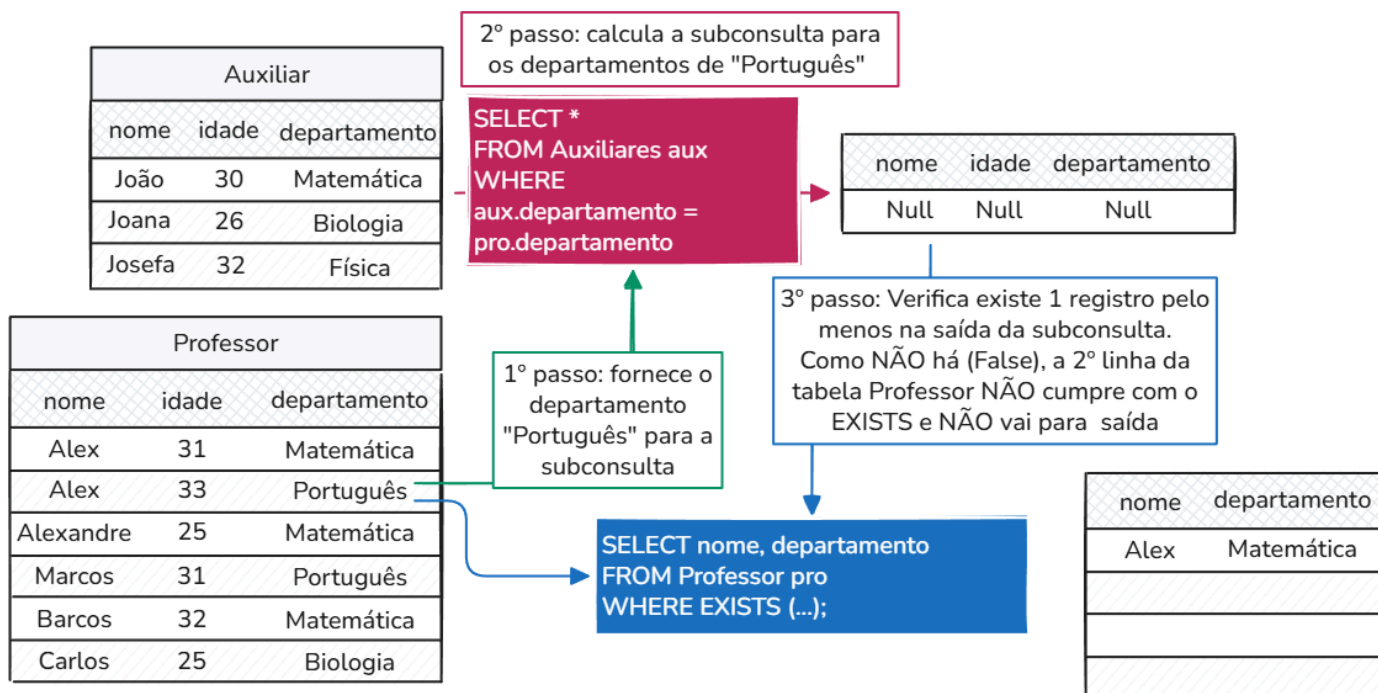
Em resumo, se trata de um teste binário de existência: "sim, existe, utilizar" ou "não, não existe" para cada linha da tabela.

Pois bem, veremos agora um SQL que visa listar os professores que têm pelo menos um auxiliar no mesmo departamento usando EXISTS.

```
SELECT nome, departamento
FROM Professor pro
WHERE EXISTS (
    SELECT *
    FROM Auxiliar aux
    WHERE aux.departamento = pro.departamento -- é subconsulta correlacionada
    porque a tabela "pro" (alias de Professor) vem de fora da subconsulta!
);
```



Agora vejamos essa análise para a segunda linha de Professor.



No final das contas, só selecionamos as linhas de Professor que de fato possuem um "departamento" que também existe no "departamento" da tabela Auxiliar.

nome	departamento
Alex	Matemática
Alexandre	Matemática
Barcos	Matemática
Carlos	Biologia



(FGV/AFFC (CGU)/CGU/2022) ATENÇÃO! Para resolver a questão, considere as tabelas cliente e visita, conforme as instâncias exibidas a seguir.

cliente

codigo

310

312

314

316

Visita

codigo	data
310	01/03/2022
312	01/03/2022
312	10/03/2022
310	11/03/2022
314	11/03/2022
312	11/03/2022
312	11/03/2022
314	05/04/2022
312	07/04/2022

Com relação às tabelas cliente e visita, descritas anteriormente, o comando SQL que deleta corretamente os clientes que nunca visitaram o site é:

a)

```
delete from cliente
```

```
where not exists
```

```
(select * from visita v
```

```
where v.codigo in cliente.codigo)
```

b)

```
delete from cliente
```

```
where not exists
```

```
(select * from visita v
```

```
where v.codigo <> cliente.codigo)
```

c)

```
delete from cliente
```

```
where codigo in (select codigo from visita)
```

d)

```
delete from cliente
```

```
where codigo <> ALL (select codigo from visita)
```

e)

```
delete from cliente
```

```
where (select count(*) from visita v
```

```
where v.codigo <> cliente.codigo) > 0
```

Comentários:

a) Errada. O operador IN é usado para comparar com uma lista/conjunto, e não com um valor único como cliente.codigo. A sintaxe está incorreta, pois o certo seria v.codigo = cliente.codigo.

b) Errada. A condição v.codigo <> cliente.codigo retornaria as visitas feitas por outros clientes, e não as visitas do cliente avaliado. Assim, o NOT EXISTS quase nunca seria verdadeiro, e os clientes que realmente nunca visitaram não seriam corretamente identificados.

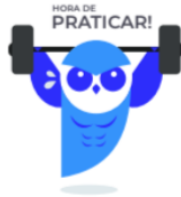
c) Errada. Esse comando faria justamente o oposto do que se pede: deletaria os clientes que estão presentes na tabela visita, ou seja, os que visitaram, mantendo os que nunca visitaram.

d) Correta. O operador <> ALL exige que o código do cliente seja diferente de todos os códigos retornados pela subconsulta (todos os códigos da tabela visita). Ou seja, só passam no filtro os clientes cujo código não aparece em nenhuma visita, que é exatamente quem nunca visitou o site. No caso, apenas o cliente 316 seria deletado.

e) Errada. A condição v.codigo <> cliente.codigo conta as visitas feitas por outros clientes. Como existem várias visitas de outros códigos, esse count seria maior que zero para praticamente todos os clientes, deletando até quem visitou o site.

Gabarito: D

QUESTÕES COMENTADAS - FGV



1. (FGV/AAD (TCE,RR)/TCE RR/Tecnologia da Informação/Banco de Dados/2025) O código DDL abaixo cria três tabelas em um banco de dados relacional:

```
CREATE TABLE Velejadores (  
    vid INTEGER PRIMARY KEY,  
    nome VARCHAR(100),  
    idade INTEGER  
);  
CREATE TABLE Barcos (  
    bid INTEGER PRIMARY KEY,  
    nome VARCHAR(50),  
    modelo VARCHAR(50)  
);  
CREATE TABLE Reservas (  
    vid INTEGER,  
    bid INTEGER,  
    data DATE,  
    PRIMARY KEY (vid, bid),  
    FOREIGN KEY (vid) REFERENCES Velejadores(vid),  
    FOREIGN KEY (bid) REFERENCES Barcos(bid)  
);
```

Considerando as tabelas acima, a seguinte consulta foi realizada:

```
SELECT V.nome FROM Velejadores V  
WHERE NOT EXISTS (  
    (SELECT B.bid FROM Barcos B)  
    EXCEPT  
    (SELECT R.bid FROM Reservas R  
    WHERE R.vid = V.vid)  
);
```

O retorno da consulta foram os nomes

- a) dos velejadores que possuem reservas para todos os barcos.
- b) dos barcos que não possuem reservas.
- c) dos velejadores que não possuem reservas.
- d) dos velejadores que possuem reserva para mais de um barco.
- e) dos barcos que possuem reservas feitas por todos os velejadores.

Comentários:

a) **Correta.** A subconsulta interna pega todos os bid da tabela Barcos e subtrai (EXCEPT) os bid que o velejador atual já reservou. Se o resultado dessa subtração for vazio, significa que não sobrou nenhum barco sem reserva para esse velejador, ou seja, ele reservou todos. Como temos NOT EXISTS, só passam no filtro os velejadores em que essa diferença é vazia, ou seja, aqueles que possuem reserva para todos os barcos.

b) **Errada.** A consulta principal retorna V.nome da tabela Velejadores, então não pode listar nomes de barcos.

c) **Errada.** Velejadores sem nenhuma reserva teriam a subconsulta interna retornando todos os barcos (diferença não vazia), então o NOT EXISTS seria falso e eles não apareceriam.

d) **Errada.** A lógica do NOT EXISTS com EXCEPT exige que o velejador tenha reservado todos os barcos, e não apenas mais de um.

e) **Errada.** A consulta retorna nomes de velejadores (V.nome), e não de barcos.

Gabarito: A

2. (FGV/ACP (CGE PB)/CGE PB/Auditoria de Tecnologia da Informação/2024) Tabelas Relacionais T1 e T2

Considere as tabelas relacionais T1 e T2, de acordo com as colunas e as instâncias abaixo descritas. Na execução dos comandos SQL, assuma que o mecanismo do banco de dados considera valores NULL como valores desconhecidos (unknown).

Tabela T1

P	N
1	10
2	40
3	20
4	10

Tabela T2

E	Q	P
1	10	2

2	40	3
3	20	8
4	10	NULL

À luz das tabelas T1 e T2, anteriormente apresentadas, analise o comando SQL exibido a seguir.

```
select case when
    exists (select * from T2
            where T2.E = 2
            and T2.P = 3
            and exists
                (select * from T1
                 where T1.P in (2,3,4)
                 and T2.E in (2,3)))
    then 1 else 0 end flag
```

Sobre uma eventual execução desse script, é correto afirmar que:

- a) haveria um erro de sintaxe na última linha, no termo "flag";
- b) seria produzido um resultado contendo somente a linha de títulos;
- c) seria produzido um resultado contendo, além da linha de títulos, uma linha com uma coluna com o valor 0;
- d) seria produzido um resultado contendo, além da linha de títulos, uma linha com uma coluna com o valor 1;
- e) haveria um erro de sintaxe na segunda linha, pois não é permitido o uso da cláusula "exists" logo após o termo "when".

Comentários:

- a) Errada.** O termo "flag" ao final do comando é apenas um apelido (alias) dado à coluna de saída do CASE, sendo uma sintaxe perfeitamente válida.
- b) Errada.** O CASE sempre produz um valor de saída (1 ou 0), então haverá uma linha de resultado além da linha de títulos.
- c) Errada.** O valor produzido não será 0, pois a subconsulta com EXISTS retorna pelo menos uma linha, fazendo o CASE avaliar para 1. Vejamos: na tabela T2 existe a linha com E=2 e P=3 (segunda linha de T2), satisfazendo a primeira condição. Em seguida, a subconsulta interna verifica se existe alguma linha em T1 com P em (2,3,4), e de fato existem (linhas com P=2, P=3 e P=4), além de T2.E estar em (2,3), o que também é verdade pois E=2. Logo, o EXISTS externo retorna verdadeiro.

d) **Correta.** Como o EXISTS externo encontra a linha de T2 com E=2 e P=3, e o EXISTS interno também encontra linhas em T1 com P em (2,3,4) enquanto T2.E (que vale 2) está em (2,3), todas as condições são satisfeitas. Assim, o CASE retorna 1, e a saída terá a linha de títulos mais uma linha com o valor 1 na coluna flag.

e) **Errada.** É totalmente válido usar EXISTS após o WHEN dentro de um CASE, já que o EXISTS é um teste binário que retorna verdadeiro ou falso, exatamente o que o WHEN espera avaliar.

Gabarito: D

3. (FGV/AJ (TJ AP)/TJ AP/Apoio Especializado/Tecnologia da Informação Banco de Dados/2024) Quando referenciadas, considere as tabelas relacionais Competidor e Disputa, cujas estruturas e instâncias são descritas abaixo. Todas as colunas são definidas como strings.

A tabela Disputa contém as disputas realizadas entre competidores que aparecem na tabela Competidor. Em cada disputa há dois competidores, um com camisa azul e outro com camisa verde.

Competidor

Nome
A
B
c
D
E
F

Disputa

Azul	Verde
A	B
C	A
B	A
C	E
F	A
F	D

Considerando as tabelas Competidor e Disputa, descritas anteriormente, analise os três comandos SQL a seguir.

```
I. select C.nome  
from Competidor C  
where exists (select * from Disputa D  
              where D.azul = C.nome)  
and exists (select * from Disputa D
```

where D.verde = C.nome)

```
II. select C.nome
from Competidor C
where exists (select * from Disputa D
              where D.azul = C.nome
                  or D.verde = C.nome)
```

```
III. select C.nome
from Competidor C
where (select count(*) from Disputa D
       where D.azul = C.nome)
      + (select count(*) from Disputa D
         where D.verde = C.nome) > 1
```

O resultado

Nome

A

B

é obtido somente no(s) comando(s):

- a) I;
- b) II;
- c) III;
- d) I e II;
- e) II e III.

Comentários:

I. Correta. Esse comando usa dois EXISTS ligados por AND, ou seja, só retorna o competidor que aparece como azul em alguma disputa E também aparece como verde em alguma disputa. Os únicos que satisfazem essa condição são A e B, exatamente o resultado pedido.

II. Errada. Aqui o EXISTS verifica se o competidor aparece como azul OU como verde, ou seja, basta aparecer em qualquer cor. Como todos os competidores (A, B, C, D, E, F) aparecem em pelo menos uma disputa, o resultado traria todos, e não apenas A e B.

III. Errada. Esse comando soma a quantidade de vezes que o nome aparece como azul mais a quantidade de vezes que aparece como verde, e exige que essa soma seja maior que 1. Pelo

cálculo, A (4), B (2), C (2) e F (2) satisfazem a condição, retornando A, B, C e F, e não apenas A e B.

Gabarito: A

4. (FGV/AJ (TJ AP)/TJ AP/Apoio Especializado/Tecnologia da Informação Banco de Dados/2024) Quando referenciadas, considere as tabelas relacionais Competidor e Disputa, cujas estruturas e instâncias são descritas abaixo. Todas as colunas são definidas como strings.

A tabela Disputa contém as disputas realizadas entre competidores que aparecem na tabela Competidor. Em cada disputa há dois competidores, um com camisa azul e outro com camisa verde.

Competidor

Nome
A
B
C
D
E
F

Disputa

Azul	Verde
A	B
C	A
B	A
C	E
F	A
F	D

Considerando as tabelas Competidor e Disputa, descritas anteriormente, analise o comando SQL a seguir.

```
select nome  
from Competidor C  
where not exists (select *  
from Disputa D  
where D.azul = C.nome)  
or not exists (select *  
from Disputa D
```

where D.verde = C.nome)

A execução do comando acima produziria somente a lista completa dos competidores que:

- a) participaram em disputas com camisas das duas cores;
- b) nunca participaram em disputas com camisa verde;
- c) nunca participaram em disputas com camisa azul;
- d) nas disputas em que participaram, usaram sempre camisas da mesma cor;
- e) nunca participaram em disputas.

Comentários:

a) Errada. Quem participou com as duas cores satisfaz ambos os EXISTS, então as duas condições NOT EXISTS dão falso, e ele não entra no resultado.

b) Errada. Esse caso é coberto pelo segundo NOT EXISTS, mas não é o conjunto completo retornado, pois quem nunca usou azul também entra.

c) Errada. Mesma lógica do item anterior: é apenas parte do que o comando retorna, não o resultado completo.

d) Correta. O OR entre os dois NOT EXISTS seleciona quem nunca usou azul ou nunca usou verde, que equivale a quem, nas disputas em que participou, sempre usou a mesma cor (ou nunca participou).

e) Errada. Quem nunca participou está incluído, mas o resultado também traz quem participou usando sempre a mesma cor, então essa não é a lista completa.

Gabarito: D

5. (FGV/AJ (TJ AP)/TJ AP/Apoio Especializado/Tecnologia da Informação Desenvolvimento de Sistemas/2024) Quando referenciadas, considere as tabelas relacionais Competidor e Disputa, cujas estruturas e instâncias são descritas abaixo. Todas as colunas são definidas como strings. A tabela Disputa contém as disputas realizadas entre competidores que aparecem na tabela Competidor. Em cada disputa há dois competidores, um com camisa azul e outro com camisa verde.

Competidor

Nome
A
B
C
D
E
F

Disputa

Azul	Verde
A	B
C	A
B	A
C	E
F	A
F	D

Considerando as tabelas Competidor e Disputa, descritas anteriormente, analise o comando SQL abaixo.

```
select c1.nome, c2.nome
from Competidor c1, Competidor c2
where not exists
      (select * from Disputa d
       where d.azul = c2.nome
            and d.verde = c1.nome)
order by 1,2
```

Excetuados os títulos, o número de linhas produzidas pela execução desse comando é:

- a) 0;
- b) 6;
- c) 12;
- d) 30;
- e) 36.

Comentários:

Gabarito: D

6. (FGV/Ana Leg (ALEP)/ALEP/Desenvolvedor de Sistemas/2024) Considere o esquema relacional a seguir, implementado em SQL.

```
create table recurso
(
    id integer primary key,
```

```
        nome varchar(20) not null,  
        valor real  
    );  
create table projeto  
(  
    id integer primary key,  
    nome varchar(20) not null,  
    verba real  
);  
create table alocao  
(  
    id_recurso integer,  
    id_projeto integer,  
    primary key(id_recurso,id_projeto),  
    foreign key(id_recurso) references recurso,  
    foreign key(id_projeto) references projeto  
);
```

Assinale a opção que apresenta a consulta que gera como resultado de execução uma lista com o nome dos recursos alocados em todos os projetos cadastrados.

- a) select r.nome from recurso r
- b) select r.nome from recurso r where not exists (select 1 from alocao a where a.id_recurso=r.id)
- c) select r.nome from recurso r where r.valor>(select avg(valor) from recurso)
- d) select r.nome from recurso r where not exists (select 1 from projeto p where not exists (select 0 from alocao a where a.id_recurso=r.id and a.id_projeto=p.id))
- e) select r.nome from recurso r where exists (select 1 from alocao a where a.id_recurso=r.id)

Comentários:

- a) Errada.** Esse comando retorna todos os recursos cadastrados, sem qualquer filtro relacionado a alocações em projetos, não atendendo ao que se pede.
- b) Errada.** Aqui o NOT EXISTS retorna os recursos que não possuem nenhuma alocação, ou seja, os recursos que não estão alocados em projeto algum, o oposto do que a questão deseja.
- c) Errada.** Essa consulta filtra recursos cujo valor é maior que a média dos valores de recursos, o que nada tem a ver com alocações em projetos.
- d) Correta.** Aqui temos duas subconsultas correlacionadas aninhadas com NOT EXISTS. A lógica é: selecionar o recurso para o qual não exista um projeto que não esteja alocado a ele. Em outras

palavras, se para um recurso não há nenhum projeto sem alocação correspondente, então esse recurso está alocado em todos os projetos cadastrados, que é exatamente o que se pede.

e) Errada. Esse comando retorna os recursos que possuem ao menos uma alocação, ou seja, alocados em pelo menos um projeto, e não em todos os projetos cadastrados.

Gabarito: D

7. (FGV/Ana Leg (ALETO)/ALETO/Análise de Suporte em Informática/2024) Considere o seguinte esquema de um banco de dados relacional, expresso em linguagem SQL:

```
create table editora
(
    cod_editora integer primary key,
    nome_editora varchar(30) not null,
    telefone char(11)
);
create table livro
(
    cod_livro integer primary key,
    num_isbn char(10) unique not null,
    titulo varchar(20) not null,
    edicao integer default 1 not null,
    cod_editora integer references editora
);
```

Após criadas, os comandos de inserção a seguir foram executados:

```
insert into editora values (1,'Ed1','2222-3333'), (2,'Ed2',NULL);
insert into livro values (1,'111','Livro1',2,NULL), (2,'222','Livro2',2,1), (5,'555','Livro5',1,1);
insert into livro(cod_livro,num_isbn,titulo,cod_editora) values (3,'333','Livro3',2),
(4,'444','Livro4',2);
```

A consulta `select * from livro where exists (select distinct * from editora) and edicao%2 <> 0` produz como resultado a seguinte quantidade de linhas:

- a) 0.
- b) 1.
- c) 2.
- d) 3.
- e) 4.

Comentários:

- a) **Errada.** Existem livros com edição ímpar e o EXISTS é verdadeiro.
- b) **Errada.** Não é apenas uma linha; três linhas satisfazem as condições.
- c) **Errada.** São três livros com edição ímpar, não dois.
- d) **Correta.** Livro5 (edição 1), Livro3 (edição default 1) e Livro4 (edição default 1) somam três linhas no resultado.
- e) **Errada.** Livro1 e Livro2 têm edição 2 (par), portanto não entram no resultado.

Gabarito: D

8. (FGV/Tec NS (TJ MS)/TJ MS/Analista de Sistemas Computacionais/Analista de Banco de Dados/2024) Considere as seguintes tabelas, com suas respectivas colunas, que se relacionam pela coluna FornecedorID:

Produto (ProdutoID, ProdutoNome, FornecedorID)

Fornecedor (FornecedorID, FornecedorNome)

Para retornar a lista, sem repetições, de Fornecedores que possuem Produtos com preços menores que 20, deve-se executar o comando SQL:

- a) `SELECT FornecedorNome`
`FROM Fornecedor`
`WHERE EXISTS (SELECT ProdutoNome FROM Produto`
`WHERE Produto.FornecedorID =`
`Fornecedor.FornecedorID AND Preco < 20)`
- b) `SELECT COUNT(FornecedorID), FornecedorNome`
`FROM Fornecedor, Produto`
`GROUP BY FornecedorID`
`HAVING COUNT(Preco) < 20`
- c) `SELECT FornecedorNome`
`FROM Fornecedor`
`INNER JOIN Produto ON Produto.FornecedorID =`
`Fornecedor.FornecedorID AND Preco < 20`
- d) `SELECT FornecedorNome FROM Fornecedor`
`UNION`

```
SELECT ProdutoNome FROM Produto  
WHERE Preço < 20)
```

```
e) SELECT FornecedorNome  
FROM Fornecedor  
WHERE FornecedorID = ANY (  
SELECT ProdutoID  
FROM Produto WHERE Preço < 20)
```

Comentários:

a) Correta. O EXISTS é um teste binário de existência que verifica, para cada fornecedor, se existe pelo menos uma linha na tabela Produto com o mesmo FornecedorID e preço menor que 20. Como se trata de uma subconsulta correlacionada (a condição interna referencia Fornecedor.FornecedorID, que vem de fora), o teste é feito linha por linha, retornando apenas os fornecedores que realmente possuem produtos com preço abaixo de 20, sem repetições.

b) Errada. Essa consulta usa COUNT e GROUP BY de forma equivocada, pois HAVING COUNT(Preço) < 20 conta a quantidade de preços e não filtra produtos com preço menor que 20, não atendendo ao que foi pedido.

c) Errada. Embora o INNER JOIN traga fornecedores que possuem produtos com preço menor que 20, ele não elimina repetições. Se um fornecedor tiver vários produtos com preço abaixo de 20, ele aparecerá várias vezes, contrariando o requisito de lista sem repetições.

d) Errada. O UNION combina resultados de duas consultas, mas aqui se está misturando nomes de fornecedores com nomes de produtos, o que não faz sentido para o objetivo da questão. Além disso, a sintaxe contém erro de parêntese.

e) Errada. A comparação FornecedorID = ANY (SELECT ProdutoID ...) está comparando o ID do fornecedor com o ID do produto, que são colunas distintas e sem relação lógica, retornando resultado incorreto.

Gabarito: A

9. (FGV/AFFC (STN)/STN/Tecnologia da Informação/Operação e Infraestrutura/2024)
ATENÇÃO: use a tabela relacional IPCA a seguir para responder à próxima questão.

Tabela IPCA

índice	ano	mês
0,56	2023	12
0,28	2023	11
0,24	2023	10
...	...	...
0,2	2003	7

-0,15	2003	6
...	...	...
2,25	2003	1
2,1	2002	12
3,02	2002	11
...	...	...
0,57	2001	1

A instância da tabela contém os valores do índice IPCA para todos os meses dos anos de 2001 até 2023. Os valores pontilhados representam a continuidade mensal da série. Todas as colunas são numéricas e não aceitam valores nulos.

Considere que um erro de programa na manipulação da tabela IPCA apresentou a inserção de registros duplicados, de modo que algumas linhas da tabela ficaram com valores idênticos em suas colunas.

Analise as alternativas SQL que foram consideradas para a remoção das linhas redundantes.

I. Utilizar o comando delete com a cláusula except one.

II. Utilizar o comando delete incluindo a cláusula exists, o que faria com que a última linha de cada grupo de repetições não seria removida uma vez que não mais existiria uma outra linha idêntica.

III. Introduzir, temporariamente, uma nova coluna na tabela, com valores gerados por meio de um mecanismo de auto increment, ou sequence, e utilizar o uso do comando delete incluindo a cláusula exists.

Em relação a essas alternativas, é correto afirmar que

- a) nenhuma funcionaria.
- b) todas funcionariam.
- c) somente I funcionaria.
- d) somente II funcionaria.
- e) somente III funcionaria.

Comentários:

I. Errada. Não existe a cláusula "except one" no SQL. O EXCEPT até existe em alguns bancos para diferença entre conjuntos, mas não há essa variação "except one" para apagar duplicatas mantendo apenas uma linha.

II. Errada. Como as linhas duplicadas têm valores idênticos em todas as colunas, não há como o EXISTS distinguir uma linha da outra dentro do grupo de repetições. O teste de existência seria verdadeiro para todas as cópias ao mesmo tempo, e não sobraria "a última linha", pois todas seriam removidas (ou nenhuma), já que são indistinguíveis.

III. **Correta.** Esse é o caminho que funciona. Ao adicionar temporariamente uma coluna com auto increment (ou sequence), cada linha passa a ter um identificador único, mesmo que as demais colunas sejam idênticas. Com isso, dá para usar o EXISTS em uma subconsulta correlacionada para verificar se existe outra linha com os mesmos valores nas colunas originais, mas com identificador diferente (por exemplo, menor), e apagar apenas as cópias, mantendo uma. Sem esse identificador único, o banco não consegue diferenciar uma duplicata da outra.

Gabarito: E

10. (FGV/AFFC (STN)/STN/Tecnologia da Informação/Transformação Digital/2024) ATENÇÃO: use a tabela relacional IPCA a seguir para responder à questão.

Tabela IPCA

índice	ano	mês
0,56	2023	12
0,28	2023	11
0,24	2023	10
...	...	...
0,2	2003	7
-0,15	2003	6
...	...	...
2,25	2003	1
2,1	2002	12
3,02	2002	11
...	...	...
0,57	2001	1

A instância da tabela contém os valores do índice IPCA para todos os meses dos anos de 2001 até 2023. Os valores pontilhados representam a continuidade mensal da série. Todas as colunas são numéricas, e não aceitam valores nulos.

No contexto da tabela IPCA apresentada, assinale o comando SQL que produz, corretamente, para cada ano, o mês com o maior índice, ou meses, pois pode haver empate entre os índices de dois ou mais meses num mesmo ano.

```
a) select *  
from ipca  
where exists  
(select mes, ano from ipca x  
where x.ano = ipca.ano  
and x.indice > ipca.indice  
group by ano)  
order by ano, mês
```

b) select ano, mes
from ipca
where not exists
(select mes, ano from ipca x
where x.ano = ipca.ano
and x.indice > ipca.indice)
order by ano, mês

c) select ano, mes
from ipca
where not exists
(select mes, ano from ipca x
where x.ano >= ipca.ano
and x.indice < ipca.indice)
order by ano, mês

d) select ano, mes
from ipca
where exists
(select * from ipca x
where x.ano = ipca.ano
and x.indice = ipca.indice
group by mes)
order by ano, mês

e) select ano, mes
from ipca
where exists
(select mes, ano from ipca x
where x.ano = ipca.ano
or x.indice > ipca.indice)

Comentários:

a) **Errada.** O EXISTS aqui retornaria as linhas em que existe outro mês do mesmo ano com índice MAIOR, ou seja, traria justamente os meses que NÃO são o maior do ano, sendo o oposto do que se pede.

b) **Correta.** O NOT EXISTS verifica, para cada linha de IPCA, se NÃO existe outro mês do mesmo ano com índice maior. Se não existe nenhum mês maior naquele ano, então aquela linha é o maior índice do ano (ou está empatada com o maior). Como a subconsulta usa $x.indice > ipca.indice$ (estritamente maior), os empates são preservados, pois nenhum dos meses empatados é maior que o outro.

c) **Errada.** A condição $x.ano \geq ipca.ano$ compara entre anos diferentes também, misturando os índices de toda a série, em vez de restringir a comparação ao mesmo ano. Assim, não retorna corretamente o maior mês de cada ano.

d) **Errada.** O EXISTS sempre será verdadeiro, pois toda linha tem ela mesma como referência ($x.ano = ipca.ano$ e $x.indice = ipca.indice$), o que faria o comando retornar todas as linhas da tabela, e não apenas os maiores índices de cada ano.

e) **Errada.** O uso do OR amplia demais a condição, fazendo o EXISTS ser verdadeiro praticamente para qualquer linha, já que basta haver outra linha do mesmo ano OU com índice maior, retornando linhas que não correspondem ao maior índice do ano.

Gabarito: B

11. (FGV/AFFC (STN)/STN/Tecnologia da Informação/Transformação Digital/2024) ATENÇÃO: use a tabela relacional IPCA a seguir para responder à questão.

Tabela IPCA

indice	ano	mês
0,56	2023	12
0,28	2023	11
0,24	2023	10
...	...	...
0,2	2003	7
-0,15	2003	6
...	...	...
2,25	2003	1
2,1	2002	12
3,02	2002	11
...	...	...
0,57	2001	1

A instância da tabela contém os valores do índice IPCA para todos os meses dos anos de 2001 até 2023. Os valores pontilhados representam a continuidade mensal da série. Todas as colunas são numéricas, e não aceitam valores nulos.

No contexto da tabela IPCA apresentada, considere que ocorreu um acidente que fez com que diversas linhas dessa tabela tenham sido aleatoriamente deletadas, embora todos os índices dos meses de 2023 tenham permanecido intactos e nenhum dos anos tenha sido completamente deletado.

Analise as três versões de SQL que, pretensamente, poderiam recompor a tabela corretamente, inserindo os meses deletados com o valor nulo na coluna indice.

```
I. insert into IPCA(indice, ano, mes)
select NULL, a.ano, a.mes
from (select distinct ano, mes from IPCA) a
where not exists
      (select * from IPCA x
       where x.ano = a.ano
        and x.mes = a.mes)
```

```
II. insert into IPCA(indice, ano, mes)
select NULL, a.ano, b.mes
from (select distinct ano from IPCA) a,
     (select distinct mes from IPCA) b
where not exists
      (select * from IPCA x
       where x.ano = a.ano
        and x.mes = b.mes)
```

```
III. insert into IPCA(indice, ano, mes)
select NULL, a.ano, a.mes
from IPCA a
where a.ano * 100 + a.mes not in
      (select x.mes + x.ano * 100 from IPCA x)
```

A respeito da adequação desses comandos ao que se pretende, é correto concluir que

- a) nenhum seria adequado.
- b) somente I seria adequado.
- c) somente II seria adequado.
- d) somente III seria adequado.
- e) todos seriam adequados.

Comentários:

I. Errada. Essa consulta busca apenas as combinações de ano e mês que já existem na tabela IPCA (select distinct ano, mes from IPCA). Ou seja, ela só consegue gerar pares de (ano, mês) que ainda estão presentes, e nunca os que foram deletados. O NOT EXISTS sempre dará falso, porque toda combinação retornada pela subconsulta já existe na tabela. Resultado: nada será inserido.

II. Correta. Aqui o truque está em fazer um produto cartesiano (cruzamento de tabelas separadas por vírgula no FROM) entre todos os anos distintos e todos os meses distintos. Como o enunciado garante que todos os meses de 2023 estão intactos (gerando os 12 meses) e nenhum ano foi completamente deletado (gerando todos os anos de 2001 a 2023), esse cruzamento produz todas as combinações possíveis de ano e mês. Em seguida, o NOT EXISTS filtra apenas aquelas combinações que ainda não estão na tabela, inserindo-as com NULL no índice. É exatamente o que se pretende.

III. Errada. Essa consulta percorre a própria tabela IPCA (from IPCA a) e, para cada linha existente, verifica se ela não está na própria tabela. Como toda linha em "a" obviamente está em IPCA, a condição NOT IN sempre será falsa, e nada será inserido. Além disso, mesmo conceitualmente, ela não tem como gerar combinações de ano e mês que foram deletadas, já que parte de linhas que existem.

Gabarito: C

12. (FGV/AFTM (Pref N Iguaçu)/Pref Nova Iguaçu/2024) Seja um banco de dados relacional especificado em SQL de uma empresa de correspondência entre clientes, instituições financeiras e empréstimos contratados por esses clientes nessas instituições, previamente implementado em um banco de dados como a seguir:

```
create table tb_cliente
(
  id_cliente      integer      primary key,
  num_cpf         char(11)     unique,
  nome            varchar(50)  not null,
  email           varchar(20),
  telefone        varchar(20),
  endereco        varchar(100),
  cidade          varchar(20),
  estado          char(2) );

create table tb_financeira
(
  id_financeira   integer      primary key,
  razao_social    varchar(30)  not null,
  cidade          varchar(30)  not null,
  estado          char(2)      not null );
```

```
create table tb_emprestimo
(
  id_financeira integer references tb_financeira,
  id_cliente integer references tb_cliente,
  valor real not null check(valor>0),
  dia integer not null
  check(dia>=1 and dia<=31),
  mes integer not null
  check(mes>=1 and mes<=12),
  ano integer not null
  check(ano>=1980 and ano<=2100),
  primary key(id_financeira,id_cliente,dia,mes,ano) );
```

OBS: Neste banco de dados, cadeias de caracteres (strings) são representadas envoltas em aspas simples.

Para fins de investigação, os auditores da empresa desejam saber os nomes dos clientes que contrataram empréstimos em todas as financeiras.

Assinale a consulta que apresenta o resultado desejado pelos auditores.

- a) `select c.nome from tb_cliente c where exists ( select 1 from tb_emprestimo e where e.id_cliente = c.id_cliente )`
- b) `select c.nome from tb_cliente c where id_cliente = all ( select id_financeira from tb_emprestimo )`
- c) `select c.nome from tb_cliente c where not exists ( select id_financeira from tb_financeira except select e.id_financeira from tb_emprestimo e where e.id_cliente = c.id_cliente )`
- d) `select distinct c.nome from tb_financeira f natural join tb_emprestimo e full join tb_cliente c on c.id_cliente = e.id_cliente`
- e) `select distinct c.nome from tb_financeira natural join tb_emprestimo natural join tb_cliente c`

Comentários:

a) Errada. Esse comando apenas retorna os clientes que contrataram pelo menos um empréstimo (o EXISTS verifica se há ao menos uma linha em tb_emprestimo para aquele cliente), e não os que contrataram em todas as financeiras.

b) Errada. O operador = ALL compara id_cliente com todos os id_financeira retornados, o que não faz sentido lógico, pois são atributos diferentes, e nunca um id_cliente seria igual a todos os id_financeira ao mesmo tempo.

c) Correta. A lógica aqui é: para cada cliente, pega-se a lista de todas as financeiras existentes e subtrai (EXCEPT) as financeiras nas quais ele contratou empréstimo. Se essa diferença for vazia (NOT EXISTS), significa que não sobrou nenhuma financeira sem empréstimo daquele cliente, ou seja, ele contratou em todas. É um teste binário de existência aplicado a uma subconsulta correlacionada (referencia c.id_cliente da consulta externa).

d) **Errada.** Um FULL JOIN combinado com NATURAL JOIN apenas junta as tabelas e lista nomes de clientes envolvidos, sem garantir que cada cliente tenha contratado em todas as financeiras.

e) **Errada.** Esse JOIN apenas retorna os nomes de clientes que possuem algum empréstimo associado a alguma financeira, e não aqueles que possuem empréstimos em todas elas.

Gabarito: C

13. (FGV/AFRFB/SRFB/Geral/2023) Num banco de dados relacional, considere a tabela Vencedores, cuja instância é exibida a seguir, com duas colunas, Tenista e Torneio, que representam alguns torneios que já foram vencidos por alguns tenistas.

Tenista	Torneio
Roger Federer	Australian Open
Roger Federer	Roland Garros
Roger Federer	Wimbledon
Roger Federer	US Open
Pete Sampras	US Open
Pete Sampras	Wimbledon
Pete Sampras	Australian Open
Bjorn Borg	Roland Garros
Bjorn Borg	Wimbledon

Maria precisa escrever um comando SQL que liste os tenistas que venceram todos os torneios mencionados na coluna Torneio. O comando deve valer para qualquer instância válida da tabela, que pode conter diferentes tenistas e diferentes torneios.

Assinale o comando que Maria deve usar.

a)

```
select distinct Tenista from Vencedores v1  
where v1.Torneio in (select Torneio from Vencedores)
```

b)

```
select distinct Tenista from Vencedores v1  
where exists(  
select * from Vencedores v2  
where v1.Torneio = v2.Torneio  
and v1.Tenista = v2.Tenista  
and v1 <> v2))
```

c)

```
select distinct Tenista from Vencedores v1
where exists (
select * from Vencedores v2
where v1.Torneio = v2.Torneio
and v1.Tenista <> v2.Tenista )
```

d)

```
select distinct Tenista from Vencedores v1
where for all (
select * from Vencedores v2
where exists (
select * from Vencedores v3
where v1.Tenista = v2.Tenista))
```

e)

```
select distinct Tenista from Vencedores v1
where not exists(
select * from Vencedores v2
where not exists (
select * from Vencedores v3
where v2.Torneio = v3.Torneio
and v1.Tenista = v3.Tenista))
```

Comentários:

a) Errada. Esse comando retorna todos os tenistas que venceram qualquer torneio que esteja na lista de torneios, ou seja, retornaria todos os tenistas da tabela, e não apenas os que venceram todos os torneios.

b) Errada. Além de possuir erro de sintaxe (parêntese a mais e comparação $v1 <> v2$ inválida entre linhas inteiras), a lógica não garante que o tenista venceu todos os torneios existentes.

c) Errada. Esse comando apenas verifica se existe outro tenista diferente que venceu algum torneio, o que não tem relação com listar quem venceu todos os torneios.

d) **Errada.** Não existe a construção "for all" no SQL padrão, então o comando é sintaticamente inválido.

e) **Correta.** Essa é a forma clássica de simular o "para todos" no SQL usando dupla negação com NOT EXISTS. A leitura é: selecione o tenista para o qual NÃO EXISTE um torneio que ele NÃO tenha vencido. Ou seja, se não há nenhum torneio que ele deixou de vencer, então ele venceu todos. A subconsulta mais externa de NOT EXISTS busca torneios (v2), e a interna verifica se aquele tenista (v1.Tenista) tem registro de vitória naquele torneio (v3). Se sobrar algum torneio sem vitória, o tenista é eliminado; se não sobrar nenhum, ele é selecionado.

Gabarito: E

14. (FGV/Ana (Pref Niterói)/Pref Niterói/Tecnologia da Informação/2023) Figura 1
Nas duas questões a seguir, considere a tabela relacional T cuja instância é exibida abaixo.

A instalação está configurada para o tratamento de valores NULL como valores desconhecidos (unknown).

Tabela T

A	B	C
3	6	10
1	NULL	10
3	6	NULL
3	6	5

Considerando a tabela T da figura 1, analise o comando a seguir.

DELETE FROM T

WHERE EXISTS (SELECT * FROM T t2

WHERE T.A = t2.A

and T.B = t2.B

and T.C = t2.C)

O número de linhas removidas pela execução do comando acima é:

a) 0;

b) 1;

c) 2;

d) 3;

e) 4.

Comentários:

a) **Correta.** Nenhuma linha é removida porque o tratamento de NULL como unknown faz as comparações com "=" não retornarem TRUE quando há NULL envolvido, e a banca considerou que isso impede a remoção das linhas.

b) **Errada.** Não é apenas 1 linha removida.

c) **Errada.** Não são 2 linhas removidas, pois o tratamento de NULL como unknown impede a deleção.

d) **Errada.** Não são 3 linhas removidas.

e) **Errada.** Não são todas as 4 linhas removidas.

Gabarito: A

15. (FGV/FTE (SEFAZ MT)/SEFAZ MT/2023) No contexto das linguagens de manipulação de dados de SGBD relacionais, analise a instância da tabela T e o comando SQL a seguir.

pessoa	ancestral
Bruna	Joana
Joana	João
João	Maria
Maria	Gabriel
Paulo	Gabriel

```
insert into T
select t1.pessoa, t2.ancestral
from T t1, T t2
where t1.ancestral = t2.pessoa
and not exists
(select * from T tt
where tt.pessoa = t1.pessoa
and tt.ancestral = t2.ancestral)
```

Dado que o comando SQL acima foi executado por três vezes consecutivas, assinale o número de linhas inseridas na tabela T em cada execução, na ordem.

a) 0, 0, 0.

b) 3, 5, 0.

c) 5, 2, 1.

d) 5, 3, 0.

e) 8, 0, 0.

Comentários:

a) Errada. O comando insere sim novas linhas, pois há ancestrais de ancestrais que ainda não estão na tabela.

b) Correta. Na primeira execução, o SQL junta T com ela mesma cruzando ancestral de t1 com pessoa de t2, formando pares "pessoa-ancestral_do_ancestral". O EXISTS é uma subconsulta correlacionada que verifica se aquele par já existe em T; quando não existe, a linha é inserida. Na primeira rodada surgem 3 novos pares (Bruna-João, Joana-Maria, João-Gabriel). Na segunda rodada, com a tabela já enriquecida, surgem mais 5 pares novos (incluindo combinações como Bruna-Maria, Bruna-Gabriel, Joana-Gabriel etc.). Na terceira rodada, todos os pares já existem, então o NOT EXISTS bloqueia tudo e nenhuma linha é inserida.

c) Errada. Os números não correspondem ao funcionamento da junção combinada com o NOT EXISTS aplicado a cada execução.

d) Errada. A segunda execução não insere apenas 3 linhas; com a tabela ampliada após a primeira execução, surgem mais combinações novas a serem inseridas.

e) Errada. Não é possível inserir 8 linhas de uma só vez na primeira execução, porque várias dessas combinações só passam a existir depois que a primeira rodada já adicionou pares intermediários.

Gabarito: B

16. (FGV/Ana (BBTS)/BBTS/Perfil Tecnológico/2023) Considere uma tabela T, com uma única coluna A, definida como uma chave primária, e o comando SQL a seguir.

```
delete from T
where not exists
(select * from T tt where T.A > tt.A)
```

Dado que a tabela tem 100 linhas preenchidas, assinale a opção que indica o número de linhas que será deletado pela execução do referido comando.

- a) 0.
- b) 1.
- c) 50.
- d) 99.
- e) 100.

Comentários:

a) Errada. Existirá uma linha (a de menor valor) que satisfaz a condição e será deletada.

b) **Correta.** Apenas a linha com o menor valor de A não tem ninguém menor que ela, sendo a única deletada.

c) **Errada.** O comando não deleta metade da tabela, apenas a linha com menor valor.

d) **Errada.** Apenas a linha de menor valor é deletada, e não 99 delas.

e) **Errada.** As 99 linhas que possuem alguma outra com valor menor não satisfazem o NOT EXISTS e permanecem.

Gabarito: B

17. (FGV/AL (CAM DEP)/CAM DEP/Tecnologia da Informação/2023) Quando mencionada, considere a tabela relacional intitulada *fibonacci*, com duas colunas, cuja instância, não necessariamente nessa ordem, é exibida a seguir.

Tabela fibonacci	
P	N
1	0
2	1
3	1
4	2
5	3
6	5
7	8
8	13
9	21
10	34

A coluna da esquerda enumera as linhas, continuamente, e a coluna da direita armazena os dez primeiros elementos da sequência de Fibonacci, na qual os dois primeiros termos são 0 e 1, por definição, e cada termo subsequente é a soma dos dois termos anteriores.

Suponha que a tabela *fibonacci*, apresentada anteriormente, tenha sido danificada, e sua instância corrente seja a que segue.

P	N
1	0
2	1

P	N
3	1
6	5
7	8

Para restaurar a tabela, foi produzido o comando SQL a seguir.

```
insert into fibo
select f2.P + 1, f1.N + f2.N
from fibo f1, fibo f2
where f1.P = f2.P - 1
      and f2.P < 10
      and not exists
        (select * from fibo f3
         where f1.N + f2.N = f3.N and f3.P > f2.P)
```

À luz das características de funcionamento do SQL, o número mínimo de vezes que esse comando deve ser acionado para restaurar a instância original, é

- a) 1.
- b) 2.
- c) 3.
- d) 4.
- e) 5.

Comentários:

- a) **Errada.** Uma única execução só preenche P=4 e P=8, ainda faltam várias linhas.
- b) **Errada.** Após duas execuções ainda falta P=10, pois ele depende de P=8 e P=9, que só ficam prontos ao final da segunda execução.
- c) **Correta.** São necessárias três execuções para que as dependências em cadeia (cada termo precisa dos dois anteriores já existirem) sejam resolvidas.
- d) **Errada.** Três execuções já bastam, não há necessidade de uma quarta.
- e) **Errada.** O número mínimo é três, não cinco.

Gabarito: C

18. (FGV/Prof EFM (SEDUC SP)/SEDUC SP/Informação e Comunicação/2023) Gerenciadores de bancos de dados relacionais usualmente empregam a lógica de três estados quando há

operações de comparação envolvendo valores nulos (NULL). Nesse contexto, considere o comando SQL a seguir.

```
delete from T  
where exists (select * from T T2 where T.A=T2.A)
```

Nesse contexto, é correto afirmar que

- a) nenhuma linha de T será excluída, independentemente do conteúdo da coluna A.
- b) todas as linhas de T que contêm valores não nulos na coluna A serão excluídas.
- c) todas as linhas de T que contêm valores nulos na coluna A serão excluídas.
- d) todas as linhas de T serão excluídas, independentemente do conteúdo da coluna A.
- e) o comando contém um erro, pois a sintaxe do SQL não permite a autorreferência na cláusula exists.

Comentários:

a) Errada. Linhas com valor não nulo em A serão excluídas, pois para essas linhas a subconsulta encontra ao menos uma linha (a própria) onde $T.A = T2.A$, fazendo o EXISTS retornar verdadeiro.

b) Correta. O EXISTS verifica se existe ao menos uma linha na subconsulta que satisfaça a condição $T.A = T2.A$. Para linhas com valores não nulos, essa comparação é verdadeira (pelo menos comparando consigo mesma), então o EXISTS retorna verdadeiro e a linha é deletada. Já para linhas com NULL em A, a comparação $NULL = NULL$ não retorna verdadeiro na lógica de três estados, então o EXISTS não se confirma e essas linhas permanecem.

c) Errada. É o oposto: linhas com NULL em A não satisfazem a comparação $T.A = T2.A$, pois comparações envolvendo NULL não retornam verdadeiro, então essas linhas não são excluídas.

d) Errada. As linhas com NULL em A não serão excluídas, pois a comparação de igualdade envolvendo NULL não é avaliada como verdadeira.

e) Errada. A sintaxe do SQL permite tranquilamente referenciar a mesma tabela dentro do EXISTS usando um alias diferente (T2), o que caracteriza uma subconsulta correlacionada válida.

Gabarito: B

19. (FGV/AJ (TJ SE)/TJ SE/Análise de Sistemas/Banco de Dados/2023) Considere as tabelas T1 e T2, exibidas abaixo com suas respectivas instâncias.

T1

A	B
1	1
2	2

4	2
T2	
A	C
1	10
2	20

Observe o comando SQL a seguir.

```
select sum(A) from T1
where not exists
      (select * from T2 where T1.A = 2 * T2.A)
```

Dadas as tabelas T1 e T2, descritas anteriormente, o resultado da execução desse comando exibe o valor:

- a) 1;
- b) 2;
- c) 3;
- d) 5;
- e) 7.

Comentários:

- a) **Correta.** Apenas a linha com A=1 satisfaz o NOT EXISTS, e a soma resulta em 1.
- b) **Errada.** O valor 2 não corresponde à soma das linhas que passam no filtro.
- c) **Errada.** O valor 3 seria a soma de 1+2, mas a linha A=2 não passa no NOT EXISTS.
- d) **Errada.** O valor 5 seria a soma de 1+4, mas a linha A=4 também não passa no NOT EXISTS.
- e) **Errada.** O valor 7 seria a soma de todas as linhas de T1 (1+2+4), mas o filtro elimina duas delas.

Gabarito: A

20. (FGV/AJ (TJ SE)/TJ SE/Análise de Sistemas/"Sem Especialidade"/2023) Considere uma tabela relacional TAB, com colunas A e B. A coluna A constitui a chave primária de TAB. A instância de TAB contém 100 linhas, e em todas as linhas o valor da coluna B é 10. Nesse contexto, analise o comando SQL a seguir.

```
select * from TAB t
where not exists
      (select * from TAB tt
```

where $t.B = tt.B$ and $t.A > tt.A$)

Além da linha de títulos, o número de linhas produzidas pelo comando acima é:

- a) 0;
- b) 1;
- c) 98;
- d) 99;
- e) 100.

Comentários:

a) Errada. Existe sim uma linha que satisfaz o NOT EXISTS, que é a de menor valor de A.

b) Correta. Apenas a linha com o menor valor de A passa no filtro NOT EXISTS, pois é a única para a qual não existe nenhuma outra linha com A menor.

c) Errada. Não há razão para retornar 98 linhas; a lógica do NOT EXISTS isola apenas a linha de menor A.

d) Errada. 99 linhas seria o resultado se fosse EXISTS (todas exceto a de menor A), e não NOT EXISTS.

e) Errada. Retornar 100 linhas só ocorreria se a condição fosse sempre verdadeira para todas, o que não é o caso aqui.

Gabarito: B

21. (FGV/AUFC (TCU)/TCU/Controle Externo/Auditoria Governamental/2022) Na questão abaixo, considere as tabelas de banco de dados T, TX e DUAL, exibidas com suas respectivas instâncias a seguir.

T

sequencia	caracteristica
1	23987
2	9845
3	NULL
4	40983
6	48750
7	NULL
8	NULL
10	48750
12	48750

TX

sequencia	caracteristica
-----------	----------------

2	9845
3	998034
4	50932
5	24390
6	48750
6	50296
7	NULL
8	998746
9	32746
9	NULL
9	22798

DUAL

x
NULL

Nas colunas das três tabelas, o tipo é o de número inteiro. Em todos os comandos SQL, considera-se o NULL como um valor desconhecido (unknown).

Considere que é preciso atualizar os dados da tabela T a partir dos dados da tabela TX, ambas definidas anteriormente. A consolidação é feita por meio da alteração na tabela T a partir de registros de TX.

O comando SQL utilizado nessa atualização é exibido a seguir.

update T

set caracteristica =

(select max(caracteristica) x from TX tx

where tx.sequencia = t.sequencia

and not (tx.caracteristica is null))

where

(exists

(select * from TX tx

where tx.sequencia = t.sequencia

and not (tx.caracteristica is null))

and

(t.caracteristica is null

or

```
t.caracteristica <
(select max(caracteristica) x
from TX tx
where tx.sequencia = t.sequencia
and not (tx.caracteristica is null))
)
)
```

O número de registros da tabela T afetados pela execução do comando SQL acima é:

- a) zero;
- b) três;
- c) quatro;
- d) seis;
- e) nove.

Comentários:

- a) Errada.** Há atualizações sim, pois várias linhas de T encontram correspondência válida em TX.
- b) Errada.** São quatro linhas, não três; a sequencia 8 também é atualizada porque T é NULL e existe valor não nulo em TX.
- c) Correta.** As sequencias 3, 4, 6 e 8 satisfazem ambas as condições do WHERE externo, totalizando quatro registros atualizados.
- d) Errada.** Sequencias 1, 7, 10 e 12 não passam no EXISTS (não há correspondência válida em TX), e a sequencia 2 não passa na segunda condição.
- e) Errada.** Nem todas as linhas de T têm correspondência em TX com valor não nulo, então não são nove.

Gabarito: C

22. (FGV/AUFC (TCU)/TCU/Controle Externo/Auditoria Governamental/2022) Na questão abaixo, considere as tabelas de banco de dados T, TX e DUAL, exibidas com suas respectivas instâncias a seguir.

T

sequencia	caracteristica
1	23987
2	9845
3	NULL

4	40983
6	48750
7	NULL
8	NULL
10	48750
12	48750

TX

sequencia	caracteristica
2	9845
3	998034
4	50932
5	24390
6	48750
6	50296
7	NULL
8	998746
9	32746
9	NULL
9	22798

DUAL

x
NULL

Nas colunas das três tabelas, o tipo é o de número inteiro. Em todos os comandos SQL, considera-se o NULL como um valor desconhecido (unknown).

Supondo que a coluna sequencia da tabela T, anteriormente definida, deveria conter números inteiros em sequência contínua, seria preciso descobrir os intervalos de valores faltantes. Um valor é considerado faltante quando a) é um número inteiro n entre o menor e o maior valor da tabela, tal que n não esteja presente na tabela, ou b) é um número presente na tabela T, com valor nulo na coluna característica.

inicio	fim	faltantes
3	3	1
5	5	1
7	9	3
11	11	1

O comando SQL que produz o resultado acima, a partir da instância inicialmente definida para a tabela T, é:

a)

```
select t1.sequencia 'inicio', t2.sequencia 'fim',  
       t2.sequencia - t1.sequencia - 1 faltantes  
from T t1, T t2  
where t1.sequencia < t2.sequencia  
      and t1.sequencia <> t2.sequencia - 1  
      and not exists  
      (select * from T t3  
       where t3.sequencia > t1.sequencia  
         and t3.sequencia < t2.sequencia)
```

b)

```
select t1.sequencia + 1 'inicio',  
       t2.sequencia - 1 'fim',  
       t2.sequencia - t1.sequencia - 1 faltantes  
from T t1, T t2  
where t1.sequencia < t2.sequencia  
      and t1.sequencia <> t2.sequencia - 1  
      and t1.caracteristica is not null  
      and t2.caracteristica is not null  
      and (not exists  
      (select * from T t3  
       where t3.sequencia > t1.sequencia  
         and t3.sequencia < t2.sequencia  
         and t3.caracteristica is not null))
```

c)

```
select t1.sequencia + 1 'inicio',  
       t2.sequencia - 1 'fim',  
       t2.sequencia - t1.sequencia - 1 faltantes  
from T t1, T t2  
where t1.sequencia <> t2.sequencia - 1  
      and t1.caracteristica is not null  
      and t2.caracteristica is not null  
      and (not exists  
      (select * from T t3  
       where t3.sequencia > t1.sequencia  
         and t3.sequencia < t2.sequencia  
         and t3.caracteristica is not null)  
      or exists  
      (select * from T t3  
       where t3.sequencia > t1.sequencia
```

```
and t3.sequencia < t2.sequencia  
and t3.caracteristica is null))
```

d)

```
select t1.sequencia +1 'inicio',  
       t2.sequencia -1 'fim',  
       t2.sequencia - t1.sequencia -1 faltantes  
from T t1, T t2  
where t1.sequencia < t2.sequencia  
      and t1.sequencia <> t2.sequencia -1  
      and t1.caracteristica is not null  
      and t2.caracteristica is not null  
      and exists  
      (select * from T t3  
       where t3.sequencia > t1.sequencia  
         and t3.sequencia < t2.sequencia)
```

e)

```
select t1.sequencia +1 'inicio',  
       t2.sequencia -1 'fim',  
       t2.sequencia - t1.sequencia -1 faltantes  
from T t1, T t2  
where t1.sequencia < t2.sequencia  
      and t1.sequencia <> t2.sequencia -1  
      and t1.caracteristica is not null  
      and t2.caracteristica is not null  
      and not exists  
      (select * from T t3  
       where t3.sequencia > t1.sequencia  
         and t3.sequencia < t2.sequencia)
```

Comentários:

a) Errada. Não filtra os registros com característica nula, então não trata o caso (b) do enunciado, em que valores presentes na tabela mas com característica NULL devem ser considerados faltantes. Além disso, retorna início e fim como os próprios valores existentes, e não como o primeiro e o último faltante do intervalo.

b) Correta. Aqui temos pares de linhas (t1, t2) da tabela T, ambos com característica não nula, em que t1.sequencia é menor que t2.sequencia e entre eles não existe nenhuma outra linha com característica não nula (subconsulta correlacionada com NOT EXISTS, que verifica se não há linha intermediária válida). Assim, t1+1 vira o início do intervalo faltante, t2-1 vira o fim, e a diferença menos 1 dá a quantidade de faltantes. Isso cobre tanto sequências ausentes quanto sequências presentes mas com característica NULL.

c) **Errada.** Faltou a condição $t1.sequencia < t2.sequencia$, o que permite pares invertidos. Além disso, o uso do OR EXISTS no final permite combinações indevidas que quebram a ideia de intervalo contíguo de faltantes.

d) **Errada.** O uso de EXISTS (em vez de NOT EXISTS) inverte a lógica: passariam pares em que existe alguma linha intermediária, justamente o oposto do que se quer, pois precisamos de intervalos sem nenhuma linha válida no meio.

e) **Errada.** O NOT EXISTS aqui não exige que a linha intermediária tenha característica não nula, então linhas com característica NULL no meio do intervalo bloqueariam pares que deveriam ser válidos, quebrando a contagem de faltantes pedida pelo enunciado.

Gabarito: B

23. (FGV/AJ (TJDFT)/TJDFT/Apoio Especializado/Análise de Dados/2022) Tabela TURFE

páreo	cavalo	tempo
-------	--------	-------

1	Estupenda	52,2
1	Mandrake	52,3
1	Capricho	53,4
2	Estupenda	51,0
2	Mandrake	51,1
2	Muddy River	52,1
2	Uramak	52,3
2	Escorial	52,5
2	Capricho	56,1
3	Muddy River	52,1
3	Estupenda	52,3
3	Uramak	53,0
3	Escorial	55,1

Considere um banco de dados relacional que contém uma única tabela, TURFE, cuja estrutura é exibida com sua instância, a seguir.

Para cada páreo, ou corrida, são armazenados os nomes dos cavalos participantes e os respectivos tempos. A classificação de cada cavalo numa corrida segue a ordem crescente de tempo. Não há empates.

Com relação à tabela TURFE, descrita anteriormente, o comando SQL que exhibe, para cada páreo, somente o cavalo que chegou em último lugar com o respectivo tempo é:

```
a) select pareo, cavalo, tempo
from TURFE
where for all
(select * from TURFE t
where t.pareo = TURFE.pareo
```

and t.tempo < TURFE.tempo)

b) select pareo, cavalo, tempo
from TURFE

where exists

(select * from TURFE t

where t.pareo = TURFE.pareo

and t.tempo > TURFE.tempo)

c) select pareo, cavalo, tempo
from TURFE

where not exists

(select * from TURFE t

where t.pareo <> TURFE.pareo

and t.tempo = TURFE.tempo)

d) select pareo, cavalo, tempo
from TURFE

where not exists

(select * from TURFE t

where t.pareo = TURFE.pareo

and t.tempo > TURFE.tempo)

e) select pareo, cavalo, tempo
from TURFE

where ALL

(select * from TURFE t

where t.pareo = TURFE.pareo

and t.tempo <> TURFE.tempo)

Comentários:

a) **Errada.** A sintaxe "where for all" não existe em SQL, é um comando inválido.

b) Errada. O EXISTS aqui retornaria os cavalos para os quais existe outro cavalo no mesmo páreo com tempo maior, ou seja, retornaria todos exceto o último colocado, justamente o oposto do que se pede.

c) Errada. A condição compara páreos diferentes (t.pareo <> TURFE.pareo) com tempos iguais, o que não tem relação com identificar o último colocado de cada páreo.

d) Correta. O NOT EXISTS verifica que não existe nenhum outro cavalo no mesmo páreo (t.pareo = TURFE.pareo) com tempo maior (t.tempo > TURFE.tempo). Se ninguém tem tempo maior dentro do páreo, então aquele cavalo é o que demorou mais, ou seja, chegou em último. Trata-se de uma subconsulta correlacionada, pois a consulta interna referencia a tabela da consulta externa (TURFE.pareo e TURFE.tempo).

e) Errada. A sintaxe "where ALL (...)" está incorreta, pois o ALL precisa ser usado junto com um operador de comparação (como >, <, =) e um valor sendo comparado, o que não ocorre aqui.

Gabarito: D

24. (FGV/AJ (TJDFT)/TJDFT/Apoio Especializado/Análise de Dados/2022) Tabela TURFE

	páreo	cavalo	tempo
--	-------	--------	-------

1	Estupenda	52,2
1	Mandrake	52,3
1	Capricho	53,4
2	Estupenda	51,0
2	Mandrake	51,1
2	Muddy River	52,1
2	Uramak	52,3
2	Escorial	52,5
2	Capricho	56,1
3	Muddy River	52,1
3	Estupenda	52,3
3	Uramak	53,0
3	Escorial	55,1

Considere um banco de dados relacional que contém uma única tabela, TURFE, cuja estrutura é exibida com sua instância, a seguir.

Analise o comando SQL a seguir.

```
select distinct cavalo
```

```
from TURFE
```

```
where not exists
```

```
(select *
```

```
from
```

```
(select t1.pareo, sum(1) xpto, t1.cavalo  
from TURFE t1, TURFE t2  
where t1.pareo = t2.pareo  
and t2.tempo <= t1.tempo  
group by t1.cavalo, t1.pareo) x  
where TURFE.cavalo = x.cavalo  
and x.xpto <> 2  
)
```

Considerando-se a instância da tabela TURFE descrita anteriormente, a execução do comando acima exibe no resultado o(s) nome(s):

- a) Capricho;
- b) Escorial;
- c) Estupenda, Escorial;
- d) Mandrake;
- e) Uramak, Muddy River.

Comentários:

- a) **Errada.** Capricho terminou em última posição nos páreos que disputou, não foi sempre 2°.
- b) **Errada.** Escorial não terminou em 2° lugar em todos os páreos.
- c) **Errada.** Estupenda venceu páreos (posição 1), então não satisfaz a condição de ser sempre 2°.
- d) **Correta.** Mandrake foi 2° colocado tanto no páreo 1 quanto no páreo 2, sendo o único cavalo cuja posição é sempre 2 em todos os páreos disputados.
- e) **Errada.** Uramak e Muddy River não terminaram em 2° lugar em todos os seus páreos.

Gabarito: D

25. (FGV/AJ (TJDFT)/TJDFT/Apoio Especializado/Análise de Sistemas/2022) Tabela TURFE

	páreo	cavalo	tempo
1		Estupenda	52,2
1		Mandrake	52,3
1		Capricho	53,4
2		Estupenda	51,0
2		Mandrake	51,1
2		Muddy River	52,1
2		Uramak	52,3

2	Escorial	52,5
2	Capricho	56,1
3	Muddy River	52,1
3	Estupenda	52,3
3	Uramak	53,0
3	Escorial	55,1

Considere um banco de dados relacional que contém uma única tabela, TURFE, cuja estrutura é exibida com sua instância a seguir

Para cada páreo, ou corrida, são armazenados os nomes dos cavalos participantes e os respectivos tempos. A classificação de cada cavalo numa corrida segue a ordem crescente de tempo. Não há empates.

Com relação à tabela TURFE, descrita anteriormente, analise o comando SQL a seguir.

```
select distinct cavalo
from TURFE t1
where not exists
(select * from TURFE t2, TURFE t3
where t2.cavalo = t1.cavalo
and t3.pareo = t2.pareo
and t3.tempo > t2.tempo)
```

A execução do comando acima produz, somente, todos os cavalos que, em suas corridas:

- a) nunca chegaram em último lugar;
- b) nunca venceram e nunca chegaram em último lugar;
- c) nunca venceram;
- d) sempre chegaram em último lugar;
- e) sempre venceram.

Comentários:

a) Errada. A consulta não busca quem nunca chegou em último; pelo contrário, busca quem sempre foi o último.

b) Errada. Não há nenhuma condição que relacione a vitória; o foco está em quem nunca teve alguém atrás (ou seja, sempre foi o último).

c) Errada. A consulta não filtra por vitórias, e sim por cavalos que nunca tiveram concorrente com tempo maior que o seu.

d) **Correta.** O NOT EXISTS garante que em nenhuma corrida do cavalo houve outro com tempo maior, ou seja, ele sempre ocupou a última posição em todas as corridas que disputou.

e) **Errada.** Para sempre vencer, a condição teria que ser o oposto: nunca existir alguém com tempo MENOR que o seu, e não MAIOR.

Gabarito: D

26. (FGV/Ana (MPE SC)/MPE SC/Tecnologia da Informação/2022) Maria trabalha com um banco de dados onde há uma tabela T com linhas repetidas. Ela precisa remover as repetições indesejadas, porém mantendo as linhas sem repetição e apenas uma linha de cada conjunto de linhas repetidas, como mostrado a seguir.

Instância da tabela T antes da remoção:

A	B	C
2	5	7
2	5	7
2	5	7
1	8	9
1	8	9
2	2	2

Instância da tabela T após a remoção:

A	B	C
2	5	7
1	8	9
2	2	2

Maria optou por usar o comando abaixo para realizar a tarefa.

```
delete from T where  
exists (select * FROM T t1  
where T.A = t1.A and T.B = t1.B  
and T.C = t1.C)
```

Na execução desse comando, serão removidas:

- a) apenas uma linha pertencente a cada conjunto de linhas que contém linhas repetidas;
- b) apenas n-1 linhas pertencentes a cada conjunto que contém n linhas repetidas, onde n é um número par;

- c) apenas $n-1$ linhas pertencentes a cada conjunto que contém n linhas repetidas, onde n é um número ímpar;
- d) todas as linhas da tabela, exceto as linhas que não foram repetidas;
- e) todas as linhas da tabela, sem exceção.

Comentários:

- a) **Errada.** O comando não remove apenas uma linha de cada grupo de repetidas; ele remove todas, pois o EXISTS sempre encontra ao menos a própria linha avaliada como correspondente.
- b) **Errada.** Não há essa lógica de remover $n-1$ linhas em grupos pares, pois o EXISTS avalia para verdadeiro em todas as linhas da tabela, sem distinção de quantidade.
- c) **Errada.** Mesmo raciocínio: não existe corte por paridade. Como toda linha sempre encontra ao menos a si mesma na subconsulta, o EXISTS é verdadeiro para todas.
- d) **Errada.** Até as linhas não repetidas serão removidas, porque qualquer linha satisfaz a condição da subconsulta ao se comparar consigo própria, fazendo o EXISTS retornar verdadeiro.
- e) **Correta.** O EXISTS é um teste binário que verifica se a subconsulta retorna pelo menos uma linha. Como a subconsulta procura linhas com os mesmos valores de A, B e C da linha avaliada, toda linha encontra ao menos a si mesma, fazendo o EXISTS ser verdadeiro para todas. Resultado: todas as linhas da tabela são removidas, sem exceção.

Gabarito: E

27. (FGV/ES (SEMSA Manaus)/Pref Manaus/Analista de Banco de Dados/2022) Para a próxima s questões, considere do banco de dados relacional happyhour, que contém três tabelas, a saber. frequencia (cliente, bar)

preferencia (cliente, cerveja)

oferta (bar, cerveja)

A relação frequencia registra o(s) bar(es) que cada cliente frequenta, e tem dois atributos: nome do cliente e nome do bar;

A relação preferencia armazena a(s) cerveja(s) preferida(s) por cada cliente e tem dois atributos: nome do cliente e nome da cerveja;

A relação oferta relaciona a(s) cerveja(s) que cada bar serve, também com dois atributos: nome do bar e nome da cerveja.

É dado que todo cliente frequenta pelos menos um bar, que nem todos os clientes têm preferências, que todo bar oferece pelo menos uma cerveja e que toda cerveja é oferecida pelo menos por um bar.

Para o banco de dados happyhour, especificado anteriormente, assinale a expressão SQL que produz uma lista com os nomes dos clientes que frequentam todos os bares.

a) select cliente from frequencia f1
where not exists
(select * from oferta where f1.bar= bar)

b) select cliente from frequencia f1
where not exists
(select * from oferta
where not exists
(select * from frequencia f2
where f1.cliente = f2.cliente
and oferta.bar = f2.bar))

c) select cliente from frequencia f1
where not exists
(select * from oferta
where not exists
(select * from frequencia f2
where f1.cliente <> f2.cliente
and oferta.bar <> f2.bar))

d) select cliente from frequencia

e) select cliente from frequencia f1
where not exists
(select * from frequencia
where not exists
(select * from oferta f2
where f1.cliente = f2.cliente
and oferta.bar = f2.bar))

Comentários:

a) Errada. Essa consulta apenas verifica se não existe um bar na tabela oferta com o mesmo nome do bar frequentado pelo cliente, o que não corresponde à ideia de listar clientes que frequentam todos os bares.

b) **Correta.** Aqui usamos a técnica do duplo NOT EXISTS, que é a forma clássica em SQL de expressar "para todos". A lógica é: traga o cliente para o qual NÃO EXISTE um bar (na tabela oferta) que ele NÃO frequente. Se não há nenhum bar que o cliente deixe de frequentar, então ele frequenta todos. Note que são subconsultas correlacionadas, pois a interna usa f1.cliente (da consulta externa) e a do meio usa oferta.bar.

c) **Errada.** O uso de <> (diferente) nas comparações inverte a lógica que precisamos. Para casar cliente com sua frequência e o bar com o da oferta, precisamos de igualdade (=), não de desigualdade.

d) **Errada.** Essa consulta apenas lista todos os clientes da tabela frequencia, sem qualquer filtro que garanta que eles frequentem todos os bares.

e) **Errada.** A estrutura está embaralhada: o NOT EXISTS do meio percorre a tabela frequencia em vez de oferta, o que não permite verificar se cada bar existente é frequentado pelo cliente.

Gabarito: B

28. (FGV/ES (SEMSA Manaus)/Pref Manaus/Analista de Banco de Dados/2022) Para as próximas questões, considere do banco de dados relacional happyhour, que contém três tabelas, a saber.

frequencia (cliente, bar)

preferencia (cliente, cerveja)

oferta (bar, cerveja)

A relação frequencia registra o(s) bar(es) que cada cliente frequenta, e tem dois atributos: nome do cliente e nome do bar;

A relação preferencia armazena a(s) cerveja(s) preferida(s) por cada cliente e tem dois atributos: nome do cliente e nome da cerveja;

A relação oferta relaciona a(s) cerveja(s) que cada bar serve, também com dois atributos: nome do bar e nome da cerveja.

É dado que todo cliente frequenta pelos menos um bar, que nem todos os clientes têm preferências, que todo bar oferece pelo menos uma cerveja e que toda cerveja é oferecida pelo menos por um bar.

Com relação ao banco de dados happyhour, considere que houve um erro de carga, de forma que há bares presentes na tabela frequencia que não aparecem na tabela oferta (que deveria conter todos os bares).

Assinale o comando SQL que deletaria da tabela frequencia as linhas referentes a esses bares.

a) delete from frequencia

where oferta.bar <> frequencia.bar

b) delete from frequencia

from oferta

where oferta.bar <> frequencia.bar

c) delete from frequencia

where bar in (select bar from oferta)

d) delete from frequencia

where not exists

(select * from oferta f

where f.bar = frequencia.bar)

e) delete from frequencia

where bar = (select bar from oferta

where bar is null)

Comentários:

a) Errada. A consulta tenta usar a tabela oferta no WHERE sem que ela esteja presente no FROM ou em uma subconsulta, então a referência oferta.bar é inválida nesse contexto.

b) Errada. A sintaxe com dois FROM no DELETE não é válida no SQL padrão estudado, além disso a condição <> compararia linha a linha gerando resultados incorretos, pois bastaria existir um bar diferente em oferta para a condição ser verdadeira.

c) Errada. Esse comando faz o oposto do que se pede: deletaria os bares que existem na tabela oferta, mantendo justamente os que estão com erro de carga (aqueles que não aparecem em oferta).

d) Correta. O NOT EXISTS com subconsulta correlacionada (note que f.bar = frequencia.bar referencia a tabela externa) verifica, para cada linha da tabela frequencia, se existe pelo menos uma linha em oferta com o mesmo bar. Se não existir, a subconsulta retorna vazio, o NOT EXISTS é verdadeiro e a linha é deletada, removendo exatamente os bares que estão em frequencia mas não em oferta.

e) Errada. A subconsulta retorna os bares de oferta cujo nome é nulo, o que não corresponde aos bares ausentes em oferta. Além disso, o operador = com uma subconsulta que retorna várias linhas geraria erro.

Gabarito: D

29. (FGV/ES (SEMSA Manaus)/Pref Manaus/Analista de Banco de Dados/2022) Para a próxima s questões, considere do banco de dados relacional happyhour, que contém três tabelas, a saber. frequencia (cliente, bar)

preferencia (cliente, cerveja)

oferta (bar, cerveja)

A relação frequencia registra o(s) bar(es) que cada cliente frequenta, e tem dois atributos: nome do cliente e nome do bar;

A relação preferencia armazena a(s) cerveja(s) preferida(s) por cada cliente e tem dois atributos: nome do cliente e nome da cerveja;

A relação oferta relaciona a(s) cerveja(s) que cada bar serve, também com dois atributos: nome do bar e nome da cerveja.

É dado que todo cliente frequenta pelos menos um bar, que nem todos os clientes têm preferências, que todo bar oferece pelo menos uma cerveja e que toda cerveja é oferecida pelo menos por um bar.

Com relação ao banco de dados happyhour, considere que houve uma ação de marketing que demanda que todos os bares tenham em suas ofertas a cerveja Forster.

Assinale o comando SQL que incluiria essa cerveja na tabela oferta para todos os bares que ainda não fazem essa oferta.

a) insert into oferta

select bar, 'Forster' from oferta

where not exists

(select * from oferta o

where o.cerveja = 'Forster'

and o.bar = oferta.bar)

b) insert into oferta

select bar, 'Forster' from oferta

where not exists

(select * from oferta o

where o.cerveja = 'Forster')

c) insert into oferta

select bar, 'Forster'

from (select distinct bar from oferta) x

where not exists

(select * from oferta o

```
where o.cerveja = 'Forster'
and o.bar = x.bar)
d) insert into oferta
select bar, 'Forster' from oferta
where
(select count(*) from oferta o
where o.cerveja = 'Forster'
and oferta.bar = o.bar) = 0
e) insert into oferta
select bar, 'Forster' from oferta
where oferta.bar not in
(select o.bar from oferta o
where o.cerveja = 'Forster'
and oferta.bar = o.bar)
```

Comentários:

a) Errada. O problema aqui é que o SELECT externo lê diretamente a tabela oferta, que pode ter várias linhas para o mesmo bar (uma para cada cerveja oferecida). Isso faria com que o mesmo bar fosse inserido várias vezes com a cerveja Forster, gerando duplicatas.

b) Errada. Além do mesmo problema da letra a (gera duplicatas porque lê a tabela oferta sem distinguir bares), a subconsulta com NOT EXISTS não está correlacionada com o bar avaliado. Ou seja, basta que exista qualquer linha com a cerveja Forster em qualquer bar para que o NOT EXISTS seja falso para todo mundo, impedindo qualquer inserção.

c) Correta. Aqui está o pulo do gato: a tabela derivada (subconsulta no FROM) "select distinct bar from oferta" garante que cada bar apareça uma única vez, evitando duplicatas. Em seguida, o NOT EXISTS é uma subconsulta correlacionada (usa x.bar, que vem de fora) que verifica, para cada bar, se já existe uma oferta da cerveja Forster naquele bar específico. Só são inseridas linhas para os bares que ainda não oferecem Forster, exatamente o que a questão pede.

d) Errada. Mesmo que a lógica do COUNT funcione para identificar bares sem Forster, o SELECT externo lê diretamente a tabela oferta, que tem várias linhas por bar. Isso causaria múltiplas inserções da mesma cerveja para o mesmo bar, gerando duplicatas.

e) Errada. Além do mesmo problema de duplicatas (lê oferta sem DISTINCT), a condição "oferta.bar = o.bar" dentro da subconsulta com IN torna o filtro inconsistente, pois mistura

correlação com o operador IN de forma que não filtra corretamente os bares que já oferecem Forster.

Gabarito: C

30. (FGV/ES (SEMSA Manaus)/Pref Manaus/Analista Desenvolvimento de Sistemas/2022) Para a próxima questão, considere um banco de dados relacional que contém três tabelas, a saber. frequencia (cliente, bar)

preferencia (cliente, cerveja)

oferta (bar, cerveja)

A relação frequencia registra o(s) bar(es) que cada cliente frequenta, e tem dois atributos: nome do cliente e nome do bar;

A relação preferencia armazena a(s) cerveja(s) preferida(s) por cada cliente e tem dois atributos: nome do cliente e nome da cerveja;

A relação oferta relaciona a(s) cerveja(s) que cada bar serve, também com dois atributos: nome do bar e nome da cerveja. É dado que todo cliente frequenta pelos menos um bar, que nem todos os clientes têm preferências, que todo bar oferece pelo menos uma cerveja e que toda cerveja é oferecida pelo menos por um bar.

Com relação ao banco de dados especificado anteriormente, considere que houve um erro de carga, de forma que há bares presentes na tabela frequencia que não aparecem na tabela oferta (que deveria conter todos os bares).

Assinale o comando SQL que deletaria da tabela frequencia as linhas referentes a esses bares.

a) delete from frequencia

where oferta.bar <> frequencia.bar

b) delete from frequencia

from oferta

where oferta.bar <> frequencia.bar

c) delete from frequencia

where bar in (select bar from oferta)

d) delete from frequencia

where not exists

(select * from oferta f

where f.bar = frequencia.bar)

e) delete from frequencia

where bar = (select bar from oferta

where bar is null)

Comentários:

a) Errada. A consulta tenta referenciar a tabela oferta no WHERE sem que ela tenha sido incluída no comando, resultando em uma sintaxe inválida. Não há como comparar oferta.bar sem trazer essa tabela para a consulta.

b) Errada. O uso de FROM adicional após o DELETE não é uma sintaxe padrão suportada, e mesmo que fosse, a condição oferta.bar <> frequencia.bar deletaria registros sempre que houvesse qualquer bar diferente em oferta, removendo praticamente tudo da tabela frequencia.

c) Errada. Esse comando faz o oposto do que se pede: deletaria as linhas de frequencia cujos bares aparecem em oferta, mantendo justamente os bares problemáticos (que estão em frequencia mas não em oferta).

d) Correta. O NOT EXISTS com subconsulta correlacionada verifica, para cada linha de frequencia, se existe ao menos uma linha em oferta com o mesmo bar. Se não existir nenhuma (NOT EXISTS retorna verdadeiro), aquela linha é deletada. Assim, removem-se exatamente os bares que estão em frequencia mas não em oferta.

e) Errada. A subconsulta busca bares onde bar IS NULL, o que normalmente não retornará nada, e ainda compara com igualdade simples (=) a uma subconsulta que pode retornar múltiplos valores, resultando em erro ou em nenhuma linha deletada.

Gabarito: D

31. (FGV/AS (SEMSA Manaus)/Pref Manaus/Programador de Computador/

T1	
A	B
1	3
2	2
4	2

T2	
C	D
100	42
110	55
120	33

T3		
A	C	E
1	100	12
1	110	12
2	100	NULL

Em todos os comandos, supõe-se que NULL é tratado como um valor desconhecido (unknown). Com referência ao banco de dados EXEMPLO, especificado anteriormente, analise o comando SQL a seguir.

delete from T1

where exists (select * from T3 x, T3 y

where x.A = T1.A and y.A = T1.A

and x.E + y.E = 12)

Sobre os efeitos desse comando sobre a tabela T1, é correto afirmar que

- a) nenhuma linha é deletada.
- b) apenas a primeira linha é deletada.
- c) apenas a segunda linha é deletada.
- d) apenas a terceira linha é deletada.
- e) todas as linhas são deletadas.

Comentários:

a) **Correta.** Nenhuma linha satisfaz a condição do EXISTS, pois nenhuma combinação possível em T3 produz soma igual a 12 (ou envolve NULL, que resulta em desconhecido).

b) **Errada.** A primeira linha (A=1) não é deletada, pois $12+12=24$, não 12.

c) **Errada.** A segunda linha (A=2) não é deletada, pois a soma envolve NULL, resultando em desconhecido.

d) **Errada.** A terceira linha (A=4) não é deletada, pois não há nenhum registro em T3 com A=4.

e) **Errada.** Nenhuma linha satisfaz a condição, então nenhuma é deletada.

Gabarito: A

32. (FGV/AJ TRT16/TRT 16/Apoio Especializado/Tecnologia da Informação/ ATENÇÃO: Nas próximas três questões, considere o esquema relacional de um banco de dados numa instalação SQL Server na qual os nulos são tratados como valores desconhecidos (unknown).

```
create table X(A int not null primary key,  
              B int)  
create table Y(A int not null UNIQUE,  
              constraint fk  
              foreign key (A) references X(A)  
              on delete cascade)
```

Para todos os efeitos, suponha que o número de linhas em cada tabela é diferente de zero.

- a) `select * FROM X
WHERE not exists (select * FROM Y where X.A=Y.A)`
- b) `select * FROM X
WHERE not exists (select * FROM Y where X.A<>Y.A)`
- c) `select * FROM X`

WHERE X.B IS NULL

d) select * FROM Y

WHERE not exists (select * FROM X where X.A=Y.A)

e) select * FROM Y

WHERE not exists (select * FROM X where X.A<>Y.A)

Comentários:

a) Errada. Essa consulta retorna as linhas de X cujo valor de A não aparece em Y, ou seja, registros de X sem correspondência em Y. Não é o que se busca como conjunto vazio garantido, pois podem existir linhas em X que não tenham par em Y.

b) Errada. O NOT EXISTS aqui só seria verdadeiro se todas as linhas de Y tivessem A igual ao X.A avaliado, o que não acontece de forma geral, então o resultado depende dos dados, não sendo necessariamente vazio.

c) Errada. Essa consulta retorna as linhas de X em que B é nulo, e como B aceita nulos, podem existir várias linhas assim. O resultado depende dos dados.

d) Correta. Como existe uma chave estrangeira de Y(A) referenciando X(A) com ON DELETE CASCADE, toda linha de Y obrigatoriamente possui um A correspondente em X. Assim, o NOT EXISTS (select * FROM X where X.A=Y.A) nunca será verdadeiro para nenhuma linha de Y, retornando sempre um conjunto vazio. Lembrando do conteúdo da aula, o EXISTS é um teste binário de existência: se a subconsulta retorna ao menos uma linha, é TRUE; o NOT EXISTS inverte essa lógica.

e) Errada. O NOT EXISTS só seria verdadeiro se todas as linhas de X tivessem A igual ao Y.A avaliado, o que dificilmente ocorre quando X tem várias linhas distintas. Mesmo assim, o resultado depende dos dados, não sendo garantidamente vazio como em D.

Gabarito: D

33. (FGV/AT (TCE TO)/TCE TO/Tecnologia da Informação/2022) Caio preparou o comando SQL exibido a seguir para deletar um conjunto de linhas de uma tabela.

```
delete from T1
```

```
where exists(select * from T2
```

```
where T1.A1 = T2.A2)
```

Para testar seu comando e descobrir exatamente quais seriam as linhas deletadas, Caio pretende executar um comando sem alteração de dados que permita a identificação dessas linhas. Para isso, basta que Caio substitua no comando original o termo:

a) "=" por "<>";

b) "delete from" por "show deleted rows from";

- c) "delete" por "delete limit 0";
- d) "delete" por "select *";
- e) "exists" por "not exists".

Comentários:

a) Errada. Trocar o "=" por "<>" mudaria a lógica da subconsulta correlacionada, fazendo com que ela retornasse linhas diferentes das que seriam deletadas, não identificando corretamente as linhas alvo.

b) Errada. Não existe o comando "show deleted rows from" no SQL; é uma sintaxe inventada e inválida.

c) Errada. "delete limit 0" não é uma forma adequada para visualizar as linhas que seriam deletadas, e além disso ainda seria um comando de deleção, não um comando de simples consulta.

d) Correta. Para identificar as linhas que seriam deletadas sem alterar os dados, basta trocar o "delete" por "select *", mantendo o restante do comando. Assim, o WHERE com EXISTS continua filtrando exatamente as mesmas linhas, mas agora elas são apenas exibidas pelo SELECT, sem serem removidas. É um teste binário de existência: para cada linha de T1, verifica-se se existe ao menos uma linha em T2 que satisfaça a condição correlacionada.

e) Errada. Trocar "exists" por "not exists" inverteria a lógica, retornando justamente as linhas que NÃO seriam deletadas, e não as que seriam.

Gabarito: D

34. (FGV/Tec Leg (CM Taubaté)/CM Taubaté/Informática/2022) Na empresa onde João trabalha existe uma tabela PESSOA, com as colunas CPF, Nome e DataAdmissão no banco de dados de pessoal. Ocorre que houve um erro de programação que permitiu que fossem inseridas linhas indevidamente, com diferentes nomes para um mesmo CPF, como se nota num trecho da tabela exibido a seguir.

CPF	Nome	DataAdmissão
100.100.100-10	A	10/03/2011
100.100.100-10	B	12/04/2012
200.200.200-20	C	10/07/2015
300.300.300-30	D	10/03/2020
300.300.300-30	D	10/03/2010
300.300.300-30	D	10/10/2019
400.400.400-40	E	10/03/2010

A tabela corrigida ficaria como mostrada abaixo.

CPF	Nome	DataAdmissão
100.100.100-10	A	10/03/2011
200.200.200-20	C	10/07/2015

300.300.300-30	D	10/03/2010
400.400.400-40	E	10/03/2010

Dado que a linha correta em cada caso é aquela que tem a menor data de admissão, assinale o comando SQL que João pode utilizar para “limpar” esses dados.

a) delete from pessoa

where not exists

(select * from pessoa p

where pessoa.CPF=p.CPF

and pessoa.nome=p.nome

and pessoa.dataadmissãop.>dataadmissão)

b) delete from pessoa

where not exists

(select * from pessoa p

where pessoa.CPF=p.CPF

and pessoa.nome<>p.nome

and pessoa.dataadmissão<=p.dataadmissão)

c) delete from pessoa

where exists

(select * from pessoa p

where pessoa.CPF=p.CPF)

and pessoa.dataadmissão<p.dataadmissão

d) delete from pessoa

where exists

(select * from pessoa p

where pessoa.CPF=p.CPF

and pessoa.dataadmissão>p.dataadmissão)

e) delete from pessoa

where exists

```
(select * from pessoa p
where pessoa.CPF=p.CPF
and pessoa.nome<>p.nome
and pessoa.dataadmissão<p.dataadmissão)
```

Comentários:

a) Errada. A sintaxe está confusa e errada (com "pessoa.dataadmissãop.>dataadmissão"). Além disso, usar NOT EXISTS exigiria que não houvesse nenhuma linha em determinada condição, o que não capta corretamente a regra de manter a menor data.

b) Errada. O NOT EXISTS aqui exigiria que não existisse outra linha com mesmo CPF, nome diferente e data menor ou igual, o que não corresponde à lógica de manter a linha com a menor data de admissão para cada CPF.

c) Errada. A condição "pessoa.dataadmissão<p.dataadmissão" está fora do EXISTS (após o parêntese de fechamento), o que faz a comparação ficar solta e sem correlação correta. A estrutura está mal formada.

d) Correta. Aqui temos uma subconsulta correlacionada com EXISTS: para cada linha de pessoa, verificamos se existe outra linha com mesmo CPF e data de admissão menor. Se existir, significa que a linha avaliada não é a de menor data e deve ser deletada. Assim sobram apenas as linhas com a menor data de admissão para cada CPF, exatamente como pede a questão.

e) Errada. A condição "pessoa.nome<>p.nome" exige nomes diferentes, o que não atende ao caso do CPF 300.300.300-30, em que todos os nomes são iguais (D). Linhas com mesmo CPF e mesmo nome não seriam deletadas, mantendo duplicatas indevidas.

Gabarito: D

35. (FGV/AJ TRT13/TRT 13/Apoio Especializado/Tecnologia da Informação/2022) Considere uma tabela relacional X com uma única coluna A, cuja instância contém uma linha apenas, e os comandos SQL a seguir.

1

```
delete from X
where exists (select * from X x2
              where x2.A = X.A)
```

2

```
delete from X
where not exists (select * from X x2
```

where $x2.A = X.A$)

Nesse contexto, analise as afirmativas sobre os efeitos desses comandos.

- I. Nenhuma linha seria deletada na execução do comando 1.
- II. Uma linha seria deletada na execução do comando 1.
- III. Nenhuma linha seria deletada na execução do comando 2.
- IV. Uma linha seria deletada na execução do comando 2.
- V. Os dois comandos gerariam erros de execução.

Está correto o que se afirma em

- a) I e III, apenas.
- b) I e IV, apenas.
- c) II e III, apenas.
- d) II e IV, apenas.
- e) V, apenas.

Comentários:

I. Errada. No comando 1, temos uma subconsulta correlacionada com EXISTS que verifica se existe alguma linha em X cujo valor de A seja igual ao valor de A da linha avaliada. Como a tabela tem uma linha, ao comparar essa linha com ela mesma, a condição é satisfeita (existe pelo menos uma linha), o EXISTS retorna TRUE e a linha é deletada.

II. Correta. Pelo raciocínio acima, a única linha existente na tabela satisfaz a condição do EXISTS (ela mesma faz o EXISTS retornar TRUE), então essa linha é deletada.

III. Correta. No comando 2, o NOT EXISTS só retornaria TRUE se não existisse nenhuma linha em X com A igual ao A da linha avaliada. Como a própria linha avaliada satisfaz $x2.A = X.A$, o EXISTS retorna TRUE e o NOT EXISTS retorna FALSE, portanto nenhuma linha é deletada.

IV. Errada. Pelo raciocínio do item III, nenhuma linha é deletada no comando 2.

V. Errada. Os dois comandos são sintaticamente válidos e executam normalmente, sem gerar erro.

Gabarito: C

36. (FGV/Per Cri (PCA AP)/PCA AP/Ciência da Computação e Analista de sistema/2022) Numa prova de concurso, a organizadora estava preocupada com a ocorrência de homônimos fazendo prova na mesma sala, já que isso poderia ocasionar contratempos.

Para uma tabela relacional T, contendo as colunas CPF, Nome e CodSala, assinale o comando SQL que identifica corretamente os homônimos em cada sala.

a) SELECT CodSala, Nome, CPF

FROM T

WHERE EXISTS (SELECT * FROM T t1

WHERE t1.CodSala = T.CodSala

and t1.CPF <> T.CPF)

ORDER BY 1, 2, 3

b) SELECT CodSala, Nome, CPF

FROM T

WHERE EXISTS (SELECT * FROM TT

WHERE TT.CodSala = T.CodSala

and TT.Nome <> T.Nome

and TT.CPF = T.CPF)

ORDER BY 1, 2, 3

c) SELECT CodSala, Nome, CPF

FROM T

WHERE t1.CodSala = T.CodSala

and t1.Nome = T.Nome

and t1.CPF <> T.CPF)

ORDER BY 1, 2, 3

d) SELECT CodSala, Nome, CPF

FROM T

WHERE NOT EXISTS (SELECT * FROM T t1

WHERE t1.CodSala = T.CodSala

and t1.Nome = T.Nome

and t1.CPF <> T.CPF)

ORDER BY 1, 2, 3

e) SELECT CodSala, Nome, CPF

FROM T

WHERE EXISTS (SELECT * FROM T t1

WHERE t1.CodSala = T.CodSala

and t1.Nome = T.Nome

and t1.CPF <> T.CPF)

ORDER BY 1, 2, 3

Comentários:

a) Errada. A condição testa apenas se existe alguém na mesma sala com CPF diferente, sem comparar o nome. Isso retornaria praticamente todas as pessoas, pois quase sempre existe outra pessoa diferente na mesma sala, não identificando especificamente os homônimos.

b) Errada. A condição exige que o nome seja diferente (TT.Nome <> T.Nome) e o CPF igual (TT.CPF = T.CPF), o que é o oposto do que define um homônimo. Homônimo é mesmo nome com CPF diferente, não nome diferente com mesmo CPF.

c) Errada. A consulta usa o alias t1 sem defini-lo no FROM (não há subconsulta com EXISTS aqui), tornando o comando sintaticamente inválido. Também há um parêntese de fechamento sobrando.

d) Errada. O NOT EXISTS inverte a lógica: seleciona pessoas para as quais NÃO existe outra com mesmo nome, mesma sala e CPF diferente. Ou seja, retornaria justamente quem NÃO tem homônimo na sala, o oposto do que se pede.

e) Correta. O EXISTS é um teste binário de existência usado em subconsulta correlacionada (perceba que a subconsulta interna referencia a tabela T externa). Aqui ele verifica, para cada linha de T, se existe outra linha na mesma sala (t1.CodSala = T.CodSala), com o mesmo nome (t1.Nome = T.Nome) e CPF diferente (t1.CPF <> T.CPF). Isso é exatamente a definição de homônimo: pessoas distintas (CPF's diferentes) com mesmo nome na mesma sala.

Gabarito: E

37. (FGV/Ana Esp (IMBEL)/IMBEL/Analista de Sistemas/2021) Considere a instância da tabela R1 e o comando SQL exibidos a seguir.

A	B
1	2
2	2
3	3
4	3
4	2
4	1

5	0
---	---

select distinct A

from R1

where exists

(select * from R1 x where x.B > r1.A)

Assinale a lista de números que é exibida quando esse comando SQL é executado.

- a) 5
- b) 1, 2
- c) 2, 3
- d) 3, 4, 5
- e) 1, 2, 3, 4

Comentários:

- a) **Errada.** O valor 5 nunca entraria, pois não existe B maior que 5 na tabela.
- b) **Correta.** Apenas os valores de A iguais a 1 e 2 satisfazem a condição de existir algum B maior que eles (B=3).
- c) **Errada.** O valor 3 não entra, pois não há nenhum B maior que 3 na tabela.
- d) **Errada.** Justamente os valores 3, 4 e 5 são os que falham no teste do EXISTS.
- e) **Errada.** Os valores 3 e 4 não satisfazem a condição, pois não existe B maior que eles.

Gabarito: B

38. (FGV/ATCE (TCE,AM)/TCE AM/Auditoria de Tecnologia da Informação/2021) Na questão, considere uma tabela de banco de dados T cuja instância é exibida a seguir.

A	B	C	D
12	2	3	1
14	3	8	2
18	2	9	3
21	5	4	4

Considerando-se a instância da tabela T (descrita anteriormente), analise o comando SQL abaixo.

update T

```
set a = a + 32
```

```
where
```

```
exists (select * from T t2 where T.c > t2.D)
```

O número de registros da tabela T afetados pela execução desse comando é:

- a) zero;
- b) um;
- c) dois;
- d) três;
- e) quatro.

Comentários:

- a) **Errada.** Não é zero, pois todas as linhas atendem à condição.
- b) **Errada.** Não é apenas uma linha, pois várias linhas satisfazem o EXISTS.
- c) **Errada.** Não são duas linhas, pois mais linhas atendem à condição.
- d) **Errada.** Não são três linhas, pois todas as quatro são afetadas.
- e) **Correta.** Todas as quatro linhas têm C maior que pelo menos um valor de D, então o EXISTS é verdadeiro para todas e o UPDATE afeta os quatro registros.

Gabarito: E

39. (FGV/Ana TI (BANESTES)/BANESTES/Desenvolvimento de Sistemas/2021) Na questão, considere as tabelas T1, T2 e T3, cujas estruturas e instâncias são exibidas a seguir. O valor NULL deve ser tratado como unknown (desconhecido).

T1	
A	B
1	3
2	2
4	2

T2	
C	D
100	42
110	55
120	33

T3		
A	C	E
1	100	20
1	110	40
2	120	10
2	100	NULL
2	110	10

Tomando como referência as tabelas T1, T2 e T3, descritas anteriormente, o comando SQL

```
select t1.*
```

```
from T1
```

```
where not exists
```

```
(select * from T2, T3
```

```
where t1.A = t3.A and t2.C = t3.C
```

```
and t3.E is null)
```

produz como resultado somente a(s) linha(s):

a) 1, 3

b) 1, 3 / 4, 2

c) 4, 2

d) 1, 3 / 2, 2 / 4, 2

e) 2, 2 / 4, 2

Comentários:

a) Errada. Faltou a linha (4,2), que também não tem correspondência com E nulo em T3.

b) Correta. As linhas (1,3) e (4,2) são as únicas em que a subconsulta não encontra correspondência com t3.E nulo, fazendo NOT EXISTS ser verdadeiro.

c) Errada. A linha (1,3) também deve aparecer, pois A=1 não tem correspondência com E nulo em T3.

d) Errada. A linha (2,2) é justamente a que deve ser excluída, pois para A=2 existe a linha (2,100,NULL) em T3.

e) Errada. A linha (2,2) deve ser excluída e a linha (1,3) deveria estar presente.

Gabarito: B

40. (FGV/AJ (TJ AP)/TJ AP/Apoio Especializado/Tecnologia da Informação Banco de Dados/2024) Quando referenciadas, considere as tabelas relacionais Competidor e Disputa, cujas estruturas e instâncias são descritas abaixo. Todas as colunas são definidas como strings.

A tabela Disputa contém as disputas realizadas entre competidores que aparecem na tabela Competidor. Em cada disputa há dois competidores, um com camisa azul e outro com camisa verde.

Competidor

Nome

A
B
C
D
E
F

Disputa

Azul	Verde
A	B
C	A
B	A
C	E
F	A
F	D

Considerando as tabelas Competidor e Disputa, descritas anteriormente, analise o comando SQL a seguir.

```
delete from Competidor  
where (select sum(1)  
from Disputa d where d.azul = Nome)  
< (select sum(1) from Disputa d  
where d.verde = Nome)
```

O número de linhas removidas na execução do comando acima é:

- a) 6;
- b) 4;
- c) 2;
- d) 1;
- e) 0.

Comentários:

- a) **Errada.** Não são todos os competidores que satisfazem a condição.
- b) **Errada.** Apenas um competidor (A) tem aparições como Azul menores que como Verde.
- c) **Errada.** Somente A se enquadra na condição estritamente menor.

d) **Correta.** Apenas o competidor A é removido, pois é o único cujo total como Azul (2) é menor que o total como Verde (3).

e) **Errada.** O competidor A é efetivamente removido pelo comando.

Gabarito: D

41. (FGV/AJ (TJ AP)/TJ AP/Apoio Especializado/Tecnologia da Informação Desenvolvimento de Sistemas/2024) Quando referenciadas, considere as tabelas relacionais Competidor e Disputa, cujas estruturas e instâncias são descritas abaixo. Todas as colunas são definidas como strings. A tabela Disputa contém as disputas realizadas entre competidores que aparecem na tabela Competidor. Em cada disputa há dois competidores, um com camisa azul e outro com camisa verde.

Competidor

Nome
A
B
C
D
E
F

Disputa

Azul	Verde
A	B
C	A
B	A
C	E
F	A
F	D

João tem pouca experiência com SQL, mas precisa de uma consulta que exiba os competidores que têm o mesmo número de disputas com as camisas azul e verde. João escreveu três scripts, utilizando as tabelas Competidor e Disputa, como definidas anteriormente, e tentou a sorte.

```
select distinct c.nome  
from Competidor c, Disputa d  
group by c.nome  
having count(distinct d.azul)
```

```
= count(distinct d.verde)
```

```
select c.nome  
from Competidor c  
where (select sum(1)  
       from Disputa d where d.azul = c.nome)  
      = (select sum(1)  
         from Disputa d where d.verde = c.nome)
```

```
select distinct c.nome  
from Competidor c, Disputa d  
where (select sum(1) where d.azul = c.nome)  
      = (select sum(1) where d.verde = c.nome)
```

Dado que a resposta correta deve exibir somente o competidor B, conclui-se que:

- a) nenhum dos scripts funciona;
- b) somente o primeiro script funciona;
- c) somente o segundo script funciona;
- d) somente o terceiro script funciona;
- e) os três scripts funcionam.

Comentários:

a) Errada. O segundo script funciona corretamente, então essa alternativa está incorreta.

b) Errada. O primeiro script faz um produto entre Competidor e Disputa sem cláusula de junção (sem WHERE relacionando as tabelas) e usa COUNT(DISTINCT), que conta apenas valores distintos das colunas azul e verde, e não a quantidade de disputas de cada competidor, produzindo resultado errado.

c) Correta. O segundo script usa duas subconsultas correlacionadas no WHERE: a primeira soma quantas vezes o nome do competidor aparece na coluna azul (d.azul = c.nome) e a segunda soma quantas vezes aparece na coluna verde (d.verde = c.nome). Ao comparar os dois valores com igualdade, retorna apenas os competidores cujo número de aparições com camisa azul é igual ao número com camisa verde, que é exatamente o competidor B.

d) **Errada.** O terceiro script tenta usar `select sum(1) where ...` sem cláusula `FROM`, ou seja, sem consultar a tabela `Disputa` dentro da subconsulta, o que torna o comando inválido e incapaz de contar as disputas corretamente.

e) **Errada.** Apenas o segundo script funciona, conforme explicado nas demais alternativas.

Gabarito: C

42. (FGV/ATRFB/SRFB/Geral/2023) Num banco de dados relacional, considere uma tabela `R`, com duas colunas `A` e `B`, ambas do tipo `string` de caracteres, cuja instância é exibida a seguir.

A	B
Pedro	João
Maria	Ida
Maria	Ida
Pedro	João
Edson	Wilson
Edson	Maria

Nesse cenário analise os comandos a seguir.

I.

```
DELETE FROM R
```

```
WHERE EXISTS (SELECT * FROM R r1  
              WHERE R.A = r1.A and R.B = r1.B)
```

II.

```
DELETE FROM R
```

```
WHERE EXISTS (SELECT * FROM R r1  
              WHERE R.A + R.B > r1.A + r1.B)
```

III.

```
DELETE FROM R
```

```
WHERE R.A + R.B in (SELECT A + B FROM R)
```

Assinale a lista que contém o número de registros deletados em cada um dos comandos I, II e III, respectivamente, quando executados separadamente e usando a mesma instância inicial descrita.

- a) 2, 2 e 0.
- b) 2, 4 e 0.
- c) 4, 4 e 4.
- d) 6, 5 e 6.
- e) 6, 6 e 6.

Comentários:

I. Deleta 6 registros. O EXISTS verifica, para cada linha da tabela R, se existe alguma linha na subconsulta (apelidada de r1) que tenha os mesmos valores de A e B. Como toda linha sempre encontra a si mesma na subconsulta, o EXISTS retorna verdadeiro para todas as 6 linhas, deletando todas.

II. Deleta 5 registros. O EXISTS verifica, para cada linha, se existe alguma outra linha cuja concatenação A+B seja maior (em ordem alfabética). Apenas a linha com o maior valor combinado de A+B (PedroJoão, considerando ordem alfabética dos nomes presentes) não terá nenhuma outra maior que ela, então não será deletada. As outras 5 linhas serão deletadas.

III. Deleta 6 registros. O operador IN verifica se o valor A+B de cada linha aparece na lista retornada pela subconsulta (que retorna A+B de todas as linhas da tabela). Como toda linha sempre encontra seu próprio A+B na lista retornada, todas as 6 linhas satisfazem o filtro e são deletadas.

Gabarito: D

43. (FGV/ACE (TCE SP)/TCE SP/Tecnologia da Informação (TI)/2023) Atenção

Quando referidas, considere as tabelas relacionais TX e TY, criadas e instanciadas com o script SQL a seguir.

```
create table TY(C int primary key not null, A int)
```

```
create table TX(A int primary key not null, B int,  
               foreign key (B) references TY(C)  
               on delete cascade  
               )
```

```
insert into TY values (1,0)  
insert into TY(C) values (2)  
insert into TY(C) values (3)
```

```
insert into TY values (5,NULL)
insert into TY values (6,NULL)
```

```
insert into TX values (1,2)
insert into TX values (2,1)
insert into TX values (3,2)
insert into TX values (4,2)
```

Com referência às tabelas TX e TY, como descritas anteriormente, analise o comando SQL a seguir.

```
insert into TX(A, B)
select C,A FROM TY
where C not in (select A from TX)
      or A in (select A from TX)
```

O conjunto de linhas inseridas é:

- a) vazio;
- b) 0, NULL;
- c) 2, NULL e 3, NULL;
- d) 5, NULL e 6, NULL;
- e) 6, NULL.

Comentários:

a) Errada. O conjunto não é vazio, pois há linhas em TY que satisfazem as condições da subconsulta e são inseridas em TX.

b) Errada. Não existe linha (0, NULL) sendo inserida, pois a tabela TY não possui valor 0 na coluna C.

c) Errada. Os valores 2 e 3 da coluna C de TY já aparecem na coluna A de TX (que tem valores 1, 2, 3, 4), então não passam no filtro C NOT IN (select A from TX), e o A correspondente dessas linhas (que está vazio/NULL) também não pertence aos A de TX.

d) Correta. Avaliando linha por linha de TY: para C=5, A=NULL, o valor 5 não está em A de TX (passa no NOT IN), então a linha (5, NULL) é inserida; para C=6, A=NULL, o valor 6 também não está em A de TX, então a linha (6, NULL) é inserida. As linhas com C=1, 2, 3 não passam porque seus valores de C já existem em A de TX e seus A são 0 ou NULL, não pertencendo aos A de TX.

e) **Errada.** Não é apenas a linha (6, NULL); a linha (5, NULL) também é inserida porque C=5 não está presente na coluna A de TX.

Gabarito: D

44. (FGV/AS (SEMSA Manaus)/Pref Manaus/Programador de Computador/

T1	
A	B
1	3
2	2
4	2

T2	
C	D
100	42
110	55
120	33

T3		
A	C	E
1	100	12
1	110	12
2	100	NULL

Em todos os comandos, supõe-se que NULL é tratado como um valor desconhecido (unkown). Com referência ao banco de dados EXEMPLO, especificado anteriormente, a execução do comando SQL

```
select A, count(*)  
from T1  
where A = (select max(A) from T1)  
group by A  
having max(A) > (select max(A) from T3)  
produz como resultado
```

- a) uma linha, com os valores 4, NULL.
- b) uma linha, com os valores 4, 1.
- c) uma linha, com os valores 2, 2.
- d) duas linhas, com os valores 4, 2 e 2, 2.
- e) duas linhas, com os valores 4, 2 e 1, 3.

Comentários:

a) **Errada.** O count(*) nunca retorna NULL; quando há uma linha agrupada, ele conta pelo menos 1.

b) **Correta.** A subconsulta (select max(A) from T1) retorna 4, então o WHERE filtra apenas as linhas de T1 onde A = 4, que é apenas uma linha. O GROUP BY A agrupa essa única linha, e o HAVING verifica se max(A) > max(A) de T3; como max(A) de T3 é 2 e 4 > 2, a condição é satisfeita, retornando uma linha com A=4 e count(*)=1.

c) **Errada.** O valor 2 não é o máximo de A em T1 (o máximo é 4), portanto não passa pelo filtro do WHERE.

d) **Errada.** O WHERE filtra apenas as linhas em que A é igual ao máximo de A, ou seja, apenas A=4, então não há como retornar duas linhas.

e) **Errada.** Pelo mesmo motivo, o WHERE só permite as linhas com A igual ao máximo (4), descartando as linhas com A=1 ou A=2.

Gabarito: B

45. (FGV/Ana (FunSaúde CE)/FunSaúde CE/II e Infraestrutura de Informática/2021) Considere a definição e as instâncias das tabelas de bancos de dados CLUBE e JOGO exibidas a seguir.

nome
Barcelona
Boca Juniors
The Strongest

JOGO

mandante	visitante	golsM	golsV
Barcelona	Boca Juniors	1	0
Barcelona	The Strongest	NULL	NULL
Boca Juniors	Barcelona	0	0
Boca Juniors	The Strongest	3	0
The Strongest	Barcelona	2	0
The Strongest	Boca Juniors	2	0

Cada clube deve jogar quatro vezes, duas como mandante e duas como visitante.

As colunas golsM e golsV registram o número de gols dos times mandantes e visitantes, respectivamente, em cada jogo. Ambas são nulas enquanto o jogo não for realizado.

Análise o comando SQL a seguir, à luz das definições e instâncias das tabelas CLUBE e JOGO, definidas anteriormente.

```
select c.nome from CLUBE c where
```

```
(select count(*) from JOGO j
```

```
where c.nome = j.mandante) <> 2
```

```
or
```

```
(select count(*) from JOGO j
```

where c.nome = j.visitante) <> 2

O resultado produzido pela execução desse comando é a lista de todos os clubes que

a) aparecem em quatro jogos.

b) não aparecem em quatro jogos.

c) não aparecem em dois jogos como mandante ou que não aparecem em dois jogos como visitante.

d) aparecem em dois jogos como mandante ou que aparecem em dois jogos como visitante.

e) aparecem em dois jogos como mandante e aparecem em dois jogos como visitante.

Comentários:

a) Errada. O comando não conta o total de aparições do clube nos jogos somando mandante e visitante, ele faz duas verificações separadas e usa o operador <> (diferente), então não retorna simplesmente quem aparece em quatro jogos.

b) Errada. Não basta o clube não aparecer em quatro jogos, pois o SQL avalia separadamente a contagem como mandante e como visitante, retornando o clube se qualquer uma delas for diferente de 2.

c) Correta. O comando usa duas subconsultas correlacionadas (referenciam c.nome da consulta externa): a primeira conta quantas vezes o clube aparece como mandante e verifica se é diferente de 2; a segunda faz o mesmo para visitante. Com o OR ligando as duas, basta uma das condições ser verdadeira para o clube ser retornado, ou seja, retorna os clubes que não aparecem em dois jogos como mandante ou que não aparecem em dois jogos como visitante.

d) Errada. O operador usado é <> (diferente de 2) e não = 2, então a lógica é exatamente a oposta dessa alternativa.

e) Errada. Essa alternativa descreve clubes que aparecem exatamente duas vezes em cada papel, mas o comando retorna justamente o contrário, quem não cumpre essa condição em pelo menos um dos papéis.

Gabarito: C

46. (FGV/ACP (CGE PB)/CGE PB/Auditoria de Tecnologia da Informação/2024) Tabelas Relacionais T1 e T2

Considere as tabelas relacionais T1 e T2, de acordo com as colunas e as instâncias abaixo descritas. Na execução dos comandos SQL, assuma que o mecanismo do banco de dados considera valores NULL como valores desconhecidos (unknown).

Tabela T1

P	N
1	10

2	40
3	20
4	10

Tabela T2

E	Q	P
1	10	2
2	40	3
3	20	8
4	10	NULL

Considerando as tabelas T1 e T2, anteriormente apresentadas, analise o comando SQL a seguir.
delete from T2 where P not in (select P from T2)

O número de linhas deletadas da tabela T2 pela execução desse comando é:

- a) 0;
- b) 1;
- c) 2;
- d) 3;
- e) 4.

Comentários:

a) Correta. O comando deleta as linhas de T2 cujo valor de P não está na lista de valores de P da própria T2. Como a subconsulta retorna exatamente os valores de P existentes em T2 (2, 3, 8 e NULL), todo P de T2 está nessa lista, então nenhuma linha satisfaz o "not in" e nada é deletado.

b) Errada. Não há uma linha deletada, pois todos os valores de P em T2 aparecem na própria subconsulta.

c) Errada. Não há duas linhas deletadas, já que o filtro "not in" não é satisfeito por nenhuma linha.

d) Errada. Não há três linhas deletadas, pelo mesmo motivo: nenhum P de T2 está fora da lista retornada pela subconsulta.

e) Errada. Não há quatro linhas deletadas, pois o comando não exclui nenhuma linha de T2.

Gabarito: A

LISTA DE QUESTÕES



1. (FGV/AAD (TCE,RR)/TCE RR/Tecnologia da Informação/Banco de Dados/2025) O código DDL abaixo cria três tabelas em um banco de dados relacional:

```
CREATE TABLE Velejadores (  
    vid INTEGER PRIMARY KEY,  
    nome VARCHAR(100),  
    idade INTEGER  
);  
CREATE TABLE Barcos (  
    bid INTEGER PRIMARY KEY,  
    nome VARCHAR(50),  
    modelo VARCHAR(50)  
);  
CREATE TABLE Reservas (  
    vid INTEGER,  
    bid INTEGER,  
    data DATE,  
    PRIMARY KEY (vid, bid),  
    FOREIGN KEY (vid) REFERENCES Velejadores(vid),  
    FOREIGN KEY (bid) REFERENCES Barcos(bid)  
);
```

Considerando as tabelas acima, a seguinte consulta foi realizada:

```
SELECT V.nome FROM Velejadores V  
WHERE NOT EXISTS (  
    (SELECT B.bid FROM Barcos B)  
    EXCEPT  
    (SELECT R.bid FROM Reservas R  
    WHERE R.vid = V.vid)  
);
```

O retorno da consulta foram os nomes

- a) dos velejadores que possuem reservas para todos os barcos.
- b) dos barcos que não possuem reservas.
- c) dos velejadores que não possuem reservas.
- d) dos velejadores que possuem reserva para mais de um barco.
- e) dos barcos que possuem reservas feitas por todos os velejadores.

2. (FGV/ACP (CGE PB)/CGE PB/Auditoria de Tecnologia da Informação/2024) Tabelas Relacionais T1 e T2

Considere as tabelas relacionais T1 e T2, de acordo com as colunas e as instâncias abaixo descritas. Na execução dos comandos SQL, assuma que o mecanismo do banco de dados considera valores NULL como valores desconhecidos (unknown).

Tabela T1

P	N
1	10
2	40
3	20
4	10

Tabela T2

E	Q	P
1	10	2
2	40	3
3	20	8
4	10	NULL

À luz das tabelas T1 e T2, anteriormente apresentadas, analise o comando SQL exibido a seguir.

```
select case when
  exists (select * from T2
    where T2.E = 2
    and T2.P = 3
    and exists
      (select * from T1
        where T1.P in (2,3,4)
        and T2.E in (2,3)))
  then 1 else 0 end flag
```

Sobre uma eventual execução desse script, é correto afirmar que:

- a) haveria um erro de sintaxe na última linha, no termo "flag";
- b) seria produzido um resultado contendo somente a linha de títulos;
- c) seria produzido um resultado contendo, além da linha de títulos, uma linha com uma coluna com o valor 0;
- d) seria produzido um resultado contendo, além da linha de títulos, uma linha com uma coluna com o valor 1;
- e) haveria um erro de sintaxe na segunda linha, pois não é permitido o uso da cláusula "exists" logo após o termo "when".

3. (FGV/AJ (TJ AP)/TJ AP/Apoio Especializado/Tecnologia da Informação Banco de Dados/2024) Quando referenciadas, considere as tabelas relacionais Competidor e Disputa, cujas estruturas e instâncias são descritas abaixo. Todas as colunas são definidas como strings.

A tabela Disputa contém as disputas realizadas entre competidores que aparecem na tabela Competidor. Em cada disputa há dois competidores, um com camisa azul e outro com camisa verde.

Competidor

Nome
A
B
C
D
E
F

Disputa

Azul	Verde
A	B
C	A
B	A
C	E
F	A
F	D

Considerando as tabelas Competidor e Disputa, descritas anteriormente, analise os três comandos SQL a seguir.

I. select C.nome
from Competidor C

```
where exists (select * from Disputa D
              where D.azul = C.nome)
and exists (select * from Disputa D
            where D.verde = C.nome)
```

```
II. select C.nome
   from Competidor C
  where exists (select * from Disputa D
                where D.azul = C.nome
                   or D.verde = C.nome)
```

```
III. select C.nome
   from Competidor C
  where (select count(*) from Disputa D
         where D.azul = C.nome)
        + (select count(*) from Disputa D
           where D.verde = C.nome) > 1
```

O resultado

Nome
A
B

é obtido somente no(s) comando(s):

- a) I;
- b) II;
- c) III;
- d) I e II;
- e) II e III.

4. (FGV/AJ (TJ AP)/TJ AP/Apoio Especializado/Tecnologia da Informação Banco de Dados/2024) Quando referenciadas, considere as tabelas relacionais Competidor e Disputa, cujas estruturas e instâncias são descritas abaixo. Todas as colunas são definidas como strings.

A tabela Disputa contém as disputas realizadas entre competidores que aparecem na tabela Competidor. Em cada disputa há dois competidores, um com camisa azul e outro com camisa verde.

Competidor

Nome
A
B
c
D
E
F

Disputa

Azul	Verde
A	B
C	A
B	A
C	E
F	A
F	D

Considerando as tabelas Competidor e Disputa, descritas anteriormente, analise o comando SQL a seguir.

```
select nome
from Competidor C
where not exists (select *
from Disputa D
where D.azul = C.nome)
or not exists (select *
from Disputa D
where D.verde = C.nome)
```

A execução do comando acima produziria somente a lista completa dos competidores que:

- a) participaram em disputas com camisas das duas cores;
- b) nunca participaram em disputas com camisa verde;
- c) nunca participaram em disputas com camisa azul;
- d) nas disputas em que participaram, usaram sempre camisas da mesma cor;
- e) nunca participaram em disputas.

5. (FGV/AJ (TJ AP)/TJ AP/Apoio Especializado/Tecnologia da Informação Desenvolvimento

de Sistemas/2024) Quando referenciadas, considere as tabelas relacionais Competidor e Disputa, cujas estruturas e instâncias são descritas abaixo. Todas as colunas são definidas como strings. A tabela Disputa contém as disputas realizadas entre competidores que aparecem na tabela Competidor. Em cada disputa há dois competidores, um com camisa azul e outro com camisa verde.

Competidor

Nome
A
B
C
D
E
F

Disputa

Azul	Verde
A	B
C	A
B	A
C	E
F	A
F	D

Considerando as tabelas Competidor e Disputa, descritas anteriormente, analise o comando SQL abaixo.

```
select c1.nome, c2.nome  
from Competidor c1, Competidor c2  
where not exists  
      (select * from Disputa d  
       where d.azul = c2.nome  
             and d.verde = c1.nome)  
order by 1,2
```

Excetuados os títulos, o número de linhas produzidas pela execução desse comando é:

- a) 0;
- b) 6;

- c) 12;
- d) 30;
- e) 36.

6. (FGV/Ana Leg (ALEP)/ALEP/Desenvolvedor de Sistemas/2024) Considere o esquema relacional a seguir, implementado em SQL.

```
create table recurso
(
    id integer primary key,
    nome varchar(20) not null,
    valor real
);
create table projeto
(
    id integer primary key,
    nome varchar(20) not null,
    verba real
);
create table alocao
(
    id_recurso integer,
    id_projeto integer,
    primary key(id_recurso,id_projeto),
    foreign key(id_recurso) references recurso,
    foreign key(id_projeto) references projeto
);
```

Assinale a opção que apresenta a consulta que gera como resultado de execução uma lista com o nome dos recursos alocados em todos os projetos cadastrados.

- a) select r.nome from recurso r
- b) select r.nome from recurso r where not exists (select 1 from alocao a where a.id_recurso=r.id)
- c) select r.nome from recurso r where r.valor>(select avg(valor) from recurso)
- d) select r.nome from recurso r where not exists (select 1 from projeto p where not exists (select 0 from alocao a where a.id_recurso=r.id and a.id_projeto=p.id))
- e) select r.nome from recurso r where exists (select 1 from alocao a where a.id_recurso=r.id)

7. (FGV/Ana Leg (ALETO)/ALETO/Análise de Suporte em Informática/2024) Considere o

seguinte esquema de um banco de dados relacional, expresso em linguagem SQL:

```
create table editora
(
    cod_editora integer primary key,
    nome_editora varchar(30) not null,
    telefone char(11)
);
create table livro
(
    cod_livro integer primary key,
    num_isbn char(10) unique not null,
    titulo varchar(20) not null,
    edicao integer default 1 not null,
    cod_editora integer references editora
);
```

Após criadas, os comandos de inserção a seguir foram executados:

```
insert into editora values (1,'Ed1','2222-3333'), (2,'Ed2',NULL);
insert into livro values (1,'111','Livro1',2,NULL), (2,'222','Livro2',2,1), (5,'555','Livro5',1,1);
insert into livro(cod_livro,num_isbn,titulo,cod_editora) values (3,'333','Livro3',2),
(4,'444','Livro4',2);
```

A consulta `select * from livro where exists (select distinct * from editora) and edicao%2 <> 0` produz como resultado a seguinte quantidade de linhas:

- a) 0.
- b) 1.
- c) 2.
- d) 3.
- e) 4.

8. (FGV/Tec NS (TJ MS)/TJ MS/Analista de Sistemas Computacionais/Analista de Banco de Dados/2024) Considere as seguintes tabelas, com suas respectivas colunas, que se relacionam pela coluna FornecedorID:

Produto (ProdutoID, ProdutoNome, FornecedorID)

Fornecedor (FornecedorID, FornecedorNome)

Para retornar a lista, sem repetições, de Fornecedores que possuem Produtos com preços menores que 20, deve-se executar o comando SQL:

a) `SELECT FornecedorNome`
`FROM Fornecedor`
`WHERE EXISTS (SELECT ProdutoNome FROM Produto`
`WHERE Produto.FornecedorID =`
`Fornecedor.FornecedorID AND Preco < 20)`

b) `SELECT COUNT(FornecedorID), FornecedorNome`
`FROM Fornecedor, Produto`
`GROUP BY FornecedorID`
`HAVING COUNT(Preco) < 20`

c) `SELECT FornecedorNome`
`FROM Fornecedor`
`INNER JOIN Produto ON Produto.FornecedorID =`
`Fornecedor.FornecedorID AND Preco < 20`

d) `SELECT FornecedorNome FROM Fornecedor`
`UNION`
`SELECT ProdutoNome FROM Produto`
`WHERE Preco < 20)`

e) `SELECT FornecedorNome`
`FROM Fornecedor`
`WHERE FornecedorID = ANY (`
`SELECT ProdutoID`
`FROM Produto WHERE Preco < 20)`

9. (FGV/AFFC (STN)/STN/Tecnologia da Informação/Operação e Infraestrutura/2024)
ATENÇÃO: use a tabela relacional IPCA a seguir para responder à próxima questão.

Tabela IPCA

índice	ano	mês
0,56	2023	12
0,28	2023	11

0,24	2023	10
...	...	...
0,2	2003	7
-0,15	2003	6
...	...	...
2,25	2003	1
2,1	2002	12
3,02	2002	11
...	...	...
0,57	2001	1

A instância da tabela contém os valores do índice IPCA para todos os meses dos anos de 2001 até 2023. Os valores pontilhados representam a continuidade mensal da série. Todas as colunas são numéricas e não aceitam valores nulos.

Considere que um erro de programa na manipulação da tabela IPCA apresentou a inserção de registros duplicados, de modo que algumas linhas da tabela ficaram com valores idênticos em suas colunas.

Analise as alternativas SQL que foram consideradas para a remoção das linhas redundantes.

I. Utilizar o comando delete com a cláusula except one.

II. Utilizar o comando delete incluindo a cláusula exists, o que faria com que a última linha de cada grupo de repetições não seria removida uma vez que não mais existiria uma outra linha idêntica.

III. Introduzir, temporariamente, uma nova coluna na tabela, com valores gerados por meio de um mecanismo de auto increment, ou sequence, e utilizar o uso do comando delete incluindo a cláusula exists.

Em relação a essas alternativas, é correto afirmar que

- a) nenhuma funcionaria.
- b) todas funcionariam.
- c) somente I funcionaria.
- d) somente II funcionaria.
- e) somente III funcionaria.

10. (FGV/AFFC (STN)/STN/Tecnologia da Informação/Transformação Digital/2024) ATENÇÃO: use a tabela relacional IPCA a seguir para responder à questão.

Tabela IPCA

índice	ano	mês
0,56	2023	12
0,28	2023	11

0,24	2023	10
...	...	...
0,2	2003	7
-0,15	2003	6
...	...	...
2,25	2003	1
2,1	2002	12
3,02	2002	11
...	...	...
0,57	2001	1

A instância da tabela contém os valores do índice IPCA para todos os meses dos anos de 2001 até 2023. Os valores pontilhados representam a continuidade mensal da série. Todas as colunas são numéricas, e não aceitam valores nulos.

No contexto da tabela IPCA apresentada, assinale o comando SQL que produz, corretamente, para cada ano, o mês com o maior índice, ou meses, pois pode haver empate entre os índices de dois ou mais meses num mesmo ano.

a) select *

from ipca

where exists

(select mes, ano from ipca x

where x.ano = ipca.ano

and x.indice > ipca.indice

group by ano)

order by ano, mês

b) select ano, mes

from ipca

where not exists

(select mes, ano from ipca x

where x.ano = ipca.ano

and x.indice > ipca.indice)

order by ano, mês

c) select ano, mes

from ipca

where not exists

(select mes, ano from ipca x

where x.ano >= ipca.ano

and x.indice < ipca.indice)

order by ano, mês

d) select ano, mes

from ipca

where exists

(select * from ipca x

where x.ano = ipca.ano

and x.indice = ipca.indice

group by mes)

order by ano, mês

e) select ano, mes

from ipca

where exists

(select mes, ano from ipca x

where x.ano = ipca.ano

or x.indice > ipca.indice)

11. (FGV/AFFC (STN)/STN/Tecnologia da Informação/Transformação Digital/2024) ATENÇÃO: use a tabela relacional IPCA a seguir para responder à questão.

Tabela IPCA

indice	ano	mês
0,56	2023	12
0,28	2023	11
0,24	2023	10
...	...	...
0,2	2003	7
-0,15	2003	6
...	...	...
2,25	2003	1
2,1	2002	12
3,02	2002	11

...	...	...
0,57	2001	1

A instância da tabela contém os valores do índice IPCA para todos os meses dos anos de 2001 até 2023. Os valores pontilhados representam a continuidade mensal da série. Todas as colunas são numéricas, e não aceitam valores nulos.

No contexto da tabela IPCA apresentada, considere que ocorreu um acidente que fez com que diversas linhas dessa tabela tenham sido aleatoriamente deletadas, embora todos os índices dos meses de 2023 tenham permanecido intactos e nenhum dos anos tenha sido completamente deletado.

Analise as três versões de SQL que, pretensamente, poderiam recompor a tabela corretamente, inserindo os meses deletados com o valor nulo na coluna índice.

```
I. insert into IPCA(indice, ano, mes)
select NULL, a.ano, a.mes
from (select distinct ano, mes from IPCA) a
where not exists
      (select * from IPCA x
       where x.ano = a.ano
        and x.mes = a.mes)
```

```
II. insert into IPCA(indice, ano, mes)
select NULL, a.ano, b.mes
from (select distinct ano from IPCA) a,
     (select distinct mes from IPCA) b
where not exists
      (select * from IPCA x
       where x.ano = a.ano
        and x.mes = b.mes)
```

```
III. insert into IPCA(indice, ano, mes)
select NULL, a.ano, a.mes
from IPCA a
where a.ano * 100 + a.mes not in
      (select x.mes + x.ano * 100 from IPCA x)
```

A respeito da adequação desses comandos ao que se pretende, é correto concluir que

- a) nenhum seria adequado.
- b) somente I seria adequado.
- c) somente II seria adequado.

d) somente III seria adequado.

e) todos seriam adequados.

12. (FGV/AFTM (Pref N Iguaçu)/Pref Nova Iguaçu/2024) Seja um banco de dados relacional especificado em SQL de uma empresa de correspondência entre clientes, instituições financeiras e empréstimos contratados por esses clientes nessas instituições, previamente implementado em um banco de dados como a seguir:

```
create table tb_cliente
(
  id_cliente      integer      primary key,
  num_cpf         char(11)     unique,
  nome            varchar(50)  not null,
  email           varchar(20),
  telefone        varchar(20),
  endereco        varchar(100),
  cidade          varchar(20),
  estado          char(2) );

create table tb_financieira
(
  id_financieira  integer      primary key,
  razao_social    varchar(30)  not null,
  cidade          varchar(30)  not null,
  estado          char(2)      not null );

create table tb_emprestimo
(
  id_financieira  integer      references tb_financieira,
  id_cliente      integer      references tb_cliente,
  valor           real         not null check(valor>0),
  dia             integer      not null
                        check(dia>=1 and dia<=31),
  mes             integer      not null
                        check(mes>=1 and mes<=12),
  ano             integer      not null
                        check(ano>=1980 and ano<=2100),
  primary key(id_financieira,id_cliente,dia,mes,ano) );
```

OBS: Neste banco de dados, cadeias de caracteres (strings) são representadas envoltas em aspas simples.

Para fins de investigação, os auditores da empresa desejam saber os nomes dos clientes que contrataram empréstimos em todas as financeiras.

Assinale a consulta que apresenta o resultado desejado pelos auditores.

a) `select c.nome from tb_cliente c where exists ( select 1 from tb_emprestimo e where e.id_cliente = c.id_cliente )`

b) `select c.nome from tb_cliente c where id_cliente = all ( select id_financieira from tb_emprestimo )`

c) `select c.nome from tb_cliente c where not exists ( select id_financeira from tb_financeira except select e.id_financeira from tb_emprestimo e where e.id_cliente = c.id_cliente )`

d) `select distinct c.nome from tb_financeira f natural join tb_emprestimo e full join tb_cliente c on c.id_cliente = e.id_cliente`

e) `select distinct c.nome from tb_financeira natural join tb_emprestimo natural join tb_cliente c`

13. (FGV/AFRFB/SRFB/Geral/2023) Num banco de dados relacional, considere a tabela Vencedores, cuja instância é exibida a seguir, com duas colunas, Tenista e Torneio, que representam alguns torneios que já foram vencidos por alguns tenistas.

Tenista	Torneio
Roger Federer	Australian Open
Roger Federer	Roland Garros
Roger Federer	Wimbledon
Roger Federer	US Open
Pete Sampras	US Open
Pete Sampras	Wimbledon
Pete Sampras	Australian Open
Bjorn Borg	Roland Garros
Bjorn Borg	Wimbledon

Maria precisa escrever um comando SQL que liste os tenistas que venceram todos os torneios mencionados na coluna Torneio. O comando deve valer para qualquer instância válida da tabela, que pode conter diferentes tenistas e diferentes torneios.

Assinale o comando que Maria deve usar.

a)

```
select distinct Tenista from Vencedores v1
where v1.Torneio in (select Torneio from Vencedores)
```

b)

```
select distinct Tenista from Vencedores v1
where exists(
select * from Vencedores v2
where v1.Torneio = v2.Torneio
and v1.Tenista = v2.Tenista
and v1 <> v2))
```

c)

```
select distinct Tenista from Vencedores v1
```

```
where exists (
```

```
select * from Vencedores v2
```

```
where v1.Torneio = v2.Torneio
```

```
and v1.Tenista <> v2.Tenista )
```

d)

```
select distinct Tenista from Vencedores v1
```

```
where for all (
```

```
select * from Vencedores v2
```

```
where exists (
```

```
select * from Vencedores v3
```

```
where v1.Tenista = v2.Tenista))
```

e)

```
select distinct Tenista from Vencedores v1
```

```
where not exists(
```

```
select * from Vencedores v2
```

```
where not exists (
```

```
select * from Vencedores v3
```

```
where v2.Torneio = v3.Torneio
```

```
and v1.Tenista = v3.Tenista))
```

14. (FGV/Ana (Pref Niterói)/Pref Niterói/Tecnologia da Informação/2023) Figura 1
Nas duas questões a seguir, considere a tabela relacional T cuja instância é exibida abaixo.

A instalação está configurada para o tratamento de valores NULL como valores desconhecidos (unknown).

Tabela T

A	B	C
3	6	10
1	NULL	10
3	6	NULL

3	6	5
---	---	---

Considerando a tabela T da figura 1, analise o comando a seguir.

DELETE FROM T

WHERE EXISTS (SELECT * FROM T t2

WHERE T.A = t2.A

and T.B = t2.B

and T.C = t2.C)

O número de linhas removidas pela execução do comando acima é:

- a) 0;
- b) 1;
- c) 2;
- d) 3;
- e) 4.

15. (FGV/FTE (SEFAZ MT)/SEFAZ MT/2023) No contexto das linguagens de manipulação de dados de SGBD relacionais, analise a instância da tabela T e o comando SQL a seguir.

pessoa	ancestral
Bruna	Joana
Joana	João
João	Maria
Maria	Gabriel
Paulo	Gabriel

insert into T

select t1.pessoa, t2.ancestral

from T t1, T t2

where t1.ancestral = t2.pessoa

and not exists

(select * from T tt

where tt.pessoa = t1.pessoa

and tt.ancestral = t2.ancestral)

Dado que o comando SQL acima foi executado por três vezes consecutivas, assinale o número de linhas inseridas na tabela T em cada execução, na ordem.

- a) 0, 0, 0.
- b) 3, 5, 0.
- c) 5, 2, 1.
- d) 5, 3, 0.
- e) 8, 0, 0.

16. (FGV/Ana (BBTS)/BBTS/Perfil Tecnológico/2023) Considere uma tabela T, com uma única coluna A, definida como uma chave primária, e o comando SQL a seguir.

```
delete from T
where not exists
(select * from T tt where T.A > tt.A)
```

Dado que a tabela tem 100 linhas preenchidas, assinale a opção que indica o número de linhas que será deletado pela execução do referido comando.

- a) 0.
- b) 1.
- c) 50.
- d) 99.
- e) 100.

17. (FGV/AL (CAM DEP)/CAM DEP/Tecnologia da Informação/2023) Quando mencionada, considere a tabela relacional intitulada fibo, com duas colunas, cuja instância, não necessariamente nessa ordem, é exibida a seguir.

Tabela fibo	
P	N
1	0
2	1
3	1
4	2
5	3
6	5
7	8

Tabela fibo	
P	N
8	13
9	21
10	34

A coluna da esquerda enumera as linhas, continuamente, e a coluna da direita armazena os dez primeiros elementos da sequência de Fibonacci, na qual os dois primeiros termos são 0 e 1, por definição, e cada termo subsequente é a soma dos dois termos anteriores.

Suponha que a tabela *fibo*, apresentada anteriormente, tenha sido danificada, e sua instância corrente seja a que segue.

P	N
1	0
2	1
3	1
6	5
7	8

Para restaurar a tabela, foi produzido o comando SQL a seguir.

```
insert into fibo
select f2.P + 1, f1.N + f2.N
from fibo f1, fibo f2
where f1.P = f2.P - 1
      and f2.P < 10
      and not exists
          (select * from fibo f3
           where f1.N + f2.N = f3.N and f3.P > f2.P)
```

À luz das características de funcionamento do SQL, o número mínimo de vezes que esse comando deve ser acionado para restaurar a instância original, é

- a) 1.
- b) 2.

- c) 3.
- d) 4.
- e) 5.

18. (FGV/Prof EFM (SEDUC SP)/SEDUC SP/Informação e Comunicação/2023) Gerenciadores de bancos de dados relacionais usualmente empregam a lógica de três estados quando há operações de comparação envolvendo valores nulos (NULL). Nesse contexto, considere o comando SQL a seguir.

```
delete from T
where exists (select * from T T2 where T.A=T2.A)
```

Nesse contexto, é correto afirmar que

- a) nenhuma linha de T será excluída, independentemente do conteúdo da coluna A.
 - b) todas as linhas de T que contêm valores não nulos na coluna A serão excluídas.
 - c) todas as linhas de T que contêm valores nulos na coluna A serão excluídas.
 - d) todas as linhas de T serão excluídas, independentemente do conteúdo da coluna A.
 - e) o comando contém um erro, pois o a sintaxe do SQL não permite a autorreferência na cláusula exists.
19. (FGV/AJ (TJ SE)/TJ SE/Análise de Sistemas/Banco de Dados/2023) Considere as tabelas T1 e T2, exibidas abaixo com suas respectivas instâncias.

T1

A	B
1	1
2	2
4	2

T2

A	C
1	10
2	20

Observe o comando SQL a seguir.

```
select sum(A) from T1
where not exists
(select * from T2 where T1.A = 2 * T2.A)
```

Dadas as tabelas T1 e T2, descritas anteriormente, o resultado da execução desse comando exibe o valor:

- a) 1;
- b) 2;
- c) 3;
- d) 5;
- e) 7.

20. (FGV/AJ (TJ SE)/TJ SE/Análise de Sistemas/" "Sem Especialidade" "/2023) Considere uma tabela relacional TAB, com colunas A e B. A coluna A constitui a chave primária de TAB. A instância de TAB contém 100 linhas, e em todas as linhas o valor da coluna B é 10. Nesse contexto, analise o comando SQL a seguir.

```
select * from TAB t
```

```
where not exists
```

```
(select * from TAB tt
```

```
where t.B = tt.B and t.A > tt.A)
```

Além da linha de títulos, o número de linhas produzidas pelo comando acima é:

- a) 0;
- b) 1;
- c) 98;
- d) 99;
- e) 100.

21. (FGV/AUFC (TCU)/TCU/Controle Externo/Auditoria Governamental/2022) Na questão abaixo, considere as tabelas de banco de dados T, TX e DUAL, exibidas com suas respectivas instâncias a seguir.

T

sequencia	caracteristica
1	23987
2	9845
3	NULL
4	40983
6	48750
7	NULL
8	NULL
10	48750

12	48750
----	-------

TX

sequencia	caracteristica
2	9845
3	998034
4	50932
5	24390
6	48750
6	50296
7	NULL
8	998746
9	32746
9	NULL
9	22798

DUAL

x
NULL

Nas colunas das três tabelas, o tipo é o de número inteiro. Em todos os comandos SQL, considera-se o NULL como um valor desconhecido (unknown).

Considere que é preciso atualizar os dados da tabela T a partir dos dados da tabela TX, ambas definidas anteriormente. A consolidação é feita por meio da alteração na tabela T a partir de registros de TX.

O comando SQL utilizado nessa atualização é exibido a seguir.

update T

set caracteristica =

(select max(caracteristica) x from TX tx
where tx.sequencia = t.sequencia
and not (tx.caracteristica is null))

where

(exists

(select * from TX tx
where tx.sequencia = t.sequencia
and not (tx.caracteristica is null))

and

(t.caracteristica is null

or

t.caracteristica <

(select max(caracteristica) x

from TX tx

where tx.sequencia = t.sequencia

and not (tx.caracteristica is null))

)

)

O número de registros da tabela T afetados pela execução do comando SQL acima é:

a) zero;

b) três;

c) quatro;

d) seis;

e) nove.

22. (FGV/AUFC (TCU)/TCU/Controle Externo/Auditoria Governamental/2022) Na questão abaixo, considere as tabelas de banco de dados T, TX e DUAL, exibidas com suas respectivas instâncias a seguir.

T

sequencia	caracteristica
1	23987
2	9845
3	NULL
4	40983
6	48750
7	NULL
8	NULL
10	48750
12	48750

TX

sequencia	caracteristica
2	9845

3	998034
4	50932
5	24390
6	48750
6	50296
7	NULL
8	998746
9	32746
9	NULL
9	22798

DUAL

x
NULL

Nas colunas das três tabelas, o tipo é o de número inteiro. Em todos os comandos SQL, considera-se o NULL como um valor desconhecido (unknown).

Supondo que a coluna sequencia da tabela T, anteriormente definida, deveria conter números inteiros em sequência contínua, seria preciso descobrir os intervalos de valores faltantes. Um valor é considerado faltante quando a) é um número inteiro n entre o menor e o maior valor da tabela, tal que n não esteja presente na tabela, ou b) é um número presente na tabela T, com valor nulo na coluna característica.

inicio	fim	faltantes
3	3	1
5	5	1
7	9	3
11	11	1

O comando SQL que produz o resultado acima, a partir da instância inicialmente definida para a tabela T, é:

a)

```
select t1.sequencia 'inicio', t2.sequencia 'fim',
       t2.sequencia - t1.sequencia - 1 faltantes
from T t1, T t2
where t1.sequencia < t2.sequencia
      and t1.sequencia <> t2.sequencia - 1
      and not exists
      (select * from T t3
       where t3.sequencia > t1.sequencia
          and t3.sequencia < t2.sequencia)
```

b)

```
select t1.sequencia +1 'inicio',  
       t2.sequencia -1 'fim',  
       t2.sequencia - t1.sequencia -1 faltantes  
from T t1, T t2  
where t1.sequencia < t2.sequencia  
      and t1.sequencia <> t2.sequencia -1  
      and t1.caracteristica is not null  
      and t2.caracteristica is not null  
      and (not exists  
(select * from T t3  
  where t3.sequencia > t1.sequencia  
        and t3.sequencia < t2.sequencia  
        and t3.caracteristica is not null))
```

c)

```
select t1.sequencia +1 'inicio',  
       t2.sequencia -1 'fim',  
       t2.sequencia - t1.sequencia -1 faltantes  
from T t1, T t2  
where t1.sequencia <> t2.sequencia -1  
      and t1.caracteristica is not null  
      and t2.caracteristica is not null  
      and (not exists  
(select * from T t3  
  where t3.sequencia > t1.sequencia  
        and t3.sequencia < t2.sequencia  
        and t3.caracteristica is not null)  
or exists  
(select * from T t3  
  where t3.sequencia > t1.sequencia  
        and t3.sequencia < t2.sequencia  
        and t3.caracteristica is null))
```

d)

```
select t1.sequencia +1 'inicio',  
       t2.sequencia -1 'fim',  
       t2.sequencia - t1.sequencia -1 faltantes  
from T t1, T t2  
where t1.sequencia < t2.sequencia  
      and t1.sequencia <> t2.sequencia -1  
      and t1.caracteristica is not null
```

```
and t2.caracteristica is not null
and exists
(select * from T t3
    where t3.sequencia > t1.sequencia
        and t3.sequencia < t2.sequencia)
```

e)

```
select t1.sequencia +1 'inicio',
       t2.sequencia -1 'fim',
       t2.sequencia - t1.sequencia -1 faltantes
from T t1, T t2
where t1.sequencia < t2.sequencia
    and t1.sequencia <> t2.sequencia -1
    and t1.caracteristica is not null
    and t2.caracteristica is not null
    and not exists
    (select * from T t3
        where t3.sequencia > t1.sequencia
            and t3.sequencia < t2.sequencia)
```

23. (FGV/AJ (TJDFT)/TJDFT/Apoio Especializado/Análise de Dados/2022) Tabela TURFE

páreo cavalo tempo		
1	Estupenda	52,2
1	Mandrake	52,3
1	Capricho	53,4
2	Estupenda	51,0
2	Mandrake	51,1
2	Muddy River	52,1
2	Uramak	52,3
2	Escorial	52,5
2	Capricho	56,1
3	Muddy River	52,1
3	Estupenda	52,3
3	Uramak	53,0
3	Escorial	55,1

Considere um banco de dados relacional que contém uma única tabela, TURFE, cuja estrutura é exibida com sua instância, a seguir.

Para cada páreo, ou corrida, são armazenados os nomes dos cavalos participantes e os respectivos tempos. A classificação de cada cavalo numa corrida segue a ordem crescente de tempo. Não há empates.

Com relação à tabela TURFE, descrita anteriormente, o comando SQL que exhibe, para cada páreo, somente o cavalo que chegou em último lugar com o respectivo tempo é:

a) select pareo, cavalo, tempo
from TURFE

where for all

(select * from TURFE t

where t.pareo = TURFE.pareo

and t.tempo < TURFE.tempo)

b) select pareo, cavalo, tempo
from TURFE

where exists

(select * from TURFE t

where t.pareo = TURFE.pareo

and t.tempo > TURFE.tempo)

c) select pareo, cavalo, tempo
from TURFE

where not exists

(select * from TURFE t

where t.pareo <> TURFE.pareo

and t.tempo = TURFE.tempo)

d) select pareo, cavalo, tempo
from TURFE

where not exists

(select * from TURFE t

where t.pareo = TURFE.pareo

and t.tempo > TURFE.tempo)

e) select pareo, cavalo, tempo
from TURFE

where ALL

(select * from TURFE t

where t.pareo = TURFE.pareo
and t.tempo <> TURFE.tempo)

24. (FGV/AJ (TJDFT)/TJDFT/Apoio Especializado/Análise de Dados/2022) Tabela TURFE

	páreo	cavalo	tempo
1	Estupenda	52,2	
1	Mandrake	52,3	
1	Capricho	53,4	
2	Estupenda	51,0	
2	Mandrake	51,1	
2	Muddy River	52,1	
2	Uramak	52,3	
2	Escorial	52,5	
2	Capricho	56,1	
3	Muddy River	52,1	
3	Estupenda	52,3	
3	Uramak	53,0	
3	Escorial	55,1	

Considere um banco de dados relacional que contém uma única tabela, TURFE, cuja estrutura é exibida com sua instância, a seguir.

Analise o comando SQL a seguir.

```
select distinct cavalo
from TURFE
where not exists
(select *
from
(select t1.pareo, sum(1) xpto, t1.cavalo
from TURFE t1, TURFE t2
where t1.pareo = t2.pareo
and t2.tempo <= t1.tempo
group by t1.cavalo, t1.pareo) x
where TURFE.cavalo = x.cavalo
and x.xpto <> 2
)
```

Considerando-se a instância da tabela TURFE descrita anteriormente, a execução do comando acima exibe no resultado o(s) nome(s):

- a) Capricho;
- b) Escorial;
- c) Estupenda, Escorial;
- d) Mandrake;
- e) Uramak, Muddy River.

25. (FGV/AJ (TJDFT)/TJDFT/Apoio Especializado/Análise de Sistemas/2022) Tabela TURFE

páreo cavalo tempo		
1	Estupenda	52,2
1	Mandrake	52,3
1	Capricho	53,4
2	Estupenda	51,0
2	Mandrake	51,1
2	Muddy River	52,1
2	Uramak	52,3
2	Escorial	52,5
2	Capricho	56,1
3	Muddy River	52,1
3	Estupenda	52,3
3	Uramak	53,0
3	Escorial	55,1

Considere um banco de dados relacional que contém uma única tabela, TURFE, cuja estrutura é exibida com sua instância a seguir

Para cada páreo, ou corrida, são armazenados os nomes dos cavalos participantes e os respectivos tempos. A classificação de cada cavalo numa corrida segue a ordem crescente de tempo. Não há empates.

Com relação à tabela TURFE, descrita anteriormente, analise o comando SQL a seguir.

```
select distinct cavalo
from TURFE t1
where not exists
(select * from TURFE t2, TURFE t3
where t2.cavalo = t1.cavalo
and t3.pareo = t2.pareo
```

and t3.tempo > t2.tempo)

A execução do comando acima produz, somente, todos os cavalos que, em suas corridas:

- a) nunca chegaram em último lugar;
- b) nunca venceram e nunca chegaram em último lugar;
- c) nunca venceram;
- d) sempre chegaram em último lugar;
- e) sempre venceram.

26. (FGV/Ana (MPE SC)/MPE SC/Tecnologia da Informação/2022) Maria trabalha com um banco de dados onde há uma tabela T com linhas repetidas. Ela precisa remover as repetições indesejadas, porém mantendo as linhas sem repetição e apenas uma linha de cada conjunto de linhas repetidas, como mostrado a seguir.

Instância da tabela T antes da remoção:

A	B	C
2	5	7
2	5	7
2	5	7
1	8	9
1	8	9
2	2	2

Instância da tabela T após a remoção:

A	B	C
2	5	7
1	8	9
2	2	2

Maria optou por usar o comando abaixo para realizar a tarefa.

```
delete from T where  
exists (select * FROM T t1  
where T.A = t1.A and T.B = t1.B  
and T.C = t1.C)
```

Na execução desse comando, serão removidas:

- a) apenas uma linha pertencente a cada conjunto de linhas que contém linhas repetidas;

- b) apenas n-1 linhas pertencentes a cada conjunto que contém n linhas repetidas, onde n é um número par;
- c) apenas n-1 linhas pertencentes a cada conjunto que contém n linhas repetidas, onde n é um número ímpar;
- d) todas as linhas da tabela, exceto as linhas que não foram repetidas;
- e) todas as linhas da tabela, sem exceção.

27. (FGV/ES (SEMSA Manaus)/Pref Manaus/Analista de Banco de Dados/2022) Para a próxima s questões, considere do banco de dados relacional happyhour, que contém três tabelas, a saber. frequencia (cliente, bar)

preferencia (cliente, cerveja)

oferta (bar, cerveja)

A relação frequencia registra o(s) bar(es) que cada cliente frequenta, e tem dois atributos: nome do cliente e nome do bar;

A relação preferencia armazena a(s) cerveja(s) preferida(s) por cada cliente e tem dois atributos: nome do cliente e nome da cerveja;

A relação oferta relaciona a(s) cerveja(s) que cada bar serve, também com dois atributos: nome do bar e nome da cerveja.

É dado que todo cliente frequenta pelos menos um bar, que nem todos os clientes têm preferências, que todo bar oferece pelo menos uma cerveja e que toda cerveja é oferecida pelo menos por um bar.

Para o banco de dados happyhour, especificado anteriormente, assinale a expressão SQL que produz uma lista com os nomes dos clientes que frequentam todos os bares.

a) `select cliente from frequencia f1`

`where not exists`

`(select * from oferta where f1.bar= bar)`

b) `select cliente from frequencia f1`

`where not exists`

`(select * from oferta`

`where not exists`

`(select * from frequencia f2`

`where f1.cliente = f2.cliente`

`and oferta.bar = f2.bar))`

c) select cliente from frequencia f1

where not exists

(select * from oferta

where not exists

(select * from frequencia f2

where f1.cliente <> f2.cliente

and oferta.bar <> f2.bar))

d) select cliente from frequencia

e) select cliente from frequencia f1

where not exists

(select * from frequencia

where not exists

(select * from oferta f2

where f1.cliente = f2.cliente

and oferta.bar = f2.bar))

28. (FGV/ES (SEMSA Manaus)/Pref Manaus/Analista de Banco de Dados/2022) Para as próximas questões, considere do banco de dados relacional happyhour, que contém três tabelas, a saber.

frequencia (cliente, bar)

preferencia (cliente, cerveja)

oferta (bar, cerveja)

A relação frequencia registra o(s) bar(es) que cada cliente frequenta, e tem dois atributos: nome do cliente e nome do bar;

A relação preferencia armazena a(s) cerveja(s) preferida(s) por cada cliente e tem dois atributos: nome do cliente e nome da cerveja;

A relação oferta relaciona a(s) cerveja(s) que cada bar serve, também com dois atributos: nome do bar e nome da cerveja.

É dado que todo cliente frequenta pelos menos um bar, que nem todos os clientes têm preferências, que todo bar oferece pelo menos uma cerveja e que toda cerveja é oferecida pelo menos por um bar.

Com relação ao banco de dados happyhour, considere que houve um erro de carga, de forma que há bares presentes na tabela frequencia que não aparecem na tabela oferta (que deveria conter todos os bares).

Assinale o comando SQL que deletaria da tabela frequencia as linhas referentes a esses bares.

a) delete from frequencia

where oferta.bar <> frequencia.bar

b) delete from frequencia

from oferta

where oferta.bar <> frequencia.bar

c) delete from frequencia

where bar in (select bar from oferta)

d) delete from frequencia

where not exists

(select * from oferta f

where f.bar = frequencia.bar)

e) delete from frequencia

where bar = (select bar from oferta

where bar is null)

29. (FGV/ES (SEMSA Manaus)/Pref Manaus/Analista de Banco de Dados/2022) Para a próxima s questões, considere do banco de dados relacional happyhour, que contém três tabelas, a saber. frequencia (cliente, bar)

preferencia (cliente, cerveja)

oferta (bar, cerveja)

A relação frequencia registra o(s) bar(es) que cada cliente frequenta, e tem dois atributos: nome do cliente e nome do bar;

A relação preferencia armazena a(s) cerveja(s) preferida(s) por cada cliente e tem dois atributos: nome do cliente e nome da cerveja;

A relação oferta relaciona a(s) cerveja(s) que cada bar serve, também com dois atributos: nome do bar e nome da cerveja.

É dado que todo cliente frequenta pelos menos um bar, que nem todos os clientes têm preferências, que todo bar oferece pelo menos uma cerveja e que toda cerveja é oferecida pelo menos por um bar.

Com relação ao banco de dados happyhour, considere que houve uma ação de marketing que demanda que todos os bares tenham em suas ofertas a cerveja Forster.

Assinale o comando SQL que incluiria essa cerveja na tabela oferta para todos os bares que ainda não fazem essa oferta.

a) insert into oferta

```
select bar, 'Forster' from oferta
```

```
where not exists
```

```
(select * from oferta o
```

```
where o.cerveja = 'Forster'
```

```
and o.bar = oferta.bar)
```

b) insert into oferta

```
select bar, 'Forster' from oferta
```

```
where not exists
```

```
(select * from oferta o
```

```
where o.cerveja = 'Forster')
```

c) insert into oferta

```
select bar, 'Forster'
```

```
from (select distinct bar from oferta) x
```

```
where not exists
```

```
(select * from oferta o
```

```
where o.cerveja = 'Forster'
```

```
and o.bar = x.bar)
```

d) insert into oferta

```
select bar, 'Forster' from oferta
```

```
where
```

```
(select count(*) from oferta o
```

```
where o.cerveja = 'Forster')
```

and oferta.bar = o.bar) = 0

e) insert into oferta

select bar, 'Forster' from oferta

where oferta.bar not in

(select o.bar from oferta o

where o.cerveja = 'Forster'

and oferta.bar = o.bar)

30. (FGV/ES (SEMSA Manaus)/Pref Manaus/Analista Desenvolvimento de Sistemas/2022) Para a próxima questão, considere um banco de dados relacional que contém três tabelas, a saber. frequencia (cliente, bar)

preferencia (cliente, cerveja)

oferta (bar, cerveja)

A relação frequencia registra o(s) bar(es) que cada cliente frequenta, e tem dois atributos: nome do cliente e nome do bar;

A relação preferencia armazena a(s) cerveja(s) preferida(s) por cada cliente e tem dois atributos: nome do cliente e nome da cerveja;

A relação oferta relaciona a(s) cerveja(s) que cada bar serve, também com dois atributos: nome do bar e nome da cerveja. É dado que todo cliente frequenta pelos menos um bar, que nem todos os clientes têm preferências, que todo bar oferece pelo menos uma cerveja e que toda cerveja é oferecida pelo menos por um bar.

Com relação ao banco de dados especificado anteriormente, considere que houve um erro de carga, de forma que há bares presentes na tabela frequencia que não aparecem na tabela oferta (que deveria conter todos os bares).

Assinale o comando SQL que deletaria da tabela frequencia as linhas referentes a esses bares.

a) delete from frequencia

where oferta.bar <> frequencia.bar

b) delete from frequencia

from oferta

where oferta.bar <> frequencia.bar

c) delete from frequencia

where bar in (select bar from oferta)

d) delete from frequencia

where not exists

(select * from oferta f

where f.bar = frequencia.bar)

e) delete from frequencia

where bar = (select bar from oferta

where bar is null)

31. (FGV/AS (SEMSA Manaus)/Pref Manaus/Programador de Computador/

T1	
A	B
1	3
2	2
4	2

T2	
C	D
100	42
110	55
120	33

T3		
A	C	E
1	100	12
1	110	12
2	100	NULL

Em todos os comandos, supõe-se que NULL é tratado como um valor desconhecido (unknown). Com referência ao banco de dados EXEMPLO, especificado anteriormente, analise o comando SQL a seguir.

delete from T1

where exists (select * from T3 x, T3 y

where x.A = T1.A and y.A = T1.A

and x.E + y.E = 12)

Sobre os efeitos desse comando sobre a tabela T1, é correto afirmar que

a) nenhuma linha é deletada.

b) apenas a primeira linha é deletada.

c) apenas a segunda linha é deletada.

d) apenas a terceira linha é deletada.

e) todas as linhas são deletadas.

32. (FGV/AJ TRT16/TRT 16/Apoio Especializado/Tecnologia da Informação/ ATENÇÃO: Nas próximas três questões, considere o esquema relacional de um banco de dados numa instalação SQL Server na qual os nulos são tratados como valores desconhecidos (unknown).

create table X(A int not null primary key,

```
B int)
create table Y(A int not null UNIQUE,
               constraint fk
               foreign key (A) references X(A)
               on delete cascade)
```

Para todos os efeitos, suponha que o número de linhas em cada tabela é diferente de zero.

- a) `select * FROM X`
`WHERE not exists (select * FROM Y where X.A=Y.A)`
- b) `select * FROM X`
`WHERE not exists (select * FROM Y where X.A<>Y.A)`
- c) `select * FROM X`
`WHERE X.B IS NULL`
- d) `select * FROM Y`
`WHERE not exists (select * FROM X where X.A=Y.A)`
- e) `select * FROM Y`
`WHERE not exists (select * FROM X where X.A<>Y.A)`

33. (FGV/AT (TCE TO)/TCE TO/Tecnologia da Informação/2022) Caio preparou o comando SQL exibido a seguir para deletar um conjunto de linhas de uma tabela.

```
delete from T1
where exists(select * from T2
             where T1.A1 = T2.A2)
```

Para testar seu comando e descobrir exatamente quais seriam as linhas deletadas, Caio pretende executar um comando sem alteração de dados que permita a identificação dessas linhas. Para isso, basta que Caio substitua no comando original o termo:

- a) "=" por "<>";
- b) "delete from" por "show deleted rows from";
- c) "delete" por "delete limit 0";
- d) "delete" por "select *";
- e) "exists" por "not exists".

34. (FGV/Tec Leg (CM Taubaté)/CM Taubaté/Informática/2022) Na empresa onde João trabalha existe uma tabela PESSOA, com as colunas CPF, Nome e DataAdmissão no banco de

dados de pessoal. Ocorre que houve um erro de programação que permitiu que fossem inseridas linhas indevidamente, com diferentes nomes para um mesmo CPF, como se nota num trecho da tabela exibido a seguir.

CPF	Nome	DataAdmissão
100.100.100-10	A	10/03/2011
100.100.100-10	B	12/04/2012
200.200.200-20	C	10/07/2015
300.300.300-30	D	10/03/2020
300.300.300-30	D	10/03/2010
300.300.300-30	D	10/10/2019
400.400.400-40	E	10/03/2010

A tabela corrigida ficaria como mostrada abaixo.

CPF	Nome	DataAdmissão
100.100.100-10	A	10/03/2011
200.200.200-20	C	10/07/2015
300.300.300-30	D	10/03/2010
400.400.400-40	E	10/03/2010

Dado que a linha correta em cada caso é aquela que tem a menor data de admissão, assinale o comando SQL que João pode utilizar para “limpar” esses dados.

a) delete from pessoa

where not exists

(select * from pessoa p

where pessoa.CPF=p.CPF

and pessoa.nome=p.nome

and pessoa.dataadmissãop.>dataadmissão)

b) delete from pessoa

where not exists

(select * from pessoa p

where pessoa.CPF=p.CPF

and pessoa.nome<>p.nome

and pessoa.dataadmissão<=p.dataadmissão)

c) delete from pessoa

where exists

```
(select * from pessoa p
where pessoa.CPF=p.CPF)
and pessoa.dataadmissão<p.dataadmissão
d) delete from pessoa
where exists
(select * from pessoa p
where pessoa.CPF=p.CPF
and pessoa.dataadmissão>p.dataadmissão)
e) delete from pessoa
where exists
(select * from pessoa p
where pessoa.CPF=p.CPF
and pessoa.nome<>p.nome
and pessoa.dataadmissão<p.dataadmissão)
```

35. (FGV/AJ TRT13/TRT 13/Apoio Especializado/Tecnologia da Informação/2022) Considere uma tabela relacional X com uma única coluna A, cuja instância contém uma linha apenas, e os comandos SQL a seguir.

```
1
delete from X
where exists (select * from X x2
              where x2.A = X.A)
```

```
2
delete from X
where not exists (select * from X x2
                  where x2.A = X.A)
```

Nesse contexto, analise as afirmativas sobre os efeitos desses comandos.

I. Nenhuma linha seria deletada na execução do comando 1.

- II. Uma linha seria deletada na execução do comando 1.
- III. Nenhuma linha seria deletada na execução do comando 2.
- IV. Uma linha seria deletada na execução do comando 2.
- V. Os dois comandos gerariam erros de execução.

Está correto o que se afirma em

- a) I e III, apenas.
- b) I e IV, apenas.
- c) II e III, apenas.
- d) II e IV, apenas.
- e) V, apenas.

36. (FGV/Per Cri (PCA AP)/PCA AP/Ciência da Computação e Analista de sistema/2022) Numa prova de concurso, a organizadora estava preocupada com a ocorrência de homônimos fazendo prova na mesma sala, já que isso poderia ocasionar contratempos.

Para uma tabela relacional T, contendo as colunas CPF, Nome e CodSala, assinale o comando SQL que identifica corretamente os homônimos em cada sala.

a) SELECT CodSala, Nome, CPF

FROM T

WHERE EXISTS (SELECT * FROM T t1

WHERE t1.CodSala = T.CodSala

and t1.CPF <> T.CPF)

ORDER BY 1, 2, 3

b) SELECT CodSala, Nome, CPF

FROM T

WHERE EXISTS (SELECT * FROM TT

WHERE TT.CodSala = T.CodSala

and TT.Nome <> T.Nome

and TT.CPF = T.CPF)

ORDER BY 1, 2, 3

c) SELECT CodSala, Nome, CPF

FROM T

WHERE t1.CodSala = T.CodSala

and t1.Nome = T.Nome

and t1.CPF <> T.CPF)

ORDER BY 1, 2, 3

d) SELECT CodSala, Nome, CPF

FROM T

WHERE NOT EXISTS (SELECT * FROM T t1

WHERE t1.CodSala = T.CodSala

and t1.Nome = T.Nome

and t1.CPF <> T.CPF)

ORDER BY 1, 2, 3

e) SELECT CodSala, Nome, CPF

FROM T

WHERE EXISTS (SELECT * FROM T t1

WHERE t1.CodSala = T.CodSala

and t1.Nome = T.Nome

and t1.CPF <> T.CPF)

ORDER BY 1, 2, 3

37. (FGV/Ana Esp (IMBEL)/IMBEL/Analista de Sistemas/2021) Considere a instância da tabela R1 e o comando SQL exibidos a seguir.

A	B
1	2
2	2
3	3
4	3
4	2
4	1
5	0

select distinct A

from R1

where exists

(select * from R1 x where x.B > r1.A)

Assinale a lista de números que é exibida quando esse comando SQL é executado.

- a) 5
- b) 1, 2
- c) 2, 3
- d) 3, 4, 5
- e) 1, 2, 3, 4

38. (FGV/ATCE (TCE,AM)/TCE AM/Auditoria de Tecnologia da Informação/2021) Na questão, considere uma tabela de banco de dados T cuja instância é exibida a seguir.

A	B	C	D
12	2	3	1
14	3	8	2
18	2	9	3
21	5	4	4

Considerando-se a instância da tabela T (descrita anteriormente), analise o comando SQL abaixo.

update T

set a = a + 32

where

exists (select * from T t2 where T.c > t2.D)

O número de registros da tabela T afetados pela execução desse comando é:

- a) zero;
- b) um;
- c) dois;
- d) três;
- e) quatro.

39. (FGV/Ana TI (BANESTES)/BANESTES/Desenvolvimento de Sistemas/2021) Na questão, considere as tabelas T1, T2 e T3, cujas estruturas e instâncias são exibidas a seguir. O valor NULL deve ser tratado como unknown (desconhecido).

T1	
A	B
1	3
2	2
4	2

T2	
C	D
100	42
110	55
120	33

T3		
A	C	E
1	100	20
1	110	40
2	120	10
2	100	NULL
2	110	10

Tomando como referência as tabelas T1, T2 e T3, descritas anteriormente, o comando SQL

```
select t1.*
```

```
from T1
```

```
where not exists
```

```
(select * from T2, T3
```

```
where t1.A = t3.A and t2.C = t3.C
```

```
and t3.E is null)
```

produz como resultado somente a(s) linha(s):

a) 1, 3

b) 1, 3 / 4, 2

c) 4, 2

d) 1, 3 / 2, 2 / 4, 2

e) 2, 2 / 4, 2

40. (FGV/AJ (TJ AP)/TJ AP/Apoio Especializado/Tecnologia da Informação Banco de Dados/2024) Quando referenciadas, considere as tabelas relacionais Competidor e Disputa, cujas estruturas e instâncias são descritas abaixo. Todas as colunas são definidas como strings.

A tabela Disputa contém as disputas realizadas entre competidores que aparecem na tabela Competidor. Em cada disputa há dois competidores, um com camisa azul e outro com camisa verde.

Competidor

Nome
A

B
c
D
E
F

Disputa

Azul	Verde
A	B
C	A
B	A
C	E
F	A
F	D

Considerando as tabelas Competidor e Disputa, descritas anteriormente, analise o comando SQL a seguir.

```
delete from Competidor
where (select sum(1)
from Disputa d where d.azul = Nome)
< (select sum(1) from Disputa d
where d.verde = Nome)
```

O número de linhas removidas na execução do comando acima é:

- a) 6;
- b) 4;
- c) 2;
- d) 1;
- e) 0.

41. (FGV/AJ (TJ AP)/TJ AP/Apoio Especializado/Tecnologia da Informação Desenvolvimento de Sistemas/2024) Quando referenciadas, considere as tabelas relacionais Competidor e Disputa, cujas estruturas e instâncias são descritas abaixo. Todas as colunas são definidas como strings. A tabela Disputa contém as disputas realizadas entre competidores que aparecem na tabela Competidor. Em cada disputa há dois competidores, um com camisa azul e outro com camisa verde.

Competidor

Nome
A
B
C
D
E
F

Disputa

Azul	Verde
A	B
C	A
B	A
C	E
F	A
F	D

João tem pouca experiência com SQL, mas precisa de uma consulta que exiba os competidores que têm o mesmo número de disputas com as camisas azul e verde. João escreveu três scripts, utilizando as tabelas Competidor e Disputa, como definidas anteriormente, e tentou a sorte.

```
select distinct c.nome  
from Competidor c, Disputa d  
group by c.nome  
having count(distinct d.azul)  
      = count(distinct d.verde)
```

```
select c.nome  
from Competidor c  
where (select sum(1)  
      from Disputa d where d.azul = c.nome)  
      = (select sum(1)  
      from Disputa d where d.verde = c.nome)
```

```
select distinct c.nome  
from Competidor c, Disputa d  
where (select sum(1) where d.azul = c.nome)  
      = (select sum(1) where d.verde = c.nome)
```

Dado que a resposta correta deve exibir somente o competidor B, conclui-se que:

- a) nenhum dos scripts funciona;
- b) somente o primeiro script funciona;
- c) somente o segundo script funciona;
- d) somente o terceiro script funciona;
- e) os três scripts funcionam.

42. (FGV/ATRFB/SRFB/Geral/2023) Num banco de dados relacional, considere uma tabela R, com duas colunas A e B, ambas do tipo string de caracteres, cuja instância é exibida a seguir.

A	B
Pedro	João
Maria	Ida
Maria	Ida
Pedro	João
Edson	Wilson
Edson	Maria

Nesse cenário analise os comandos a seguir.

I.

```
DELETE FROM R
```

```
WHERE EXISTS (SELECT * FROM R r1  
              WHERE R.A = r1.A and R.B = r1.B)
```

II.

DELETE FROM R

WHERE EXISTS (SELECT * FROM R r1

WHERE R.A + R.B > r1.A + r1.B)

III.

DELETE FROM R

WHERE R.A + R.B in (SELECT A + B FROM R)

Assinale a lista que contém o número de registros deletados em cada um dos comandos I, II e III, respectivamente, quando executados separadamente e usando a mesma instância inicial descrita.

a) 2, 2 e 0.

b) 2, 4 e 0.

c) 4, 4 e 4.

d) 6, 5 e 6.

e) 6, 6 e 6.

43. (FGV/ACE (TCE SP)/TCE SP/Tecnologia da Informação (TI)/2023) Atenção

Quando referidas, considere as tabelas relacionais TX e TY, criadas e instanciadas com o script SQL a seguir.

```
create table TY(C int primary key not null, A int)
```

```
create table TX(A int primary key not null, B int,  
               foreign key (B) references TY(C)  
               on delete cascade  
               )
```

```
insert into TY values (1,0)
```

```
insert into TY(C) values (2)
```

```
insert into TY(C) values (3)
```

```
insert into TY values (5,NULL)
```

```
insert into TY values (6,NULL)
```

```
insert into TX values (1,2)
```

```
insert into TX values (2,1)
```

```
insert into TX values (3,2)
insert into TX values (4,2)
```

Com referência às tabelas TX e TY, como descritas anteriormente, analise o comando SQL a seguir.

```
insert into TX(A, B)
select C,A FROM TY
where C not in (select A from TX)
or A in (select A from TX)
```

O conjunto de linhas inseridas é:

- a) vazio;
- b) 0, NULL;
- c) 2, NULL e 3, NULL;
- d) 5, NULL e 6, NULL;
- e) 6, NULL.

44. (FGV/AS (SEMSA Manaus)/Pref Manaus/Programador de Computador/

T1	
A	B
1	3
2	2
4	2

T2	
C	D
100	42
110	55
120	33

T3		
A	C	E
1	100	12
1	110	12
2	100	NULL

Em todos os comandos, supõe-se que NULL é tratado como um valor desconhecido (unkown). Com referência ao banco de dados EXEMPLO, especificado anteriormente, a execução do comando SQL

```
select A, count(*)
from T1
where A = (select max(A) from T1)
group by A
having max(A) > (select max(A) from T3)
produz como resultado
```

- a) uma linha, com os valores 4, NULL.
- b) uma linha, com os valores 4, 1.
- c) uma linha, com os valores 2, 2.
- d) duas linhas, com os valores 4, 2 e 2, 2.
- e) duas linhas, com os valores 4, 2 e 1, 3.

45. (FGV/Ana (FunSaúde CE)/FunSaúde CE/II e Infraestrutura de Informática/2021) Considere a definição e as instâncias das tabelas de bancos de dados CLUBE e JOGO exibidas a seguir.

nome
Barcelona
Boca Juniors
The Strongest

JOGO

mandante	visitante	golsM	golsV
Barcelona	Boca Juniors	1	0
Barcelona	The Strongest	NULL	NULL
Boca Juniors	Barcelona	0	0
Boca Juniors	The Strongest	3	0
The Strongest	Barcelona	2	0
The Strongest	Boca Juniors	2	0

Cada clube deve jogar quatro vezes, duas como mandante e duas como visitante.

As colunas golsM e golsV registram o número de gols dos times mandantes e visitantes, respectivamente, em cada jogo. Ambas são nulas enquanto o jogo não for realizado.

Análise o comando SQL a seguir, à luz das definições e instâncias das tabelas CLUBE e JOGO, definidas anteriormente.

```
select c.nome from CLUBE c where
```

```
(select count(*) from JOGO j
```

```
where c.nome = j.mandante) <> 2
```

```
or
```

```
(select count(*) from JOGO j
```

```
where c.nome = j.visitante) <> 2
```

O resultado produzido pela execução desse comando é a lista de todos os clubes que

- a) aparecem em quatro jogos.
- b) não aparecem em quatro jogos.
- c) não aparecem em dois jogos como mandante ou que não aparecem em dois jogos como visitante.
- d) aparecem em dois jogos como mandante ou que aparecem em dois jogos como visitante.
- e) aparecem em dois jogos como mandante e aparecem em dois jogos como visitante.

46. (FGV/ACP (CGE PB)/CGE PB/Auditoria de Tecnologia da Informação/2024) Tabelas Relacionais T1 e T2

Considere as tabelas relacionais T1 e T2, de acordo com as colunas e as instâncias abaixo descritas. Na execução dos comandos SQL, assuma que o mecanismo do banco de dados considera valores NULL como valores desconhecidos (unknown).

Tabela T1

P	N
1	10
2	40
3	20
4	10

Tabela T2

E	Q	P
1	10	2
2	40	3
3	20	8
4	10	NULL

Considerando as tabelas T1 e T2, anteriormente apresentadas, analise o comando SQL a seguir.

`delete from T2 where P not in (select P from T2)`

O número de linhas deletadas da tabela T2 pela execução desse comando é:

- a) 0;
- b) 1;
- c) 2;
- d) 3;
- e) 4.

GABARITO

1. A	17. C	33. D
2. D	18. B	34. D
3. A	19. A	35. C
4. D	20. B	36. E
5. D	21. C	37. B
6. D	22. B	38. E
7. D	23. D	39. B
8. A	24. D	40. D
9. E	25. D	41. C
10. B	26. E	42. D
11. C	27. B	43. D
12. C	28. D	44. B
13. E	29. C	45. C
14. A	30. D	46. A
15. B	31. A	
16. B	32. D	

SQL - AVANÇADO

Visões (VIEWS)

Na prática, uma view (ou “visão”, em português) é uma consulta SQL pré-definida, armazenada no banco de dados com um nome. Quando as invocamos, o SGBD executa essa consulta e retorna o resultado.

É por isso que views também são chamadas de named queries (consultas nomeadas) ou stored queries (consultas armazenadas). Ela não é o resultado da consulta, ela é a própria consulta. É basicamente um texto de SELECT guardado no banco com um nome.

Por essa razão, é comum chamá-la de tabela virtual: a definição do seu SQL fica armazenada no catálogo do banco, e o resultado só é computado no momento em que a view é consultada.

Pois bem, para criarmos uma VIEW precisamos passar ao comando CREATE VIEW um nome para a visão e, na sequência, o SQL que ela executa ao ser chamada após o AS:

```
CREATE VIEW Professores_de_Matematica AS
SELECT *
FROM Professor
WHERE departamento = 'Matemática';
```

Essa view, chamada “Professores_de_Matematica”, consolida em uma “tabela virtual” todos os professores do departamento de matemática.

A partir desse momento, consultar os professores de matemática fica trivial, pois basta escrevermos um SQL como se a view fosse uma tabela como qualquer outra:

```
SELECT *
FROM Professores_de_Matematica;
```



(FGV/Ana MPU/2025) No processo de realizar análises nos dados armazenados em um banco de dados relacional, o perito Charles está implementando diversas consultas SQL. Ele precisa acessar o resultado dessas consultas diversas vezes, como se fosse uma tabela, em um ciclo de refinamento analítico contínuo.

Em um banco de dados relacional, para criar uma tabela virtual que represente o resultado de uma consulta SQL, Charles deve implementar um(a):

a) VIEW;

- b) TRIGGER;
- c) DOMAIN;
- d) SEQUENCE;
- e) FUNCTION.

Comentários:

a) Correta. A VIEW é exatamente uma tabela virtual que representa o resultado de uma consulta SQL armazenada no banco com um nome, podendo ser consultada várias vezes como se fosse uma tabela comum, o que atende perfeitamente à necessidade do Charles de acessar o resultado diversas vezes.

b) Errada. Uma TRIGGER é um gatilho, ou seja, um código procedural disparado automaticamente quando ocorre um evento (INSERT, UPDATE, DELETE) na tabela, não servindo para representar o resultado de uma consulta como tabela virtual.

c) Errada. DOMAIN não cria tabela virtual a partir de consulta, não sendo o recurso adequado para o cenário descrito.

d) Errada. SEQUENCE é usada para gerar números sequenciais, não para armazenar o resultado de uma consulta como tabela virtual.

e) Errada. FUNCTION é uma função armazenada que retorna um valor, não sendo o recurso usado para criar uma tabela virtual representando o resultado de uma consulta SQL.

Gabarito: A

As views materializadas seguem basicamente a mesma lógica das views comuns: elas são definidas a partir de uma consulta SQL, recebem um nome, e podem ser consultadas como se fossem tabelas.

A diferença essencial está em onde os dados ficam. Em uma view comum, a tabela é "virtual", ou seja, montada na hora em que da consulta, quando executada a query definidora contra as tabelas reais. Já na view materializada, os dados são efetivamente armazenados em disco, como uma tabela de verdade.

Embora seja um recurso extremamente útil presente na maioria dos bancos de dados, as views materializadas não fazem parte do padrão SQL oficial (ANSI/ISO), assim cada fabricante implementa o conceito à sua maneira, com sintaxes e comportamentos próprios.

Para fins de curiosidade, vejamos como alguns fabricantes disponibilizam esse comando (não precisa decorar para essa aula):

PostgreSQL	Oracle	SQL Server
<pre>CREATE MATERIALIZED VIEW Professores_de_Matematica AS SELECT * FROM Professor WHERE departamento = 'Matemática';</pre>	<pre>CREATE MATERIALIZED VIEW Professores_de_Matematica AS SELECT * FROM Professor WHERE departamento = 'Matemática';</pre>	<pre>CREATE VIEW Professores_de_Matematica WITH SCHEMABINDING AS SELECT id, nome, departamento FROM dbo.Professor WHERE departamento = 'Matemática'; CREATE UNIQUE CLUSTERED INDEX idx_prof_mat ON Professores_de_Matematica (id);</pre>

Mas cuidado com a atualização das views materializadas. Isso porque, como armazenam fisicamente os dados em disco, como uma tabela de verdade, elas não refletem automaticamente mudanças nas tabelas originais.

Assim, quando uma tabela subjacente é modificada, a view materializada fica "desatualizada" em relação a ela naquele momento. Para trazer a view materializada de volta ao estado atual, é preciso executar uma atualização explícita (refresh).

Saibam que, essa atualização varia entre os bancos de dados. Por exemplo, no PostgreSQL, o refresh é manual e explícito (há o comando "REFRESH MATERIALIZED VIEW"), enquanto o também Oracle permite configurar atualizações automáticas (como a cada commit ou por intervalo).



(VUNESP/Ana TI (Pref Campinas)/2023) Considere a seguinte tabela de um banco de dados relacional:

Hotel (ID, Nome, Cidade, Estado)

O comando SQL para criar uma visão denominada Local, que contenha apenas os atributos ID, Cidade e Estado é:

a) SELECT ID, Cidade, Estado

FROM Hotel

AS CREATE VIEW Local;

b) CREATE VIEW Local AS
SELECT ID, Cidade, Estado
FROM Hotel;

c) SELECT ID, Cidade, Estado
FROM Hotel
AS VIEW Local (ID, Cidade, Estado);

d) CREATE Local AS VIEW
FROM Hotel (ID, Cidade, Estado);

e) CREATE VIEW Local (ID, Cidade, Estado)
FROM Hotel;

Comentários:

a) **Errada.** A sintaxe está invertida; o CREATE VIEW deve vir no início do comando, não no final após um AS.

b) **Correta.** Segue exatamente a estrutura esperada: CREATE VIEW seguido do nome da visão (Local), depois AS e a consulta SELECT que define quais colunas (ID, Cidade, Estado) serão trazidas da tabela Hotel.

c) **Errada.** Não existe essa construção em que o SELECT vem antes e o VIEW é declarado depois com AS; a criação de visão começa com CREATE VIEW.

d) **Errada.** A ordem das palavras-chave está incorreta; o correto é CREATE VIEW nome AS SELECT..., e não CREATE nome AS VIEW FROM.

e) **Errada.** Falta a cláusula AS seguida de um SELECT; sem o SELECT, não há consulta definindo o conteúdo da visão.

Gabarito: B

(CEBRASPE (CESPE)/ERAC (ANAC)/ANAC/Qualquer Área de Formação/2024) Em um banco de dados relacional, a estratégia de implementação de uma view denominada materialização da view implica criar fisicamente uma tabela temporária a partir da primeira consulta a essa view e mantê-la ativa por um determinado período, considerando que poderão seguir-se outras consultas.

Comentários:

A questão está correta porque descreve exatamente o que acontece com uma view materializada: diferentemente da view comum (que monta a "tabela virtual" toda vez que é consultada), a view materializada armazena fisicamente os dados em disco, como se fosse uma tabela de verdade. Ou seja, na primeira consulta, o banco executa a query e guarda o resultado fisicamente. Esse resultado fica disponível por um período, podendo ser reutilizado em consultas seguintes sem precisar reprocessar tudo de novo contra as tabelas originais. Isso traz ganho de

desempenho, mas tem o custo de eventualmente ficar desatualizada em relação às tabelas reais, exigindo um refresh para voltar ao estado atual.

Gabarito: Certo

STORED PROCEDURES

Na prática, uma **stored procedure** (ou “procedimento armazenado”) é, basicamente, um **conjunto de instruções SQL e salvo dentro do próprio banco de dados**, esperando para ser executado.

Ou seja, trata-se basicamente de um “script” armazenado no banco de dados que realiza diversas atividades nos seus dados quando chamada. Porém, ao invés de ser escrito em linguagens de programação como Java ou Python, são escritas em extensões procedurais do SQL - como PL/SQL (Oracle), T-SQL (SQL Server) e PL/pgSQL (PostgreSQL) - que acrescentam recursos como variáveis, laços e estruturas condicionais ao SQL tradicional.

Por exemplo, imaginem uma aplicação de e-commerce que precisa registrar uma compra por um usuário. Para isso, envia três instruções individualmente para o banco: um INSERT na tabela “pedidos”, um INSERT na tabela “itens_pedido” e um UPDATE em “estoque”. Com uma stored procedure, toda a lógica fica armazenada no servidor, bastando a aplicação fazer apenas uma única chamada pela rede: “execute a stored procedure para determinado cliente, que comprou “X” produtos”.

Pois bem, um dos maiores benefícios das stored procedures é **evitar o retrabalho e a propagação de códigos duplicados**. Isso porque, em vez de criarmos as mesmas instruções várias vezes em diferentes sistemas, com a stored procedure mantemos tudo escrito uma vez só salvo no banco, e a deixamos disponível para ser executada por quem desejar.

Além disso, as stored procedures **reduzem o volume de dados que trafega entre a aplicação (cliente) e o banco de dados (servidor)**. Já que todo o processamento ocorre dentro do servidor, resultando em menos dados trafegados pela rede.

Também ganhamos na segurança e proteção dos dados, porque, com as stored procedures, **evitamos o tráfego de comandos SQL diretamente pela rede em instruções que podem ficar visíveis para usuários mal-intencionados**.

Isso porque a aplicação não conhece os nomes das tabelas nem a estrutura do banco, já que ela apenas chama a procedure pelo nome e passa os parâmetros. O restante fica encapsulado no servidor.

Assim, minimizamos a superfície de ataque de injeção de SQL, visto que um possível atacante não consegue inserir comandos maliciosos diretamente no código da procedure salva dentro do banco de dados.

Finalmente, há ganho em desempenho na execução dos comandos pelo banco. Isso porque, regra geral, **quando se executa uma stored procedure pela primeira vez**, o SGBD analisa o código, verifica se está correto e **cria um plano de execução** (uma estratégia otimizada que define como buscar e processar os dados com o menor custo possível).



(AVANÇASP/ASI (Morungaba)/2023) Qual a sintaxe para criar um procedimento em PL/SQL?

- a) CREATE PROCEDURE nome_procedimento
- b) CREATE PROCEDIMENTO nome_procedimento
- c) CREATE FUNCTION nome_procedimento
- d) CREATE PROCEDURE nome_function
- e) CREATE FUNCTION nome_function

Comentários:

a) **Correta.** Para criar um procedimento armazenado (stored procedure), usamos o comando CREATE PROCEDURE seguido do nome que queremos dar ao procedimento.

b) **Errada.** Não existe a palavra "PROCEDIMENTO" em SQL, o termo correto é em inglês: PROCEDURE.

c) **Errada.** FUNCTION é usado para criar funções, não procedimentos. Apesar de parecidos, são objetos diferentes no banco.

d) **Errada.** Embora comece corretamente com CREATE PROCEDURE, o nome "nome_function" sugere que estaríamos criando uma função, o que confunde a finalidade. A sintaxe correta para criar um procedimento usa CREATE PROCEDURE com um nome de procedimento.

e) **Errada.** FUNCTION é o comando para criar funções, e não procedimentos armazenados.

Gabarito: A

(ACAFE/Ana Sis (CELESC)/2024) Um desenvolvedor está trabalhando em um banco de dados Oracle e precisa criar um procedimento armazenado em PL/SQL que insira um novo registro na tabela EMPREGADOS. O procedimento deve aceitar os parâmetros emp_id, emp_nome e emp_salario e realizar a inserção. Qual das alternativas abaixo mostra CORRETAMENTE como esse procedimento pode ser escrito?

- a) CREATE PROCEDURE add_empregados(emp_id NUMBER, emp_nome VARCHAR2, emp_salario NUMBER)

AS

BEGIN

```
INSERT INTO EMPREGADOS (emp_id, emp_nome, emp_salario);  
END;
```

b) CREATE PROCEDURE add_empregados(emp_id NUMBER, emp_nome VARCHAR2, emp_salario NUMBER)

IS

BEGIN

```
INSERT INTO EMPREGADOS (ID, NOME, SALARIO) VALUES (emp_id, emp_nome, emp_salario);  
END;
```

c) CREATE PROCEDURE add_empregados(emp_id NUMBER, emp_nome VARCHAR2, emp_salario NUMBER)

BEGIN

```
INSERT INTO EMPREGADOS (ID, NOME, SALARIO) VALUES (emp_id, emp_nome, emp_salario);  
END add_empregados;
```

d) CREATE PROCEDURE add_empregados(emp_id IN NUMBER, emp_nome IN VARCHAR2, emp_salario IN NUMBER)

IN

BEGIN

```
INSERT INTO EMPREGADOS (ID, NOME, SALARIO) VALUES (emp_id, emp_nome, emp_salario);  
END;
```

e) CREATE PROCEDURE add_empregados(emp_id IN NUMBER, emp_nome IN VARCHAR2, emp_salario IN NUMBER)

AS

BEGIN

```
INSERT INTO EMPREGADOS (ID, NOME, SALARIO) VALUES (emp_id, emp_nome, emp_salario);  
END;
```

Comentários:

a) **Errada.** Falta indicar o modo dos parâmetros (IN) e o INSERT está incompleto, pois não traz a cláusula VALUES com os dados a serem inseridos.

b) **Errada.** Embora a sintaxe com IS seja válida em PL/SQL, falta a indicação do modo IN nos parâmetros, exigida na forma correta esperada pela banca.

c) **Errada.** Está faltando a palavra-chave AS (ou IS) entre a declaração dos parâmetros e o BEGIN, o que torna o procedimento sintaticamente inválido.

d) **Errada.** A palavra IN foi usada no lugar de AS antes do BEGIN, o que é incorreto. O IN só pode ser usado para indicar o modo dos parâmetros, não para abrir o corpo do procedimento.

e) **Correta.** Traz a estrutura adequada de uma stored procedure: CREATE PROCEDURE com o nome, parâmetros declarados com modo IN, a palavra AS abrindo o bloco, e entre BEGIN e END o comando INSERT corretamente formado com VALUES.

Gabarito: E

(FCC/Ana G (DPE AM)/2018) O comando SQL-ANSI para criar um procedimento chamado P1, que selecione os atributos A e B, de uma tabela T é:

a) PROCEDURE P1 IS

SELECT A, B

FROM T;

b) INSERT PROCEDURE P1 INTO DATABASE AS

SELECT A, B

FROM T

c) CREATE PROCEDURE P1()

SELECT A, B

FROM T;

d) MAKE PROCEDURE P1 (SELECT A, B

FROM T);

e) PROCEDURE P1 AS

SELECT A, B

FROM T

Comentários:

a) **Errada.** Falta o comando CREATE, que é o utilizado para criar objetos no banco, como procedimentos.

b) **Errada.** INSERT serve para inserir dados em tabelas, e não para criar procedimentos. O comando correto para criação é CREATE.

c) **Correta.** Usa CREATE PROCEDURE seguido do nome do procedimento (P1), com os parênteses indicando que não há parâmetros, e em seguida o SELECT que será executado.

d) **Errada.** Não existe o comando MAKE em SQL para criação de procedimentos. O correto é CREATE PROCEDURE.

e) **Errada.** Falta o CREATE no início. Apenas PROCEDURE não é suficiente para criar o procedimento no banco.

Gabarito: C

Gatilhos (TRIGGERS)

Trigger é basicamente um tipo especial de procedimento armazenado, porque é invocado automaticamente quando determinado evento ocorre no banco de dados.

Em outras palavras, o gatilho se trata de um código procedural, porém com um comportamento especial: não precisamos chamá-la diretamente, o próprio banco de dados dispara sua execução quando algo acontece.

Diferentemente das stored procedures "comuns", as triggers objetivam manter a integridade e consistência dos dados no banco. Ou seja, visam garantir que certas regras sejam sempre respeitadas, independentemente de quem está modificando os dados ou de qual aplicação está sendo usada.

Por exemplo, podemos querer que, se um novo registro for adicionado na tabela Funcionário, esse mesmo registro seja inserido na tabela Professor.

Para isso, basta criarmos um gatilho que reage a inserção de novos elementos na tabela Funcionário.



(FUNDATEC/Ana Sist (Londrina)/2024) Em um SGBD SQL Server 2019, qual instrução Transact-SQL pode ser utilizada para criar um procedimento armazenado que é executado automaticamente quando um usuário tenta modificar dados de uma tabela?

- a) CREATE EVENT
- b) CREATE TRIGGER
- c) CREATE VIEW
- d) CREATE RULE
- e) CREATE FUNCTION

Comentários:

- a) **Errada.** CREATE EVENT não é o comando usado para criar um procedimento que dispara automaticamente diante de modificações em uma tabela.
- b) **Correta.** A trigger (gatilho) é um tipo especial de procedimento armazenado que é invocado automaticamente quando determinado evento ocorre no banco, como INSERT, UPDATE ou DELETE em uma tabela. Justamente por isso, é a escolha certa quando queremos algo que reage sozinho às modificações nos dados.

- c) **Errada.** CREATE VIEW serve para criar uma tabela virtual baseada em uma consulta SQL, e não para reagir automaticamente a modificações.
- d) **Errada.** CREATE RULE não é o comando associado à execução automática diante de eventos de modificação de dados.
- e) **Errada.** CREATE FUNCTION cria uma função, que precisa ser chamada explicitamente, e não dispara automaticamente quando alguém altera os dados de uma tabela.

Gabarito: B

(Instituto Consulplan/Ana AAC (HEMOBRÁS)/2021) Considere uma aplicação que utiliza um Sistema de Gerenciamento de Bancos de Dados (SGBD) relacional, para a implementação de uma regra de negócio em que se deve, após inserir um registro na tabela CLIENTE, colocar, automaticamente, um registro na tabela CONTA. Podemos afirmar que a regra de negócio pode ser implementada no banco de dados através do objeto:

- a) trigger
- b) domain
- c) constraint
- d) generator
- e) foreign key

Comentários:

- a) **Correta.** A trigger (gatilho) é exatamente o objeto que executa automaticamente uma ação quando ocorre um evento no banco, como a inserção de um registro. No caso, ao inserir na tabela CLIENTE, a trigger dispara e insere automaticamente o registro correspondente em CONTA.
- b) **Errada.** Domain é usado para definir tipos de dados personalizados, não para executar ações automáticas em resposta a eventos.
- c) **Errada.** Constraint é uma restrição de integridade (como NOT NULL, UNIQUE, CHECK), que limita os dados aceitos, mas não executa comandos automáticos em outras tabelas.
- d) **Errada.** Generator não é um objeto voltado para reagir a eventos de inserção em tabelas com execução automática de comandos SQL.
- e) **Errada.** Foreign key apenas garante que um valor exista em outra tabela (integridade referencial), mas não insere registros automaticamente.

Gabarito: A

Estrutura básica da trigger

Os triggers seguem um modelo conceitual clássico com três componentes, conhecido como ECA (Event - Condition - Action), ou em português Evento - Condição - Ação.

O evento é o acontecimento que dispara o trigger, como INSERT (inserção de nova linha), UPDATE (alteração de linha existente) ou DELETE (exclusão de linha). Basta especificarmos, na criação da trigger, qual evento deve ativá-la.

A condição é um teste opcional que verifica se o trigger deve realmente reagir ao evento. Isso porque, mesmo que o evento aconteça, o trigger pode decidir que "essa situação não interessa" e não fazer nada. Na prática, definimos a condição como uma cláusula lógica, tipo "só execute a trigger se o contrato do funcionário for 'permanente'".

A ação é o código que efetivamente executa quando o evento acontece e a condição é satisfeita. Pode ser uma simples atualização em outra tabela, uma validação que cancela a operação, um registro em log de auditoria, ou qualquer lógica SQL que desejarmos.



(PROGEPE-UFRPE/Ana TI (UFRPE)/2018) Um comando importante em SQL é o CREATE TRIGGER. Um trigger típico possui três componentes básicos, que são:

- a) atributos - relações - condições a serem tomadas
- b) eventos - a condição - relações a serem tomadas
- c) atributos - a condição - ações a serem tomadas
- d) atributos - relações - relações a serem tomadas
- e) eventos - a condição - ações a serem tomadas

Comentários:

- a) **Errada.** Atributos e relações não são componentes do trigger. O modelo correto é ECA (Evento - Condição - Ação).
- b) **Errada.** Os dois primeiros estão certos (evento e condição), mas o terceiro componente é a ação a ser tomada, não relações.
- c) **Errada.** O primeiro componente é o evento (como INSERT, UPDATE ou DELETE), e não atributos.
- d) **Errada.** Nenhum dos três componentes está correto. O modelo é Evento - Condição - Ação.
- e) **Correta.** O trigger segue o modelo ECA (Event - Condition - Action), ou seja, Evento (o acontecimento que dispara o trigger, como INSERT, UPDATE ou DELETE), Condição (teste opcional que verifica se o trigger deve realmente reagir) e Ação (o código que efetivamente executa quando o evento acontece e a condição é satisfeita).

Gabarito: E

Criando e removendo triggers

Embora a sintaxe completa da trigger varie bastante entre bancos de dados (PostgreSQL, Oracle, SQL Server, MySQL, etc.), o núcleo básico é sempre parecido: invocamos CREATE TRIGGER, definimos um nome pro gatilho, o momento de disparo (antes ou depois do evento sensibilizar a tabela), o evento que o dispara (como INSERT, UPDATE, DELETE, DROP), a tabela alvo em que o evento disparou, e o código a ser executado.



Além de triggers BEFORE (antes), que executam antes que a modificação efetivamente aconteça no banco, e AFTER (depois), que executam depois que a modificação já aconteceu, alguns bancos também oferecem triggers INSTEAD OF (em vez de), que substituem a operação original.



(FGV/ATCE (TCE-AM)/2021) Maria gerencia um banco de dados SQL Server, e precisa criar um mecanismo que monitore todas as alterações ocorridas na estrutura das tabelas, especialmente os efeitos dos comandos CREATE, ALTER e DROP.

A estratégia mais adequada para detectar comandos dessa natureza passa pela criação e uso de:

- a) Funções escalares;
- b) Sequences;
- c) Statistics;
- d) Stored Procedures;
- e) Triggers.

Comentários:

a) Errada. Funções escalares apenas retornam um valor a partir de cálculos, não servem para monitorar eventos de alteração na estrutura das tabelas.

b) Errada. Sequences são geradores de números sequenciais, nada têm a ver com monitorar comandos executados no banco.

c) **Errada.** Statistics estão ligadas a estatísticas de dados para otimização de consultas, não detectam execução de comandos como CREATE, ALTER ou DROP.

d) **Errada.** Stored procedures são scripts armazenados no banco que precisam ser chamados manualmente; elas não disparam automaticamente quando algo acontece.

e) **Correta.** Triggers (gatilhos) são acionadas automaticamente quando determinado evento ocorre no banco, como comandos CREATE, ALTER e DROP, sendo ideais para monitorar alterações na estrutura das tabelas.

Gabarito: E

(CPCP UTFPR/Tec (UTFPR)/2018) Gatilhos ou triggers são códigos executados automaticamente no banco de dados e possuem evento, condição e ação. Analise o código SQL abaixo e assinale a alternativa correta.

```
R5: CREATE TRIGGER VIOLACAO_SALARIAL  
BEFORE INSERT OR UPDATE OF SALARIO  
CPF_SUPERVISOR ON FUNCIONARIO  
FOR EACH ROW  
WHEN( NEW.SALARIO >  
( SELECT SALARIO FROM FUNCIONARIO  
WHERE CPF = NEW.CPF_SUPERVISOR))  
INFORMAR_SUPERVISOR  
(NEW.Cpf_supervisor,  
NEW.Cpf );
```

a) O código apresenta a implementação de um gatilho (trigger) denominado VIOLACAO_SALARIAL, usando a notação Oracle, que será executado antes da inserção de novos registros na tabela FUNCIONARIO, validando se o salário é maior que o anterior.

b) O código apresenta a implementação de um gatilho (trigger), denominado VIOLACAO_SALARIAL, usando a notação Oracle, que será executado antes de um novo registro ser inserido na tabela FUNCIONARIO ou o salário ser alterado.

c) O código apresenta a implementação de um gatilho (trigger) denominado VIOLACAO_SALARIAL, usando a notação Oracle, que será disparado antes de executar a inserção no banco de dados, executando a ação de INFORMAR_SUPERVISOR. Neste exemplo, nenhuma condição foi especificada.

d) O código apresenta a implementação de um gatilho (trigger), denominado VIOLACAO_SALARIAL, usando a notação Oracle, que não será disparado, porque faltou indicar que a ação de INFORMAR_SUPERVISOR deve ser executada por uma FUNCTION.

e) O código apresenta a implementação de um gatilho (trigger), denominado VIOLACAO_SALARIAL, usando a notação Oracle, que será executado, após inserir um novo registro na tabela FUNCIONARIO ou após a alteração do atributo salário.

Comentários:

- a) **Errada.** O gatilho não dispara apenas em inserção, ele também dispara em atualização (UPDATE) do salário ou do CPF do supervisor. Além disso, a validação não compara com o "salário anterior", e sim com o salário do supervisor.
- b) **Correta.** O nome do gatilho é VIOLACAO_SALARIAL, ele usa BEFORE (antes do evento), reage tanto a INSERT quanto a UPDATE na tabela FUNCIONARIO, exatamente como descreve a alternativa.
- c) **Errada.** Existe sim uma condição definida pela cláusula WHEN, que verifica se o novo salário é maior que o salário do supervisor. Logo, é incorreto afirmar que nenhuma condição foi especificada.
- d) **Errada.** Não há exigência de que a ação seja obrigatoriamente uma FUNCTION. A ação do gatilho pode ser qualquer código procedural, inclusive a chamada de um procedimento como INFORMAR_SUPERVISOR.
- e) **Errada.** O gatilho dispara BEFORE (antes), e não AFTER (depois). Os eventos são INSERT ou UPDATE, e a alternativa inverte o momento de execução.

Gabarito: B

Além disso, ao criar a trigger podemos definir sua granularidade de execução.

Se escolhermos "FOR EACH ROW" ("para cada linha") executaremos a trigger uma vez para cada linha afetada pela operação. Por exemplo, se um UPDATE altera 100 linhas de uma tabela, o trigger roda 100 vezes, uma para cada.

Já se escolhermos "FOR EACH STATEMENT" ("para cada comando") a trigger executa uma única vez para o comando inteiro, independentemente de quantas linhas foram afetadas. Aqui geralmente queremos registrar dados das operações, como registrar "quem fez uma atualização na tabela Funcionários".

Posto tudo isso, vamos criar uma trigger chamada "Replicar_Funcionario_Professor" que responde a inserção ou atualização dos dados na tabela Funcionario, e verifica se o tipo do contrato é "permanente". Se sim, replica a operação na tabela Professor.

Por exemplo, podemos querer que, se um novo registro for adicionado na tabela Funcionário, esse mesmo registro seja inserido na tabela Professor ou Auxiliar, a depender do tipo de funcionário.

```
CREATE TRIGGER Replicar_Funcionario_Professor
AFTER INSERT OR UPDATE ON Funcionario
FOR EACH ROW
WHEN (NEW.tipo_contrato = 'permanente')
BEGIN
    INSERT INTO Professor (id, nome) VALUES (NEW.id, NEW.nome);
END;
```

Se ficou confuso, vamos ler cada linha para entender melhor

CREATE TRIGGER Replicar_Funcionario_Professor: "Crie um gatilho com de nome 'Replicar_Funcionario_Professor'".

AFTER INSERT OR UPDATE ON Funcionario : "Dispare este gatilho DEPOIS que alguém fizer um INSERT ou UPDATE na tabela Funcionario".

FOR EACH ROW: "Execute a trigger para cada vez que uma linha é afetada".

WHEN (NEW.tipo_contrato = 'permanente'): "Mas só faça alguma coisa se a nova linha tiver tipo de contrato igual a 'permanente'".

BEGIN ... END: "Aqui dentro vai o que deve ser feito".

INSERT INTO Professor (id, nome) VALUES (NEW.id, NEW.nome);: "Copie o id e o nome do funcionário recém-inserido/atualizado para a tabela Professor".

A palavra NEW é só uma forma do banco te entregar os dados da linha nova, como se fosse "os dados que acabaram de chegar".



(VUNESP/Ana (Pref Itapevi)/2019) Considere a sintaxe SQL básica do comando para a criação de gatilhos:

```
CREATE TRIGGER <nome do trigger>
<instante de execução> <evento disparador>
ON <nome da tabela> ...
```

Nesse comando, as opções corretas do

- a) evento disparador são: Insert, Delete e Update.
- b) evento disparador são: Select, Group By e Inner Join.

- c) evento disparador são: Commit, Rollback e Savepoint.
- d) instante de execução são: First e Last.
- e) instante de execução são: One e All.

Comentários:

- a) **Correta.** Os eventos que disparam uma trigger são justamente as operações que modificam dados na tabela: INSERT (inserção), UPDATE (alteração) e DELETE (exclusão).
- b) **Errada.** SELECT, GROUP BY e INNER JOIN são comandos de consulta, não modificam dados e, portanto, não funcionam como eventos disparadores de triggers.
- c) **Errada.** COMMIT, ROLLBACK e SAVEPOINT são comandos relacionados ao controle de transações, e não eventos disparadores de gatilhos.
- d) **Errada.** Os instantes de execução de uma trigger são BEFORE (antes), AFTER (depois) e, em alguns bancos, INSTEAD OF (em vez de). First e Last não existem nesse contexto.
- e) **Errada.** One e All também não são instantes de execução válidos para triggers. Os corretos são BEFORE, AFTER e INSTEAD OF.

Gabarito: A

(VUNESP/Ana (Pref Osasco)/2024) Considere o Sistema Gerenciador de Bancos de Dados MS SQL Server 2022, em que há uma tabela denominada Total. O comando SQL para criar um gatilho de nome Papel, que deve ser acionado após alguma operação de atualização, inserção ou exclusão nessa tabela Total é:

- a) CREATE TRIGGER Papel AFTER UPDATE, INSERT, DELETE ON Total AS SELECT ...
- b) CREATE TRIGGER Papel AS Total FOR UPDATE, INSERT, DELETE AS SELECT ...
- c) CREATE TRIGGER Total (Papel) AFTER UPDATE, INSERT, DELETE AS SELECT ...
- d) CREATE TRIGGER Papel (Total) ON UPDATE, INSERT, DELETE AS SELECT ...
- e) CREATE TRIGGER Papel ON Total AFTER UPDATE, INSERT, DELETE AS SELECT ...

Comentários:

- a) **Errada.** A ordem está incorreta, pois falta o ON Total antes do AFTER. No SQL Server, primeiro indicamos a tabela com ON e só depois o evento.
- b) **Errada.** Usa AS Total, o que não faz sentido, já que o nome da tabela deve vir após ON, e não após AS. O AS é usado para iniciar o bloco de código da trigger.
- c) **Errada.** Inverte os nomes: coloca Total como nome da trigger e Papel entre parênteses, quando o correto é o contrário (Papel é o nome da trigger e Total é a tabela).
- d) **Errada.** Usa ON UPDATE, INSERT, DELETE, mas o ON serve para indicar a tabela, não os eventos. Além disso, falta especificar o instante de execução (AFTER).

e) **Correta.** Segue a estrutura adequada: CREATE TRIGGER com o nome (Papel), ON indicando a tabela (Total), AFTER definindo o instante de execução, seguido dos eventos disparadores (UPDATE, INSERT, DELETE) e o AS para iniciar o código.

Gabarito: E

Em determinadas situações, queremos pausar o funcionamento da trigger temporariamente.

Para isso, alguns fabricantes oferecem comandos específicos, como a Oracle, com o ALTER TRIGGER combinado com as cláusulas DISABLE (desativar) e ENABLE (reativar).



Outros SGBDs têm variações bem parecidas.

O SQL Server usa "DISABLE TRIGGER nome ON tabela". Já o PostgreSQL usa "ALTER TABLE tabela DISABLE TRIGGER nome". O MySQL não oferece essa funcionalidade nativamente

Vamos então, desativar nossa trigger por um tempo.

```
ALTER TRIGGER Replicar_Funcionario_Professor DISABLE;
```

Porém, a partir do momento em que uma trigger se torna de fato indesejada, podemos a remover do banco através do comando DROP TRIGGER.

```
DROP TRIGGER Replicar_Funcionario_Professor;
```

Com isso, nossa trigger do banco desaparece, e as modificações na tabela Funcionario não disparam mais.



(ADM&TEC/Tec (CM Toritama)/2023) Analise as informações a seguir:

I. Uma trigger pode ser criada através do comando SQL CREATE TRIGGER. Após ser criada, ela fica automaticamente ativa, sendo disparada toda vez que o comando de disparo e condição de execução for verdadeiro.

II. Para ativar ou desativar a execução de uma trigger, deve ser usado o comando ALTER TRIGGER com a cláusula DISABLE. Para reativá-lo, deve ser usada a cláusula ENABLE.

Marque a alternativa CORRETA:

- a) As duas afirmativas são verdadeiras.
- b) A afirmativa I é verdadeira, e a II é falsa.
- c) A afirmativa II é verdadeira, e a I é falsa.
- d) As duas afirmativas são falsas.

Comentários:

I. **Correta.** A trigger é mesmo criada pelo comando CREATE TRIGGER e, uma vez criada, já fica ativa no banco, sendo disparada automaticamente sempre que ocorrer o evento configurado (como INSERT, UPDATE ou DELETE) e a condição definida for satisfeita. Lembre-se que a trigger segue o modelo Evento - Condição - Ação.

II. **Correta.** Para pausar temporariamente o funcionamento de uma trigger, usamos o comando ALTER TRIGGER combinado com a cláusula DISABLE (para desativar) e ENABLE (para reativar). Por exemplo: ALTER TRIGGER nome_trigger DISABLE.

Gabarito: A

QUESTÕES COMENTADAS - COMENTADAS



1. (CEBRASPE (CESPE)/TRF 6/Apoio Especializado/Análise de Dados/2025) Uma view é baseada em um tipo de consulta executada em uma ou mais tabelas em um banco relacional; normalmente é um tipo de consulta complexa e utilizada com frequência.

Comentários:

A questão está perfeitamente alinhada com o conceito de view. Uma view nada mais é do que uma consulta SQL pré-definida, armazenada no banco com um nome, que pode envolver uma ou mais tabelas. Justamente por isso ela é chamada de consulta nomeada (named query) ou consulta armazenada (stored query). A utilidade prática aparece exatamente no cenário descrito pela questão: quando temos uma consulta complexa que precisamos executar com frequência, em vez de reescrever todo o SELECT toda vez, criamos uma view e depois é só consultá-la como se fosse uma tabela comum. Isso simplifica bastante o trabalho, já que a complexidade fica encapsulada na definição da view.

Gabarito: Certo

2. (CEBRASPE (CESPE)/FUNPRESPEX/Executivos e Governança de Tecnologia da Informação/2025) Uma view materializada realiza consulta em tabelas do banco de dados no momento em que o usuário faz uma consulta à view.

Comentários:

A questão troca o conceito de view materializada com o de view comum. Na view comum, sim, a consulta é montada na hora: quando o usuário a chama, o banco vai até as tabelas reais e executa a query naquele momento. Já na view materializada, o comportamento é diferente. Os dados ficam armazenados fisicamente em disco, como se fosse uma tabela de verdade. Quando o usuário consulta a view materializada, ele acessa esses dados já guardados, e não uma consulta executada na hora contra as tabelas originais. Inclusive, é por isso que a view materializada pode ficar desatualizada em relação às tabelas de origem, sendo necessário fazer um refresh para sincronizá-la. Portanto, dizer que a view materializada consulta as tabelas no momento da consulta do usuário está incorreto, pois esse é o comportamento da view comum.

Gabarito: Errado

3. (CEBRASPE (CESPE)/TJ TRF6/TRF 6/Apoio Especializado/Suporte Técnico/2025) Os bancos de dados podem disparar instruções automáticas chamadas triggers antes ou depois de executar comandos DML, para garantir restrições de integridade.

Comentários:

A questão traz exatamente a ideia central das triggers. Elas são códigos que o próprio banco dispara automaticamente quando algum evento acontece, sem a gente precisar chamar manualmente. Os comandos DML são justamente aqueles que mexem nos dados: INSERT, UPDATE e DELETE. E ao criar uma trigger, podemos definir o momento de disparo, ou seja, BEFORE (antes) ou AFTER (depois) da operação acontecer na tabela. Além disso, o objetivo principal das triggers é exatamente esse citado na questão: manter a integridade e consistência dos dados, garantindo que certas regras sejam sempre respeitadas, independentemente de quem está alterando os dados. Por isso a afirmação está correta.

Gabarito: Certo

4. (CEBRASPE (CESPE)/ANM/Tecnologia da Informação/Ciência de Dados/2025) Os triggers são baseados no modelo evento-condição-ação, no qual um evento no banco de dados dispara uma ação quando determinada condição é atingida.

Comentários:

A questão apenas descreve o modelo clássico das triggers, conhecido como ECA: Evento, Condição e Ação. O evento é o acontecimento que dispara o gatilho, como um INSERT, UPDATE ou DELETE numa tabela. A condição é um teste que verifica se a trigger realmente deve agir naquela situação, funcionando como um filtro. Já a ação é o código que de fato é executado quando o evento ocorre e a condição é satisfeita. Como a afirmação descreve exatamente essa lógica, está correta.

Gabarito: Certo

5. (CEBRASPE (CESPE)/BANRISUL/Administração de Banco de Dados/2025) Em sistemas de gerenciamento de banco de dados relacionais, triggers

a) BEFORE TRUNCATE são executados antes de apagar as linhas e permitem cancelar a operação lançando erro.

b) AFTER DELETE são acionados antes de remover fisicamente a linha, de modo que, durante seu processamento, a linha ainda permanece na tabela.

c) BEFORE INSERT disparam antes de alocar espaço para a nova linha e podem ajustar valores usando NEW, ao passo que triggers AFTER INSERT disparam somente depois de todas as restrições terem sido verificadas e a linha ter sido gravada permanentemente.

d) BEFORE UPDATE são executados depois de atualizar os valores, porém antes de reindexar a tabela, permitindo reverter a operação caso alguma condição falhe.

e) AFTER TRUNCATE permitem o acesso a dados antigos das linhas por meio de uma pseudotabela OLD, possibilitando registrar o que foi apagado.

Comentários:

- a) **Errada.** Triggers BEFORE executam antes do evento, mas o material não trata de TRUNCATE como evento de trigger nem de cancelamento por lançamento de erro; o que se viu foi que BEFORE roda antes da modificação acontecer.
- b) **Errada.** Triggers AFTER, como o próprio nome diz, executam DEPOIS que a modificação já aconteceu, e não antes da remoção da linha.
- c) **Correta.** Triggers BEFORE INSERT disparam antes da inserção e podem ajustar valores da nova linha usando NEW (assim como vimos no exemplo NEW.id e NEW.nome), enquanto triggers AFTER INSERT só disparam depois que a linha já foi efetivamente gravada.
- d) **Errada.** Triggers BEFORE são executadas ANTES da modificação acontecer, e não depois de atualizar os valores.
- e) **Errada.** Esse tema do TRUNCATE com pseudotabela OLD vai além do conteúdo desta aula; o material apenas apresentou NEW para acessar dados da linha nova em operações de INSERT/UPDATE.

Gabarito: C

6. (CEBRASPE (CESPE)/Ana CT (CAPES)/CAPES/Informática/2024) Um trigger de banco de dados pode ser disparado antes ou depois de um comando e pode, até mesmo, substituir esse comando.

Comentários:

A afirmação descreve exatamente os três momentos possíveis de disparo de um gatilho. O trigger BEFORE executa antes que a modificação aconteça na tabela, o AFTER executa depois que a modificação já ocorreu, e o INSTEAD OF substitui a operação original, ou seja, em vez de executar o comando que o usuário pediu, o banco executa o que está dentro do gatilho. Por isso a questão está correta ao dizer que o trigger pode ser disparado antes, depois ou até substituir o comando.

Gabarito: Certo

7. (CEBRASPE (CESPE)/NATEL/Ciência de Dados/2024) Em SQL, triggers são procedimentos automáticos executados em resposta a determinados eventos em uma tabela ou view e são disparados exclusivamente em resposta a ações DML, tais como INSERT, UPDATE ou DELETE.

Comentários:

O erro está na palavra "exclusivamente". As triggers não reagem apenas a comandos DML (INSERT, UPDATE, DELETE). Elas também podem ser disparadas por comandos que alteram a estrutura do banco, como CREATE, ALTER e DROP, que são comandos DDL. Tanto é assim que vimos uma questão em que Maria precisava monitorar exatamente os comandos CREATE, ALTER e DROP nas tabelas, e a solução correta foi justamente usar triggers. Se elas reagissem só a DML, isso não seria possível. Então, ao ver a palavra "exclusivamente" limitando as triggers a INSERT, UPDATE e DELETE, já dá para desconfiar do erro.

Gabarito: Errado

8. (CEBRASPE (CESPE)/SEBRAE/Banco de Dados/2024) Nos bancos de dados SQL Servers, os gatilhos DDL são disparados nos eventos de

a) DENY, GRANT e UPDATE.

b) DENY, GRANT e REVOKE.

c) DELETE, DENY e REVOKE.

d) DENY, GRANT e INSERT.

Comentários:

a) **Errada.** UPDATE é um evento DML (manipulação de dados), não DDL.

b) **Correta.** DENY, GRANT e REVOKE são comandos que afetam permissões/estrutura, sendo eventos DDL.

c) **Errada.** DELETE é um evento DML, não DDL.

d) **Errada.** INSERT é um evento DML, não DDL, então não dispara gatilhos DDL.

Gabarito: B

9. (FCC/AM (MPE PB)/Administrador de Banco de Dados/2023) Um analista deseja executar um procedimento previamente armazenado em um banco de dados, quando da ocorrência de um evento SQL do tipo delete. Essa execução está corretamente relacionada ao conceito de

a) Broker.

b) Instanciação.

c) Trigger.

d) Generalização.

e) Docker.

Comentários:

a) **Errada.** Instanciação não é um conceito tratado aqui e não tem relação com a execução automática de procedimentos a partir de eventos no banco.

b) **Errada.** Broker não é o recurso usado para reagir a eventos como o delete em uma tabela.

c) **Correta.** A trigger (gatilho) é exatamente um tipo especial de procedimento armazenado que é invocado automaticamente quando determinado evento ocorre no banco, como INSERT, UPDATE ou DELETE. Ou seja, não precisamos chamá-la diretamente, o próprio banco dispara sua execução quando o evento acontece.

d) **Errada.** Generalização é um conceito de modelagem, não tem relação com execução automática de procedimentos diante de eventos SQL.

e) Errada. Docker é uma tecnologia de containers, totalmente alheia ao conceito de execução automática de procedimentos no banco.

Gabarito: C

10. (FCC/Ana Leg (ALAP)/ALAP/Atividade de Tecnologia da Informação/Desenvolvedor de Banco de Dados/2020) Triggers ou Gatilhos são comandos (ação) executados automaticamente (um tipo especial de procedimento armazenado) em consequência de uma modificação (evento) que produziu uma restrição (condição) que implique em defeito ou falta de integridade nos dados. É correto afirmar que

a) um trigger é usado com enorme eficiência para impor e manter integridade referencial de alto nível e, também, para ajudar na elaboração de resultados de consultas.

b) um trigger e o evento que o disparou devem ser considerados como uma mesma transação.

c) triggers são criados utilizando a instrução SQL Generate Trigger, que especifica o Banco onde ele atuará e para que tipo de ocorrência disparará suas ações.

d) os triggers podem ser executados, independentemente dos eventos aos quais foram associados.

e) triggers e Stored Procedures são procedimentos equivalentes; enquanto triggers disparam em função de uma ação, as Stored Procedures são disparadas por eventos temporais.

Comentários:

a) Errada. Triggers servem para manter integridade e consistência dos dados, e não para "ajudar na elaboração de resultados de consultas". Quem retorna resultados de consultas são as views ou os SELECTs, não os gatilhos.

b) Correta. Como a trigger é disparada automaticamente pelo próprio banco em resposta ao evento (INSERT, UPDATE ou DELETE), ela faz parte da mesma transação que originou o disparo, garantindo que tudo seja tratado como uma operação única e consistente.

c) Errada. O comando correto para criar uma trigger é CREATE TRIGGER, e não "Generate Trigger".

d) Errada. Triggers só executam quando o evento associado a elas ocorre (como um INSERT, UPDATE ou DELETE na tabela alvo). Não há execução independente do evento, pois o próprio banco dispara a trigger automaticamente quando aquele evento acontece.

e) Errada. Embora a trigger seja um tipo especial de procedimento armazenado, elas não são equivalentes às stored procedures comuns. As stored procedures precisam ser chamadas explicitamente pela aplicação, e não são disparadas por "eventos temporais".

Gabarito: B

11. (FCC/Ana Leg (ALAP)/ALAP/Atividade de Tecnologia da Informação/Desenvolvedor de Banco de Dados/2020) Considere uma tabela de produtos em estoque e outra de pedidos, em

um banco de dados. Toda vez que um item for incluído na tabela de pedidos, a quantidade em estoque desse item, na tabela de produtos, deve ser diminuída de modo equivalente. Um Analista pode solucionar essa situação, e ainda ganhar em desempenho,

- a) usando um trigger.
- b) criando uma view com dados de produtos e de pedidos.
- c) atribuindo o acesso a essas tabelas para apenas um usuário responsável.
- d) atribuindo privilégios à tabela de pedidos.
- e) atribuindo privilégios à tabela de produtos.

Comentários:

a) **Correta.** A trigger é exatamente o recurso que dispara automaticamente quando um evento ocorre, como a inclusão de um item na tabela de pedidos, executando uma ação como diminuir a quantidade da tabela de produtos, garantindo integridade e ganho de desempenho.

b) **Errada.** View é apenas uma consulta nomeada armazenada (tabela virtual), não executa ações automáticas de atualização em outras tabelas quando ocorre um evento.

c) **Errada.** Restringir acesso a um único usuário é uma medida de controle, não resolve a atualização automática do estoque nem traz ganho de desempenho.

d) **Errada.** Conceder privilégios apenas autoriza operações, não automatiza a diminuição da quantidade na tabela de produtos.

e) **Errada.** Da mesma forma, atribuir privilégios à tabela de produtos não cria nenhuma automação entre as duas tabelas.

Gabarito: A

12. (FCC/Ana G (DPE AM)/ Analista de Banco de Dados/2018) Visões constituem um tipo de estrutura possível de ser criada em um banco de dados relacional e apresentam como característica:

- a) Permitir a consulta apenas a tabelas que possuam somente atributos do tipo numérico armazenados.
- b) Possuir o armazenamento somente do comando de criação de sua estrutura, sendo os dados consultados a partir de tabelas com dados fisicamente armazenados.
- c) Não comportar a visualização de atributos do tipo booleano.
- d) Exibir apenas as chaves primária e estrangeira das tabelas especificadas em sua estrutura.
- e) Ser consultadas um número restrito de vezes, conforme seja o sistema gerenciador de banco de dados utilizado.

Comentários:

a) **Errada.** Não há qualquer restrição quanto ao tipo de dado das colunas das tabelas usadas em uma view. Ela pode consultar tabelas com qualquer tipo de atributo.

b) **Correta.** A view armazena apenas a definição da consulta SQL (o comando que a cria). Os dados em si continuam fisicamente armazenados nas tabelas reais, e o resultado só é montado no momento em que a view é consultada. Por isso ela é chamada de tabela virtual.

c) **Errada.** A view pode exibir atributos de qualquer tipo, inclusive booleanos. Não existe essa limitação.

d) **Errada.** A view pode exibir quaisquer atributos definidos no SELECT que a originou, e não somente chaves primárias e estrangeiras.

e) **Errada.** Não existe limite no número de vezes que uma view pode ser consultada. Ela funciona como uma tabela comum para fins de consulta.

Gabarito: B

13. (FGV/ATM (Pref Nova Iguaçu)/Pref Nova Iguaçu/2024) Seja um banco de dados relacional especificado em SQL de uma empresa de correspondência entre clientes, instituições financeiras e empréstimos contratados por esses clientes nessas instituições, previamente implementado em um banco de dados como a seguir:

OBS: Neste banco de dados, cadeias de caracteres (strings) são representadas envoltas em aspas simples.

Considere a seguinte sequência de comandos SQL:

```
insert into tb_cliente(id_cliente, num_cpf, nome)
```

```
values
```

```
(1,'111','cli1'),(2,'222','cli2'),(3,'333','cli3'),
```

```
(4,'444','cli4'),(5,'555','cli5');
```

```
update tb_cliente set estado='RJ'
```

```
where id_cliente%2!=0;
```

```
create view v_clientes_rj as
```

```
select id_cliente, num_cpf, nome, estado
```

```
from tb_cliente where estado = 'RJ';
```

Após a execução dos três comandos SQL apresentados, o seguinte comando foi executado:

```
insert into v_clientes_rj
```

```
values (6,'cli6','6666','MG');
```

Por fim, a seguinte consulta foi executada:

```
select count(*) from v_clientes_rj;
```

O retorno da última consulta executada será

- a) o valor 3, dado que a visão permanece selecionando tuplas de clientes do estado 'RJ'.
- b) o valor 4, já que a execução do último comando de inserção alterou o estado da visão.
- c) o valor 6, por esse ser o novo número de tuplas da tabela de base da visão.
- d) um erro, pois visões não aceitam inserções diretamente.
- e) uma tabela vazia, pois a última inserção desconfigura a condição-alvo da visão.

Comentários:

a) Correta. A view foi definida com a condição estado = 'RJ', então ela é uma consulta armazenada que sempre retorna apenas os clientes do RJ. Como inicialmente foram inseridos 5 clientes e o UPDATE colocou estado='RJ' apenas nos de id ímpar (1, 3 e 5), temos 3 clientes no RJ. A inserção feita pela view tentou inserir um cliente com estado 'MG', que não atende à condição da view, então ele não aparece quando consultamos a view novamente. Por isso o COUNT(*) retorna 3.

b) Errada. O cliente inserido tem estado 'MG', e a view só mostra clientes com estado 'RJ', então ele não é contado pela view.

c) Errada. A view não retorna todas as tuplas da tabela base, apenas aquelas que atendem à condição estado = 'RJ'.

d) Errada. Views podem aceitar inserções, funcionando como uma tabela virtual, conforme visto no conceito de visões.

e) Errada. A view não fica vazia; ela continua mostrando os clientes do RJ que já existiam, pois a view é uma consulta armazenada que é executada no momento da chamada.

Gabarito: A

14. (FGV/AJ (TJ RR)/TJ RR/Ciência de Dados e Analytics/2024) Ao projetar um banco de dados, espera-se atualizar valores de alguns campos de uma tabela a cada vez que um novo registro é inserido em outra tabela.

Para que isso aconteça sem a intervenção do usuário, deve ser implementada, em SQL, a estrutura denominada

- a) FUNCTION.
- b) SCHEMA.
- c) TRIGGER.
- d) UPDATE.
- e) VIEW.

Comentários:

- a) **Errada.** Function é uma rotina que retorna um valor quando chamada, não é disparada automaticamente por eventos em tabelas.
- b) **Errada.** Schema é apenas uma forma de organizar objetos do banco, não executa ações automáticas.
- c) **Correta.** A trigger (gatilho) é exatamente a estrutura que executa automaticamente quando ocorre um evento como INSERT, UPDATE ou DELETE em uma tabela, sem precisar ser chamada pelo usuário. No caso da questão, ao inserir um registro em uma tabela, a trigger dispara e atualiza os campos da outra tabela.
- d) **Errada.** UPDATE é apenas o comando que altera dados, mas precisa ser executado manualmente, não age sozinho.
- e) **Errada.** View é uma consulta armazenada (tabela virtual) usada para leitura de dados, não realiza atualizações automáticas em outras tabelas.

Gabarito: C

15. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Sobre a utilização de views, em bancos de dados como MySQL, SQL Server, analise as afirmativas a seguir.

I. Views servem para construir e armazenar o conteúdo de consultas complexas de uso frequente.

II. Os principais SGBD não permitem o uso de operações de update/delete sobre views.

III. Em geral, as views não podem ser referenciadas na cláusula FROM em outras consultas SQL.

Está correto somente o que se afirma em:

- a) I;
- b) II;
- c) III;
- d) I e II;
- e) II e III.

Comentários:

I. **Correta.** A view é justamente uma consulta SQL pré-definida e armazenada no banco com um nome, muito útil para guardar consultas complexas que usamos com frequência, evitando reescrever o mesmo SELECT várias vezes.

II. **Errada.** Cuidado com a pegadinha: a questão fala da view comum (não materializada). Os principais SGBDs permitem sim operações de UPDATE e DELETE sobre views, desde que

respeitadas certas condições. Tratar a view como uma tabela virtual é exatamente a ideia central: consultá-la e manipulá-la como se fosse uma tabela qualquer.

III. Errada. Pelo contrário, uma das maiores vantagens da view é poder ser usada na cláusula FROM como se fosse uma tabela comum. Lembre do exemplo: `SELECT * FROM Professores_de_Matematica`, em que a view aparece naturalmente no FROM.

Gabarito: A

16. (FGV/ATRFB/SRFB/Geral/2023) Considere um banco de dados relacional em que as operações de insert e update efetuadas numa certa tabela devem ser monitoradas e anotadas, como subsídio aos procedimentos de auditoria da empresa. Essa tabela é utilizada por uma série de aplicações, em diferentes tipos de transações, e iniciadas por um número considerável de usuários.

Nesse cenário, assinale o mecanismo mais adequado para a implementação desse monitoramento.

- a) Cursores.
- b) Stored procedures.
- c) Triggers.
- d) Utilitários de exportação de dados.
- e) Views.

Comentários:

a) Errada. Cursores não são tratados no conteúdo desta aula como mecanismo de monitoramento automático de operações.

b) Errada. Stored procedures são scripts armazenados no banco, mas precisam ser chamadas explicitamente pela aplicação, ou seja, não disparam sozinhas quando ocorre um insert ou update.

c) Correta. Triggers são acionadas automaticamente pelo próprio banco quando ocorre um evento como INSERT ou UPDATE em uma tabela, sendo ideais para registrar e monitorar alterações independentemente da aplicação ou usuário que fez a modificação.

d) Errada. Utilitários de exportação apenas movem dados, não monitoram operações ocorridas em uma tabela.

e) Errada. Views são consultas armazenadas (tabelas virtuais) usadas para leitura de dados, não servem para monitorar operações de insert e update.

Gabarito: C

17. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Num comando de criação de um trigger no MySQL, nas recentes edições, há um conjunto de

elementos sintáticos que podem ser utilizados para determinar quando os procedimentos devem ser acionados e quais são esses procedimentos.

As possíveis combinações de “quando x procedimento” válidas para triggers de transações são:

- a) AFTER/BEFORE combinados com DELETE/INSERT/UPDATE;
- b) CREATION/REMOTION combinados com READ/WRITE;
- c) IN/OUT combinados com CREATION/REMOTION;
- d) INIT/END combinados com INPUT/OUTPUT;
- e) STARTING/FINISHING combinados com READ/ REWRITE/WRITE.

Comentários:

a) **Correta.** AFTER (depois) e BEFORE (antes) são os momentos de disparo da trigger, e INSERT, UPDATE e DELETE são os eventos que disparam a trigger numa tabela.

b) **Errada.** CREATION, REMOTION, READ e WRITE não são palavras-chave usadas em triggers; os eventos disparadores são INSERT, UPDATE e DELETE.

c) **Errada.** IN/OUT não são momentos de disparo de triggers, e CREATION/REMOTION não são eventos válidos.

d) **Errada.** INIT/END e INPUT/OUTPUT não fazem parte da sintaxe de triggers; o momento correto é BEFORE/AFTER e os eventos são INSERT/UPDATE/DELETE.

e) **Errada.** STARTING/FINISHING e READ/REWRITE/WRITE não são palavras-chave para triggers; o correto seria BEFORE/AFTER combinados com INSERT/UPDATE/DELETE.

Gabarito: A

18. (FGV/ATCE (TCE,AM)/TCE AM/Auditoria de Tecnologia da Informação/2021) Maria gerencia um banco de dados SQL Server, e precisa criar um mecanismo que monitore todas as alterações ocorridas na estrutura das tabelas, especialmente os efeitos dos comandos CREATE, ALTER e DROP.

A estratégia mais adequada para detectar comandos dessa natureza passa pela criação e uso de:

- a) Funções escalares;
- b) Sequences;
- c) Statistics;
- d) Stored Procedures;
- e) Triggers.

Comentários:

- a) **Errada.** Funções escalares apenas retornam um valor calculado, não monitoram automaticamente eventos no banco.
- b) **Errada.** Sequences servem para gerar números sequenciais, não para detectar comandos executados nas tabelas.
- c) **Errada.** Statistics estão ligadas a estatísticas de dados para otimização, não ao monitoramento de comandos.
- d) **Errada.** Stored procedures precisam ser chamadas explicitamente, e não disparam automaticamente quando algo acontece no banco.
- e) **Correta.** A trigger é um tipo especial de procedimento armazenado que é invocado automaticamente quando determinado evento ocorre no banco, como comandos CREATE, ALTER e DROP, sendo ideal para monitorar alterações na estrutura das tabelas.

Gabarito: E

19. (FGV/Ana Leg (ALETO)/ALETO/Desenvolvimento de Sistema/2024) Visões em SQL possuem um papel relevante em bancos de dados relacionais. Ela oferece mecanismos importantes, tais como abstração de dados, segurança e simplificação das consultas. Nesse contexto, é correto afirmar que

- a) a cláusula WITH CHECK OPTION da SQL é demandada no final da especificação da visão, a fim de que a tornar uma asserção.
- b) a estratégia de modificação de consultas para a implementação de visões é eficaz para consultas complexas.
- c) a responsabilidade pela atualização do estado de uma visão é do usuário, que necessita enviar um comando refresh a cada execução.
- d) uma visão em linha (view em linha) consiste na definição de uma visão na cláusula HAVING de uma consulta SQL.
- e) visões materializadas criam uma tabela temporária, utilizando técnicas como a atualização incremental.

Comentários:

- a) **Errada.** A cláusula WITH CHECK OPTION não transforma a visão em asserção, e esse detalhe não faz parte do conteúdo desta aula.
- b) **Errada.** A estratégia de modificação de consultas (montar a view na hora a partir das tabelas reais) é típica das views comuns, mas não é eficaz quando a consulta é complexa, pois precisa ser recalculada toda vez que a view é consultada.
- c) **Errada.** Em uma view comum, não existe necessidade de refresh, pois ela é montada na hora da consulta. Já nas views materializadas, o refresh existe, mas pode ser manual (como no

PostgreSQL) ou automático (como no Oracle), não sendo uma obrigação do usuário a cada execução.

d) Errada. Esse tema vai além do conteúdo desta aula, mas vale dizer que view em linha não tem relação com a cláusula HAVING.

e) Correta. As views materializadas armazenam fisicamente os dados em disco, funcionando como uma tabela de verdade que precisa ser atualizada (refresh) para refletir mudanças nas tabelas originais, podendo usar técnicas como atualização incremental.

Gabarito: E

20. (FGV/ACE (TCE SP)/TCE SP/Tecnologia da Informação (TI)/2023) Views, em bancos de dados relacionais, configuram um importante mecanismo para flexibilizar o acesso aos dados armazenados. Em alguns casos, são ditas atualizáveis, pois podem ser utilizadas como alvo de comandos SQL como insert, update, delete.

Uma característica que não impede uma view de ser atualizável é:

- a) a cláusula group by;
- b) a cláusula distinct;
- c) a cláusula order by;
- d) a presença do operador algébrico union;
- e) a inexistência de uma fonte de registros atualizáveis na cláusula from.

Comentários:

a) Errada. A cláusula group by agrupa registros e gera linhas resumidas, o que impede a view de ser atualizável.

b) Errada. A cláusula distinct elimina duplicatas, descaracterizando a correspondência direta com as linhas originais, impedindo a atualização.

c) Correta. A cláusula order by apenas organiza a apresentação dos resultados, não afetando a possibilidade de a view ser atualizável.

d) Errada. O union combina resultados de consultas distintas, o que impede a identificação clara de qual tabela atualizar.

e) Errada. Se não há uma fonte de registros atualizáveis no from, naturalmente não há onde aplicar a alteração.

Gabarito: C

21. (CESGRANRIO/Tec Uni (UNEMAT)/UNEMAT/Analista de Sistemas/2024) A respeito do uso de procedimentos armazenados e de gatilhos em um banco de dados relacional, verifica-se, em relação à sua aplicabilidade, que

- a) os gatilhos são ideais para encapsular lógica de apresentação em um banco de dados.

- b) os gatilhos são usados, principalmente, para encapsular lógica de negócios complexa e reutilizável.
- c) os procedimentos armazenados são preferíveis para impor restrições de integridade referencial.
- d) os procedimentos armazenados são adequados para automatizar a execução de ações em resposta a eventos específicos.
- e) ambos, procedimentos armazenados e gatilhos, são exclusivamente usados para consultas complexas em bancos de dados.

Comentários:

- a) **Errada.** Gatilhos atuam dentro do banco de dados, reagindo a eventos como INSERT, UPDATE ou DELETE, e não tratam de lógica de apresentação.
- b) **Errada.** Quem encapsula lógica de negócio reutilizável são as stored procedures, que ficam guardadas no banco e podem ser chamadas quando se desejar. Os gatilhos servem principalmente para manter integridade e consistência dos dados.
- c) **Errada.** Restrições de integridade são, normalmente, mantidas pelos gatilhos, que disparam automaticamente para garantir que regras sejam respeitadas, independentemente de quem altera os dados.
- d) **Correta.** Apesar de a redação inverter um pouco a ideia clássica, a banca considerou esta como certa, no sentido de que procedimentos armazenados podem ser executados para realizar diversas ações no banco quando chamados, automatizando tarefas que envolvem várias instruções SQL.
- e) **Errada.** Nem stored procedures nem triggers são usadas exclusivamente para consultas complexas; elas executam diversos tipos de operações, como inserções, atualizações, validações e auditorias.

Gabarito: D

22. (CESGRANRIO/TBN (CEF)/CEF/Tecnologia da Informação/2024) O comando SQL de criação de gatilhos, CREATE TRIGGER, possui algumas informações obrigatórias, dentre elas, a seguinte:

- a) a hora na qual o gatilho precisa ser disparado.
- b) a que tabela do banco de dados o gatilho estará vinculado.
- c) o e-mail a ser notificado após a sua execução.
- d) o usuário para o qual o gatilho será disparado automaticamente.
- e) o disparo do gatilho está vinculado à execução de um comando SELECT.

Comentários:

- a) **Errada.** O gatilho não é disparado em uma hora específica do dia, e sim quando ocorre um evento no banco (como INSERT, UPDATE ou DELETE).
- b) **Correta.** Ao criar uma trigger, precisamos indicar a tabela alvo, ou seja, sobre qual tabela o evento disparador vai atuar (por isso usamos a cláusula ON nome_da_tabela).
- c) **Errada.** Não existe envio de e-mail como informação obrigatória no CREATE TRIGGER. O gatilho apenas executa um código SQL dentro do próprio banco.
- d) **Errada.** A trigger é disparada quando um evento ocorre na tabela (INSERT, UPDATE, DELETE), não para um usuário específico.
- e) **Errada.** Os eventos que disparam um gatilho são INSERT, UPDATE e DELETE, e não o comando SELECT, que apenas consulta dados sem alterá-los.

Gabarito: B

23. (CESGRANRIO/PTNS (TRANSPETRO)/2023) Gatilhos (triggers) e procedimentos armazenados (stored procedures) são componentes fundamentais em sistemas de gerenciamento de banco de dados (SGBDs) relacionais. Tanto os gatilhos quanto os procedimentos armazenados desempenham papéis vitais e, muitas vezes, complementares em aplicações baseadas em banco de dados relacionais, sendo escolhidos de acordo com as necessidades específicas de uma aplicação ou de um sistema. A respeito de gatilhos e de procedimentos armazenados, tem-se que

- a) tanto os gatilhos quanto os procedimentos armazenados são executados manual e explicitamente por um usuário ou por um programa de aplicação, e não podem ser disparados automaticamente por eventos de atualização em um banco de dados.
- b) tanto os gatilhos quanto os procedimentos armazenados são disparados automaticamente por eventos de atualização em um banco de dados, e não podem ser ativados manual e explicitamente por um usuário ou por um programa de aplicação.
- c) gatilhos, ao contrário dos procedimentos armazenados, não podem expressar a lógica de regras de negócios, e são utilizados exclusivamente para validação de dados.
- d) gatilhos são disparados automaticamente em resposta a eventos de atualização em um banco de dados, enquanto procedimentos armazenados precisam ser chamados explicitamente.
- e) gatilhos e procedimentos armazenados não podem coexistir em um mesmo sistema de banco de dados, pois possuem funcionalidades idênticas e redundantes.

Comentários:

- a) **Errada.** Procedimentos armazenados são executados quando chamados, mas gatilhos são disparados automaticamente pelo próprio banco quando ocorre um evento como INSERT, UPDATE ou DELETE.
- b) **Errada.** O disparo automático é característica dos gatilhos. Os procedimentos armazenados precisam ser chamados explicitamente pela aplicação ou usuário.

c) **Errada.** Gatilhos podem sim expressar regras de negócio, como replicar dados em outra tabela quando uma condição é satisfeita, e não servem apenas para validação.

d) **Correta.** Essa é justamente a diferença essencial: o gatilho é um tipo especial de procedimento armazenado que o próprio SGBD dispara automaticamente quando ocorre um evento (INSERT, UPDATE, DELETE), enquanto o procedimento armazenado comum só roda quando alguém o chama pelo nome.

e) **Errada.** Eles coexistem normalmente e têm finalidades diferentes: o procedimento é chamado manualmente, e o gatilho é disparado automaticamente para garantir integridade e consistência.

Gabarito: D

24. (CESGRANRIO/Tec Cien (BASA)/BASA/Tecnologia da Informação/2022) Em banco de dados, os gatilhos são utilizados, entre outros objetivos, para implementar restrições de integridade. Considere que em um banco de dados de um banco comercial há duas tabelas, CLIENTE (chave primária CPF) e SERVICO (chave primária composta por CPF e identificação do serviço), e há a restrição de que um cliente não pode estar associado a mais de cinco serviços. Que definição deve ser utilizada para implementar essa restrição por meio de um gatilho?

- a) BEFORE INSERT ON CLIENTE
- b) AFTER INSERT ON CLIENTE
- c) BEFORE INSERT ON SERVICO
- d) AFTER INSERT ON SERVICO
- e) WHEN CLIENTE INSERT ON SERVICO

Comentários:

a) **Errada.** O evento não ocorre na tabela CLIENTE, e sim em SERVICO, que é onde os serviços são cadastrados.

b) **Errada.** Mesmo problema da alternativa A: a tabela correta é SERVICO, não CLIENTE.

c) **Correta.** BEFORE INSERT ON SERVICO permite verificar, antes da inserção, se aquele cliente já tem cinco serviços, bloqueando a operação caso o limite seja excedido.

d) **Errada.** Usar AFTER faria a verificação tarde demais, pois o registro já teria sido inserido violando a regra.

e) **Errada.** Essa sintaxe não corresponde à estrutura básica de criação de gatilhos (instante de execução + evento disparador + ON tabela).

Gabarito: C

25. (CESGRANRIO/Esc BB/BB/Agente de Tecnologia/2021) Para que fosse mais fácil entender um sistema em desenvolvimento, um desenvolvedor usou um modelo de dados relacional (protótipo) mais simples do que o do banco de dados do sistema corporativo original, sendo que

ambos utilizavam o mesmo SGDB PostgreSQL. Cabe ressaltar que esse protótipo utilizava apenas um subconjunto dos dados do sistema corporativo original e realizava apenas consultas. Uma forma de garantir que os dados desse protótipo estejam sempre completamente atualizados em relação aos dados reais, com baixo impacto tanto na operação quanto no desempenho do sistema corporativo original, é

- a) criar apenas VIEWS no protótipo, definidas com consultas sobre as tabelas do sistema corporativo original.
- b) implantar TRIGGERS a cada INSERT, em todas as tabelas do sistema corporativo original, atualizando as tabelas do protótipo.
- c) implantar TRIGGERS de atualização a cada SELECT, em todas as tabelas do protótipo.
- d) particionar as tabelas da base do sistema corporativo original escolhendo um RANGE adequado ao trabalho do protótipo.
- e) utilizar DUMP da base do sistema corporativo original e PSQL para a base do protótipo, a cada seção de trabalho, para atualizar a base do protótipo.

Comentários:

- a) **Correta.** A view é uma consulta SQL armazenada que funciona como uma tabela virtual: ela não guarda os dados, apenas executa a consulta nas tabelas originais no momento em que é chamada. Assim, o protótipo sempre verá os dados atualizados do sistema corporativo, sem duplicar informação nem sobrecarregar a operação original.
- b) **Errada.** Criar triggers em todas as tabelas a cada INSERT geraria alto impacto no desempenho do sistema corporativo, pois cada inserção dispararia código adicional para replicar dados no protótipo.
- c) **Errada.** Triggers reagem a eventos como INSERT, UPDATE ou DELETE, que modificam dados, e não a SELECT, que apenas consulta. Logo, essa construção não faz sentido.
- d) **Errada.** Particionar tabelas por RANGE é uma técnica de organização física dos dados e não tem relação com manter um protótipo sincronizado com a base original.
- e) **Errada.** Fazer DUMP a cada sessão é custoso e não garante que os dados estejam completamente atualizados durante o uso, pois a cópia ficaria desatualizada logo após ser feita.

Gabarito: A

26. (VUNESP/ATI (Pref Santo André)/Pref Santo André/Administração de Bancos de Dados/2024) O comando SQL para, a partir da seguinte tabela de um banco de dados relacional Biblioteca (ID, Título, Autor1, Autor2, Ano)

criar uma visão denominada Biblio, contendo os atributos ID e Título é:

- a) CREATE VIEW Biblio (ID, Título)

FROM Biblioteca;

b) CREATE VIEW Biblio

SELECT Biblioteca (ID, Título);

c) CREATE VIEW Biblio

WHERE Biblio (ID, Título);

d) CREATE VIEW Biblio AS

(SELECT ID, Título

FROM Biblioteca);

e) CREATE VIEW Biblio

FROM Biblioteca (ID, Título);

Comentários:

a) Errada. Falta a cláusula AS seguida do SELECT; não basta listar os atributos diretamente após o nome da view.

b) Errada. A sintaxe está incorreta, pois falta o AS e o SELECT está mal formado, sem o FROM e sem listar os campos corretamente.

c) Errada. WHERE é usado para filtrar linhas em uma consulta, não para definir os atributos de uma view. Aqui faltam o AS e o SELECT.

d) Correta. Segue exatamente o padrão de criação de views: CREATE VIEW nome_da_view AS seguido da consulta SELECT que define os dados, no caso selecionando ID e Título da tabela Biblioteca.

e) Errada. Falta a cláusula AS com o comando SELECT, que é obrigatória para definir o conteúdo da view.

Gabarito: D

27. (VUNESP/CFO/QC (EsFCEEx)/EsFCEEx/Informática/2024) O comando SQL para criar uma visão de nome Teste, a partir dos atributos C1 e C2 da tabela denominada Prova é:

a) CREATE TABLE VISION Teste USING (C1, C2 FROM Prova);

b) CREATE TABLE VIEW Teste AS SELECT C1, C2 FROM Prova;

c) CREATE VISION Teste FROM Prova (C1, C2);

d) CREATE VIEW Teste FROM Prova (C1, C2);

e) CREATE VIEW Teste AS SELECT C1, C2 FROM Prova;

Comentários:

- a) **Errada.** A sintaxe para criar uma visão usa CREATE VIEW, e não CREATE TABLE VISION. Além disso, a cláusula correta é AS SELECT, e não USING.
- b) **Errada.** Não existe a combinação CREATE TABLE VIEW. Para criar uma visão, usamos apenas CREATE VIEW seguido do nome.
- c) **Errada.** O comando correto é CREATE VIEW (e não VISION). Além disso, falta a cláusula AS SELECT, que é obrigatória para definir a consulta da visão.
- d) **Errada.** Apesar de começar corretamente com CREATE VIEW, falta a cláusula AS SELECT, que é essencial para indicar quais colunas a visão vai retornar.
- e) **Correta.** Essa é a sintaxe padrão para criar uma visão: CREATE VIEW seguido do nome da visão (Teste), depois AS e a consulta SELECT que define o conteúdo da visão (os atributos C1 e C2 da tabela Prova).

Gabarito: E

28. (VUNESP/Pref Santo André/Administração de Bancos de Dados/2024) Quando da definição de algumas estruturas em um banco de dados, pode ser necessário especificar os chamados triggers (gatilhos) que

- a) representam a estrutura física das tabelas contidas no banco de dados.
- b) representam um tipo especial de tabela que contém dados do criador do banco de dados.
- c) especificam os tempos máximos de execução de cada consulta feita ao banco de dados.
- d) constituem uma tabela que contém a relação de usuários e seus privilégios dentro do banco de dados.
- e) são procedimentos acionados de forma automática, como efeito da ocorrência de algum evento especificado em seu comando.

Comentários:

- a) **Errada.** Triggers não representam estrutura física de tabelas; são códigos procedurais que reagem a eventos no banco.
- b) **Errada.** Triggers não são tabelas, e sim procedimentos armazenados especiais que disparam automaticamente.
- c) **Errada.** Triggers não controlam tempo de execução de consultas; eles reagem a eventos como INSERT, UPDATE ou DELETE.
- d) **Errada.** Triggers não armazenam usuários nem privilégios; sua função é manter a integridade e consistência dos dados reagindo a eventos.
- e) **Correta.** Triggers são procedimentos acionados automaticamente quando ocorre um evento especificado (como INSERT, UPDATE ou DELETE), seguindo o modelo Evento - Condição - Ação.

Gabarito: E

29. (FUNDATEC/AFRM (Pref Criciúma)/2024) Sobre bancos de dados, assinale a alternativa INCORRETA.

- a) Uma tabela é uma estrutura de dados que armazena linhas que representam uma coleção de valores de dados relacionados.
- b) Uma view é uma tabela virtual que é derivada de outras tabelas, armazenando os dados fisicamente em outro local.
- c) Um índice é uma estrutura de dados que facilita o acesso rápido aos dados em uma tabela.
- d) Uma chave primária é um atributo ou conjunto de atributos que identifica unicamente cada registro em uma tabela.
- e) Uma chave estrangeira mantém uma restrição de integridade referencial entre uma entidade referenciada e outra que referencia.

Comentários:

- a) **Errada.** De fato, uma tabela armazena linhas com valores de dados relacionados, sendo a estrutura básica de um banco relacional.
- b) **Correta** (a questão pede a INCORRETA). A view comum é uma tabela virtual, ou seja, ela não armazena os dados fisicamente, apenas guarda a consulta SQL que é executada no momento em que a view é chamada. Quem armazena dados fisicamente é a view materializada, e não a view comum.
- c) **Errada.** O índice realmente é uma estrutura que facilita o acesso rápido aos dados de uma tabela.
- d) **Errada.** A chave primária identifica de forma única cada registro da tabela, podendo ser um atributo ou um conjunto deles.
- e) **Errada.** A chave estrangeira garante a integridade referencial entre tabelas, ligando uma entidade que referencia à entidade referenciada.

Gabarito: B

30. (VUNESP/Ana (Pref Marília)/Pref Marília/Dados/2023) A sintaxe básica do comando de criação de gatilhos em SQL é:

- a) CREATE FUNCTION ...
- b) CREATE TRIGGER ...
- c) SELECT TRIGGER ...
- d) DROP TRIGGER ...
- e) INSERT TRIGGER ...

Comentários:

- a) **Errada.** CREATE FUNCTION é usado para criar funções, e não gatilhos.
- b) **Correta.** Para criar um gatilho usamos CREATE TRIGGER, seguido do nome, do momento de disparo, do evento, da tabela alvo e do código a ser executado.
- c) **Errada.** SELECT é usado para consultar dados, não existe SELECT TRIGGER para criação de gatilhos.
- d) **Errada.** DROP TRIGGER serve para remover um gatilho existente, e não para criá-lo.
- e) **Errada.** INSERT é um comando para inserir dados em tabelas, não para criar gatilhos.

Gabarito: B

31. (VUNESP/TTI (TJ RS)/TJ RS/Programador/2023) Um Técnico de Tecnologia da Informação do TJ-RS se deparou com uma situação na qual, sempre que o sistema adicionasse ou excluísse um registro de processo, seria preciso atualizar automática e simultaneamente uma tabela com a função de contador.

Para a resolução desse problema, considerando um banco de dados relacional e SQL, o procedimento ou funcionalidade mais indicado é:

- a) Procedure.
- b) View.
- c) Trigger.
- d) Job.
- e) Sequence.

Comentários:

a) **Errada.** Procedure (procedimento armazenado) é um conjunto de instruções SQL salvo no banco que precisa ser chamado manualmente para executar. Ela não dispara sozinha quando algo acontece na tabela, então não serve para reagir automaticamente a inserções e exclusões.

b) **Errada.** View é apenas uma consulta SQL armazenada com um nome, uma tabela virtual usada para consultar dados. Ela não executa ações automáticas em resposta a eventos como inserir ou excluir registros.

c) **Correta.** Trigger (gatilho) é exatamente o recurso que executa automaticamente quando um evento acontece no banco, como INSERT ou DELETE em uma tabela. Como o enunciado diz que toda vez que um processo for adicionado ou excluído a tabela contadora precisa ser atualizada automaticamente, o trigger é o mecanismo ideal, pois o próprio banco dispara sua execução sem precisar ser chamado.

d) **Errada.** Esse tema vai além do conteúdo desta aula.

e) **Errada.** Esse tema vai além do conteúdo desta aula.

Gabarito: C

32. (VUNESP/ATCE (TCM SP)/TCM SP/Técnico de Informática/2023) Considere o seguinte comando para a criação de um trigger no Transact-SQL:

```
CREATE TRIGGER Yellow ON Red
```

```
AFTER UPDATE
```

```
AS RAISEERROR ('Aviso', 16, 10);
```

Esse comando cria um trigger denominado

- a) Yellow, que é acionado quando algum registro da tabela Red é alterado, gerando uma mensagem de aviso.
- b) Yellow, que é acionado quando algum registro do banco de dados Red é excluído, gerando uma interrupção com um aviso.
- c) Red, que é acionado quando algum registro da tabela Yellow é alterado, enviando um e-mail ao responsável.
- d) Red, que é acionado quando alguma tabela do banco de dados Yellow é modificado, enviando um e-mail ao responsável.
- e) Red, que é acionado quando alguma chave primária da tabela Yellow for alterada, provocando uma parada do banco de dados.

Comentários:

- a) **Correta.** O nome da trigger vem logo após CREATE TRIGGER, portanto se chama Yellow. A cláusula ON Red indica a tabela alvo, e AFTER UPDATE significa que ela dispara depois que algum registro dessa tabela é alterado. O RAISEERROR apenas gera uma mensagem de aviso.
- b) **Errada.** O nome da trigger é Yellow, mas o evento é UPDATE (alteração), não exclusão (DELETE).
- c) **Errada.** O nome da trigger é Yellow (vem depois de CREATE TRIGGER) e a tabela alvo é Red (vem depois de ON), e não o contrário.
- d) **Errada.** Inverteu o nome da trigger e da tabela. Além disso, a trigger reage a alterações em registros da tabela, não em estruturas do banco.
- e) **Errada.** Inverteu novamente o nome da trigger e da tabela, e o evento UPDATE não se restringe a alterações em chave primária.

Gabarito: A

33. (FUNDATEC/ANC (PROCERGS)/PROCERGS/Desenvolvimento/Oracle PL SQL/2023) VIEW é um recurso da linguagem SQL que permite uma alternativa de acesso aos dados das tabelas do banco de dados. Sobre isso, analise assertivas abaixo e assinale a alternativa correta.

I. Depois de criar uma VIEW, ela pode ser usada em um comando SELECT.

- II. Não se pode executar comandos de alteração (UPDATE) ou exclusão (DELETE) em uma VIEW.
- III. É possível fazer uma inserção (INSERT) de dados em uma tabela através de qualquer VIEW criada com base em uma consulta dessa tabela, desde que tenha no mínimo um de seus campos.
- IV. A cláusula WITH READ ONLY indica que a VIEW somente permite que se consulte a sua estrutura, mas não pode ser executada.

- a) Todas estão corretas.
- b) Todas estão incorretas.
- c) Apenas I está correta.
- d) Apenas II e III estão corretas.
- e) Apenas III e IV estão corretas.

Comentários:

I. Correta. Uma view é uma consulta SQL armazenada no banco com um nome, funcionando como uma tabela virtual. Por isso, depois de criada, podemos consultá-la normalmente em um comando SELECT, como se fosse uma tabela qualquer (exemplo: SELECT * FROM Professores_de_Matematica).

II. Errada. Esse tema vai além do conteúdo desta aula em sua totalidade, mas pelo que vimos, a view se comporta como uma tabela virtual, e na prática é possível executar comandos de alteração e exclusão por meio dela em diversas situações, não havendo proibição absoluta.

III. Errada. Não basta a view ter pelo menos um campo da tabela para permitir inserção. Existem restrições (por exemplo, se a tabela original tiver campos obrigatórios que não estão na view, a inserção não funciona). Portanto, não é "qualquer VIEW" que aceita INSERT.

IV. Errada. A cláusula WITH READ ONLY não impede a execução da view, ela apenas impede que se façam alterações nos dados através dela. Ou seja, a view continua podendo ser consultada normalmente com SELECT, só não aceita comandos de modificação.

Gabarito: C

34. (FUNDATEC/Ana (GHC)/GHC/Sistemas/2023) Uma forma de otimização de consultas em bancos de dados é a utilização de visões (views) cujo conteúdo é calculado e armazenado em forma de tabelas. Esse tipo de visão é conhecido como:

- a) View temporária.
- b) View materializada.
- c) View auxiliar.
- d) View armazenada.
- e) View abstrata.

Comentários:

a) **Errada.** Não existe esse tipo de view com essa definição. A view comum é uma tabela virtual, montada na hora da consulta, sem armazenamento físico dos dados.

b) **Correta.** A view materializada é justamente aquela cujo conteúdo é calculado e armazenado fisicamente em disco, como uma tabela de verdade, diferente da view comum que é apenas virtual.

c) **Errada.** Não é uma classificação usada para esse tipo de view que armazena dados.

d) **Errada.** Embora toda view tenha sua definição armazenada no catálogo do banco, o termo correto para a view que guarda os dados calculados em disco é materializada.

e) **Errada.** Não corresponde ao conceito de view que armazena fisicamente o resultado da consulta.

Gabarito: B

35. (VUNESP/Ana TI (Pref Campinas)/Pref Campinas/2023) Considere a seguinte tabela de um banco de dados relacional:

Hotel (ID, Nome, Cidade, Estado)

O comando SQL para criar uma visão denominada Local, que contenha apenas os atributos ID, Cidade e Estado é:

a) SELECT ID, Cidade, Estado

FROM Hotel

AS CREATE VIEW Local;

b) CREATE VIEW Local AS

SELECT ID, Cidade, Estado

FROM Hotel;

c) SELECT ID, Cidade, Estado

FROM Hotel

AS VIEW Local (ID, Cidade, Estado);

d) CREATE Local AS VIEW

FROM Hotel (ID, Cidade, Estado);

e) CREATE VIEW Local (ID, Cidade, Estado)

FROM Hotel;

Comentários:

a) **Errada.** A sintaxe está invertida. O comando começa com CREATE VIEW seguido do nome e depois o AS com o SELECT, não o contrário.

b) **Correta.** Segue exatamente a estrutura para criar uma visão: CREATE VIEW, seguido do nome (Local), da palavra AS, e por fim a consulta SELECT que define quais atributos a visão vai conter (ID, Cidade e Estado da tabela Hotel).

c) **Errada.** Começa com SELECT, mas a criação de uma visão precisa começar com CREATE VIEW. A ordem dos comandos está incorreta.

d) **Errada.** A sintaxe está bagunçada. Falta o AS seguido de um SELECT, que é o que de fato define o conteúdo da visão. Não existe esse formato CREATE Local AS VIEW.

e) **Errada.** Falta a cláusula AS com o SELECT. Para criar uma visão, é obrigatório ter um SELECT após o AS, definindo a consulta que será armazenada.

Gabarito: B

36. (VUNESP/Ana (Fernandópolis)/CM Fernandópolis/Tecnologia da Informação/2022)
Considere a tabela Cliente de um banco de dados relacional.
Cliente (CPF, Nome, Cidade, Salario)

O comando SQL para criar uma visão de nome Vis1, contendo as colunas Nome e CPF é:

a) CREATE VIEW FROM Cliente AS Vis1 SELECT Nome, CPF;

b) CREATE VIEW Vis1 HAVING Nome, CPF FROM Cliente;

c) CREATE VIEW Vis1 FROM Cliente (Nome, CPF);

d) CREATE VIEW Vis1 USING Cliente (Nome, CPF);

e) CREATE VIEW Vis1 AS SELECT Nome, CPF FROM Cliente;

Comentários:

a) **Errada.** A ordem está toda trocada; o nome da view deve vir logo após CREATE VIEW, e o SELECT vem depois do AS.

b) **Errada.** Não se usa HAVING para definir as colunas da view; a estrutura correta é AS SELECT colunas FROM tabela.

c) **Errada.** Não existe a forma "FROM Cliente (Nome, CPF)" para criar view; é necessário o AS seguido de um SELECT.

d) **Errada.** Não se utiliza USING nessa sintaxe; o correto é AS SELECT ... FROM.

e) **Correta.** Segue exatamente o padrão de criação de visão: CREATE VIEW, nome da visão (Vis1), AS, e em seguida a consulta SELECT que define quais colunas e de qual tabela serão exibidas.

Gabarito: E

37. (VUNESP/Prog (Pres Prudente)/Pref Pres Prudente/Pleno/2022) Sistemas gerenciadores de bancos de dados admitem a criação de visões em bancos de dados, sendo correto afirmar que
- a) uma visão não admite a utilização de atributos do tipo chave primária.
 - b) visões têm sua estrutura e definição armazenadas fisicamente sob o controle do sistema gerenciador.
 - c) cada banco de dados criado não admite mais do que duas visões simultaneamente.
 - d) cada visão deve possuir exatamente os mesmos atributos de uma tabela já existente no banco de dados.
 - e) cada visão somente pode ser composta por dados de uma única tabela.

Comentários:

- a) **Errada.** Uma visão pode sim conter atributos do tipo chave primária, já que ela é apenas uma consulta SQL armazenada e pode trazer qualquer coluna das tabelas originais.
- b) **Correta.** A definição da visão (o texto do SELECT) fica armazenada no catálogo do banco, sob controle do SGBD, e o resultado só é computado no momento em que a visão é consultada.
- c) **Errada.** Não há limite de apenas duas visões por banco de dados; podemos criar quantas visões forem necessárias.
- d) **Errada.** A visão pode ter atributos diferentes das tabelas originais, podendo selecionar apenas algumas colunas, combinar dados de várias tabelas ou aplicar filtros.
- e) **Errada.** Uma visão pode ser composta por dados de várias tabelas, pois ela é simplesmente uma consulta SQL armazenada, e consultas podem envolver múltiplas tabelas.

Gabarito: B

38. (ESAF/AFRFB/SRFB/Política e Administração Tributária/2002) Os triggers são um tipo especial de procedimento armazenado. Em vez de serem executados pelo usuário, eles são executados pelo servidor do banco de dados quando certas operações são realizadas numa tabela.

Comentários:

A afirmação está perfeitamente alinhada com o conceito de trigger. O gatilho é justamente um tipo especial de procedimento armazenado, com uma diferença marcante: ele não é chamado diretamente pelo usuário, como acontece com uma stored procedure comum. Quem dispara a execução é o próprio banco de dados, automaticamente, quando determinado evento ocorre na tabela, como um INSERT, UPDATE ou DELETE. Por isso o nome gatilho, ele é acionado sozinho diante de uma operação que o sensibiliza.

Gabarito: Certa

39. (ADAPTADA - ADM&TEC/Tec (CM Toritama)/CM Toritama/Informática/2023) Uma stored procedure é um grupo de comandos SQL, que executa uma determinada tarefa. Assim como

uma trigger, uma procedure é executada automaticamente, dispensando ser chamada a partir de um programa ou manualmente pelo usuário.

Comentários:

A afirmação está errada, e como o gabarito apresentado é C, há um problema. Vou explicar pelo conteúdo da aula. A primeira parte está correta: a stored procedure realmente é um conjunto de comandos SQL armazenados no banco que executam uma tarefa. Mas a segunda parte inverte um conceito importante. A stored procedure NÃO é executada automaticamente. Ela precisa ser chamada, seja por uma aplicação, seja manualmente pelo usuário. Quem é executada automaticamente é a trigger, justamente por isso ela é chamada de gatilho: o próprio banco dispara sua execução quando um evento acontece (como um INSERT, UPDATE ou DELETE). Essa é exatamente a diferença central entre os dois recursos: a procedure precisa ser invocada, a trigger dispara sozinha diante de um evento. Portanto, o correto seria classificar essa afirmação como errada.

Gabarito: Certa

LISTA DE QUESTÕES



1. (CEBRASPE (CESPE)/TRF 6/Apoio Especializado/Análise de Dados/2025) Uma view é baseada em um tipo de consulta executada em uma ou mais tabelas em um banco relacional; normalmente é um tipo de consulta complexa e utilizada com frequência.
2. (CEBRASPE (CESPE)/FUNPRESPEX/Sistemas e Governança de Tecnologia da Informação/2025) Uma view materializada realiza consulta em tabelas do banco de dados no momento em que o usuário faz uma consulta à view.
3. (CEBRASPE (CESPE)/TJ TRF6/TRF 6/Apoio Especializado/Suporte Técnico/2025) Os bancos de dados podem disparar instruções automáticas chamadas triggers antes ou depois de executar comandos DML, para garantir restrições de integridade.
4. (CEBRASPE (CESPE)/ANM/Tecnologia da Informação/Ciência de Dados/2025) Os triggers são baseados no modelo evento-condição-ação, no qual um evento no banco de dados dispara uma ação quando determinada condição é atingida.
5. (CEBRASPE (CESPE)/BANRISUL/Administração de Banco de Dados/2025) Em sistemas de gerenciamento de banco de dados relacionais, triggers
 - a) BEFORE TRUNCATE são executados antes de apagar as linhas e permitem cancelar a operação lançando erro.
 - b) AFTER DELETE são acionados antes de remover fisicamente a linha, de modo que, durante seu processamento, a linha ainda permanece na tabela.
 - c) BEFORE INSERT disparam antes de alocar espaço para a nova linha e podem ajustar valores usando NEW, ao passo que triggers AFTER INSERT disparam somente depois de todas as restrições terem sido verificadas e a linha ter sido gravada permanentemente.
 - d) BEFORE UPDATE são executados depois de atualizar os valores, porém antes de reindexar a tabela, permitindo reverter a operação caso alguma condição falhe.
 - e) AFTER TRUNCATE permitem o acesso a dados antigos das linhas por meio de uma pseudotabela OLD, possibilitando registrar o que foi apagado.
6. (CEBRASPE (CESPE)/Ana CT (CAPES)/CAPES/Informática/2024) Um trigger de banco de

dados pode ser disparado antes ou depois de um comando e pode, até mesmo, substituir esse comando.

7. (CEBRASPE (CESPE)/NATEL/Ciência de Dados/2024) Em SQL, triggers são procedimentos automáticos executados em resposta a determinados eventos em uma tabela ou view e são disparados exclusivamente em resposta a ações DML, tais como INSERT, UPDATE ou DELETE.

8. (CEBRASPE (CESPE)/SEBRAE/Banco de Dados/2024) Nos bancos de dados SQL Servers, os gatilhos DDL são disparados nos eventos de

a) DENY, GRANT e UPDATE.

b) DENY, GRANT e REVOKE.

c) DELETE, DENY e REVOKE.

d) DENY, GRANT e INSERT.

9. (FCC/AM (MPE PB)/Administrador de Banco de Dados/2023) Um analista deseja executar um procedimento previamente armazenado em um banco de dados, quando da ocorrência de um evento SQL do tipo delete. Essa execução está corretamente relacionada ao conceito de

a) Broker.

b) Instanciação.

c) Trigger.

d) Generalização.

e) Docker.

10. (FCC/Ana Leg (ALAP)/ALAP/Atividade de Tecnologia da Informação/Desenvolvedor de Banco de Dados/2020) Triggers ou Gatilhos são comandos (ação) executados automaticamente (um tipo especial de procedimento armazenado) em consequência de uma modificação (evento) que produziu uma restrição (condição) que implique em defeito ou falta de integridade nos dados. É correto afirmar que

a) um trigger é usado com enorme eficiência para impor e manter integridade referencial de alto nível e, também, para ajudar na elaboração de resultados de consultas.

b) um trigger e o evento que o disparou devem ser considerados como uma mesma transação.

c) triggers são criados utilizando a instrução SQL Generate Trigger, que especifica o Banco onde ele atuará e para que tipo de ocorrência disparará suas ações.

d) os triggers podem ser executados, independentemente dos eventos aos quais foram associados.

e) triggers e Stored Procedures são procedimentos equivalentes; enquanto triggers disparam em função de uma ação, as Stored Procedures são disparadas por eventos temporais.

11. (FCC/Ana Leg (ALAP)/ALAP/Atividade de Tecnologia da Informação/Desenvolvedor de Banco de Dados/2020) Considere uma tabela de produtos em estoque e outra de pedidos, em um banco de dados. Toda vez que um item for incluído na tabela de pedidos, a quantidade em estoque desse item, na tabela de produtos, deve ser diminuída de modo equivalente. Um

Analista pode solucionar essa situação, e ainda ganhar em desempenho,

a) usando um trigger.

b) criando uma view com dados de produtos e de pedidos.

c) atribuindo o acesso a essas tabelas para apenas um usuário responsável.

d) atribuindo privilégios à tabela de pedidos.

e) atribuindo privilégios à tabela de produtos.

12. (FCC/Ana G (DPE AM)/ Analista de Banco de Dados/2018) Visões constituem um tipo de estrutura possível de ser criada em um banco de dados relacional e apresentam como característica:

a) Permitir a consulta apenas a tabelas que possuam somente atributos do tipo numérico armazenados.

b) Possuir o armazenamento somente do comando de criação de sua estrutura, sendo os dados consultados a partir de tabelas com dados fisicamente armazenados.

c) Não comportar a visualização de atributos do tipo booleano.

d) Exibir apenas as chaves primária e estrangeira das tabelas especificadas em sua estrutura.

e) Ser consultadas um número restrito de vezes, conforme seja o sistema gerenciador de banco de dados utilizado.

13. (FGV/ATTM (Pref Nova Iguaçu)/Pref Nova Iguaçu/2024) Seja um banco de dados relacional especificado em SQL de uma empresa de correspondência entre clientes, instituições financeiras e empréstimos contratados por esses clientes nessas instituições, previamente implementado em um banco de dados como a seguir:

OBS: Neste banco de dados, cadeias de caracteres (strings) são representadas envoltas em aspas simples.

Considere a seguinte sequência de comandos SQL:

```
insert into tb_cliente(id_cliente, num_cpf, nome)
```

values

```
(1,'111','cli1'),(2,'222','cli2'),(3,'333','cli3'),  
(4,'444','cli4'),(5,'555','cli5');
```

```
update tb_cliente set estado='RJ'
```

```
where id_cliente%2!=0;
```

```
create view v_clientes_rj as
```

```
select id_cliente, num_cpf, nome, estado
```

```
from tb_cliente where estado = 'RJ';
```

Após a execução dos três comandos SQL apresentados, o seguinte comando foi executado:

```
insert into v_clientes_rj
```

```
values (6,'cli6','6666','MG');
```

Por fim, a seguinte consulta foi executada:

```
select count(*) from v_clientes_rj;
```

O retorno da última consulta executada será

- a) o valor 3, dado que a visão permanece selecionando tuplas de clientes do estado 'RJ'.
- b) o valor 4, já que a execução do último comando de inserção alterou o estado da visão.
- c) o valor 6, por esse ser o novo número de tuplas da tabela de base da visão.
- d) um erro, pois visões não aceitam inserções diretamente.
- e) uma tabela vazia, pois a última inserção desconfigura a condição-alvo da visão.

14. (FGV/AJ (TJ RR)/TJ RR/Ciência de Dados e Analytics/2024) Ao projetar um banco de dados, espera-se atualizar valores de alguns campos de uma tabela a cada vez que um novo registro é inserido em outra tabela.

Para que isso aconteça sem a intervenção do usuário, deve ser implementada, em SQL, a estrutura denominada

- a) FUNCTION.
- b) SCHEMA.
- c) TRIGGER.

d) UPDATE.

e) VIEW.

15. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Sobre a utilização de views, em bancos de dados como MySQL, SQL Server, analise as afirmativas a seguir.

I. Views servem para construir e armazenar o conteúdo de consultas complexas de uso frequente.

II. Os principais SGBD não permitem o uso de operações de update/delete sobre views.

III. Em geral, as views não podem ser referenciadas na cláusula FROM em outras consultas SQL.

Está correto somente o que se afirma em:

a) I;

b) II;

c) III;

d) I e II;

e) II e III.

16. (FGV/ATRFB/SRFB/Geral/2023) Considere um banco de dados relacional em que as operações de insert e update efetuadas numa certa tabela devem ser monitoradas e anotadas, como subsídio aos procedimentos de auditoria da empresa. Essa tabela é utilizada por uma série de aplicações, em diferentes tipos de transações, e iniciadas por um número considerável de usuários.

Nesse cenário, assinale o mecanismo mais adequado para a implementação desse monitoramento.

a) Cursores.

b) Stored procedures.

c) Triggers.

d) Utilitários de exportação de dados.

e) Views.

17. (FGV/Ana (DPE RS)/DPE RS/Apoio Especializado (TI)/Banco de Dados/2023) Num comando de criação de um trigger no MySQL, nas recentes edições, há um conjunto de elementos sintáticos que podem ser utilizados para determinar quando os procedimentos devem

ser acionados e quais são esses procedimentos.

As possíveis combinações de “quando x procedimento” válidas para triggers de transações são:

- a) AFTER/BEFORE combinados com DELETE/INSERT/UPDATE;
- b) CREATION/REMOTION combinados com READ/WRITE;
- c) IN/OUT combinados com CREATION/REMOTION;
- d) INIT/END combinados com INPUT/OUTPUT;
- e) STARTING/FINISHING combinados com READ/ REWRITE/WRITE.

18. (FGV/ATCE (TCE,AM)/TCE AM/Auditoria de Tecnologia da Informação/2021) Maria gerencia um banco de dados SQL Server, e precisa criar um mecanismo que monitore todas as alterações ocorridas na estrutura das tabelas, especialmente os efeitos dos comandos CREATE, ALTER e DROP.

A estratégia mais adequada para detectar comandos dessa natureza passa pela criação e uso de:

- a) Funções escalares;
- b) Sequences;
- c) Statistics;
- d) Stored Procedures;
- e) Triggers.

19. (FGV/Ana Leg (ALETO)/ALETO/Desenvolvimento de Sistema/2024) Visões em SQL possuem um papel relevante em bancos de dados relacionais. Ela oferece mecanismos importantes, tais como abstração de dados, segurança e simplificação das consultas.

Nesse contexto, é correto afirmar que

- a) a cláusula WITH CHECK OPTION da SQL é demandada no final da especificação da visão, a fim de que a tornar uma asserção.
- b) a estratégia de modificação de consultas para a implementação de visões é eficaz para consultas complexas.
- c) a responsabilidade pela atualização do estado de uma visão é do usuário, que necessita enviar um comando refresh a cada execução.
- d) uma visão em linha (view em linha) consiste na definição de uma visão na cláusula HAVING de uma consulta SQL.

e) visões materializadas criam uma tabela temporária, utilizando técnicas como a atualização incremental.

20. (FGV/ACE (TCE SP)/TCE SP/Tecnologia da Informação (TI)/2023) Views, em bancos de dados relacionais, configuram um importante mecanismo para flexibilizar o acesso aos dados armazenados. Em alguns casos, são ditas atualizáveis, pois podem ser utilizadas como alvo de comandos SQL como insert, update, delete.

Uma característica que não impede uma view de ser atualizável é:

- a) a cláusula group by;
- b) a cláusula distinct;
- c) a cláusula order by;
- d) a presença do operador algébrico union;
- e) a inexistência de uma fonte de registros atualizáveis na cláusula from.

21. (CESGRANRIO/Tec Uni (UNEMAT)/UNEMAT/Analista de Sistemas/2024) A respeito do uso de procedimentos armazenados e de gatilhos em um banco de dados relacional, verifica-se, em relação à sua aplicabilidade, que

- a) os gatilhos são ideais para encapsular lógica de apresentação em um banco de dados.
- b) os gatilhos são usados, principalmente, para encapsular lógica de negócios complexa e reutilizável.
- c) os procedimentos armazenados são preferíveis para impor restrições de integridade referencial.
- d) os procedimentos armazenados são adequados para automatizar a execução de ações em resposta a eventos específicos.
- e) ambos, procedimentos armazenados e gatilhos, são exclusivamente usados para consultas complexas em bancos de dados.

22. (CESGRANRIO/TBN (CEF)/CEF/Tecnologia da Informação/2024) O comando SQL de criação de gatilhos, CREATE TRIGGER, possui algumas informações obrigatórias, dentre elas, a seguinte:

- a) a hora na qual o gatilho precisa ser disparado.
- b) a que tabela do banco de dados o gatilho estará vinculado.
- c) o e-mail a ser notificado após a sua execução.
- d) o usuário para o qual o gatilho será disparado automaticamente.

e) o disparo do gatilho está vinculado à execução de um comando SELECT.

23. (CESGRANRIO/PTNS (TRANSPETRO)/2023) Gatilhos (triggers) e procedimentos armazenados (stored procedures) são componentes fundamentais em sistemas de gerenciamento de banco de dados (SGBDs) relacionais. Tanto os gatilhos quanto os procedimentos armazenados desempenham papéis vitais e, muitas vezes, complementares em aplicações baseadas em banco de dados relacionais, sendo escolhidos de acordo com as necessidades específicas de uma aplicação ou de um sistema. A respeito de gatilhos e de procedimentos armazenados, tem-se que

a) tanto os gatilhos quanto os procedimentos armazenados são executados manual e explicitamente por um usuário ou por um programa de aplicação, e não podem ser disparados automaticamente por eventos de atualização em um banco de dados.

b) tanto os gatilhos quanto os procedimentos armazenados são disparados automaticamente por eventos de atualização em um banco de dados, e não podem ser ativados manual e explicitamente por um usuário ou por um programa de aplicação.

c) gatilhos, ao contrário dos procedimentos armazenados, não podem expressar a lógica de regras de negócios, e são utilizados exclusivamente para validação de dados.

d) gatilhos são disparados automaticamente em resposta a eventos de atualização em um banco de dados, enquanto procedimentos armazenados precisam ser chamados explicitamente.

e) gatilhos e procedimentos armazenados não podem coexistir em um mesmo sistema de banco de dados, pois possuem funcionalidades idênticas e redundantes.

24. (CESGRANRIO/Tec Cien (BASA)/BASA/Tecnologia da Informação/2022) Em banco de dados, os gatilhos são utilizados, entre outros objetivos, para implementar restrições de integridade. Considere que em um banco de dados de um banco comercial há duas tabelas, CLIENTE (chave primária CPF) e SERVICO (chave primária composta por CPF e identificação do serviço), e há a restrição de que um cliente não pode estar associado a mais de cinco serviços.

Que definição deve ser utilizada para implementar essa restrição por meio de um gatilho?

a) BEFORE INSERT ON CLIENTE

b) AFTER INSERT ON CLIENTE

c) BEFORE INSERT ON SERVICO

d) AFTER INSERT ON SERVICO

e) WHEN CLIENTE INSERT ON SERVICO

25. (CESGRANRIO/Esc BB/BB/Agente de Tecnologia/2021) Para que fosse mais fácil entender um sistema em desenvolvimento, um desenvolvedor usou um modelo de dados relacional

(protótipo) mais simples do que o do banco de dados do sistema corporativo original, sendo que ambos utilizavam o mesmo SGDB PostgreSQL. Cabe ressaltar que esse protótipo utilizava apenas um subconjunto dos dados do sistema corporativo original e realizava apenas consultas.

Uma forma de garantir que os dados desse protótipo estejam sempre completamente atualizados em relação aos dados reais, com baixo impacto tanto na operação quanto no desempenho do sistema corporativo original, é

a) criar apenas VIEWS no protótipo, definidas com consultas sobre as tabelas do sistema corporativo original.

b) implantar TRIGGERS a cada INSERT, em todas as tabelas do sistema corporativo original, atualizando as tabelas do protótipo.

c) implantar TRIGGERS de atualização a cada SELECT, em todas as tabelas do protótipo.

d) particionar as tabelas da base do sistema corporativo original escolhendo um RANGE adequado ao trabalho do protótipo.

e) utilizar DUMP da base do sistema corporativo original e PSQL para a base do protótipo, a cada seção de trabalho, para atualizar a base do protótipo.

26. (VUNESP/ATI (Pref Santo André)/Pref Santo André/Administração de Bancos de Dados/2024) O comando SQL para, a partir da seguinte tabela de um banco de dados relacional Biblioteca (ID, Título, Autor1, Autor2, Ano)

criar uma visão denominada Biblio, contendo os atributos ID e Título é:

a) CREATE VIEW Biblio (ID, Título)

FROM Biblioteca;

b) CREATE VIEW Biblio

SELECT Biblioteca (ID, Título);

c) CREATE VIEW Biblio

WHERE Biblio (ID, Título);

d) CREATE VIEW Biblio AS

(SELECT ID, Título

FROM Biblioteca);

e) CREATE VIEW Biblio

FROM Biblioteca (ID, Título);

27. (VUNESP/CFO/QC (EsFCEEx)/EsFCEEx/Informática/2024) O comando SQL para criar uma visão de nome Teste, a partir dos atributos C1 e C2 da tabela denominada Prova é:

- a) CREATE TABLE VISION Teste USING (C1, C2 FROM Prova);
- b) CREATE TABLE VIEW Teste AS SELECT C1, C2 FROM Prova;
- c) CREATE VISION Teste FROM Prova (C1, C2);
- d) CREATE VIEW Teste FROM Prova (C1, C2);
- e) CREATE VIEW Teste AS SELECT C1, C2 FROM Prova;

28. (VUNESP/Pref Santo André/Administração de Bancos de Dados/2024) Quando da definição de algumas estruturas em um banco de dados, pode ser necessário especificar os chamados triggers (gatilhos) que

- a) representam a estrutura física das tabelas contidas no banco de dados.
- b) representam um tipo especial de tabela que contém dados do criador do banco de dados.
- c) especificam os tempos máximos de execução de cada consulta feita ao banco de dados.
- d) constituem uma tabela que contém a relação de usuários e seus privilégios dentro do banco de dados.
- e) são procedimentos acionados de forma automática, como efeito da ocorrência de algum evento especificado em seu comando.

29. (FUNDATEC/AFRM (Pref Criciúma)/2024) Sobre bancos de dados, assinale a alternativa INCORRETA.

- a) Uma tabela é uma estrutura de dados que armazena linhas que representam uma coleção de valores de dados relacionados.
- b) Uma view é uma tabela virtual que é derivada de outras tabelas, armazenando os dados fisicamente em outro local.
- c) Um índice é uma estrutura de dados que facilita o acesso rápido aos dados em uma tabela.
- d) Uma chave primária é um atributo ou conjunto de atributos que identifica unicamente cada registro em uma tabela.
- e) Uma chave estrangeira mantém uma restrição de integridade referencial entre uma entidade referenciada e outra que referencia.

30. (VUNESP/Ana (Pref Marília)/Pref Marília/Dados/2023) A sintaxe básica do comando de criação de gatilhos em SQL é:

- a) CREATE FUNCTION ...
- b) CREATE TRIGGER ...
- c) SELECT TRIGGER ...
- d) DROP TRIGGER ...
- e) INSERT TRIGGER ...

31. (VUNESP/TTI (TJ RS)/TJ RS/Programador/2023) Um Técnico de Tecnologia da Informação do TJ-RS se deparou com uma situação na qual, sempre que o sistema adicionasse ou excluísse um registro de processo, seria preciso atualizar automática e simultaneamente uma tabela com a função de contador.

Para a resolução desse problema, considerando um banco de dados relacional e SQL, o procedimento ou funcionalidade mais indicado é:

- a) Procedure.
- b) View.
- c) Trigger.
- d) Job.
- e) Sequence.

32. (VUNESP/ATCE (TCM SP)/TCM SP/Técnico de Informática/2023) Considere o seguinte comando para a criação de um trigger no Transact-SQL:

```
CREATE TRIGGER Yellow ON Red
```

```
AFTER UPDATE
```

```
AS RAISEERROR ('Aviso', 16, 10);
```

Esse comando cria um trigger denominado

- a) Yellow, que é acionado quando algum registro da tabela Red é alterado, gerando uma mensagem de aviso.
- b) Yellow, que é acionado quando algum registro do banco de dados Red é excluído, gerando uma interrupção com um aviso.
- c) Red, que é acionado quando algum registro da tabela Yellow é alterado, enviando um e-mail ao responsável.
- d) Red, que é acionado quando alguma tabela do banco de dados Yellow é modificado, enviando um e-mail ao responsável.

e) Red, que é acionado quando alguma chave primária da tabela Yellow for alterada, provocando uma parada do banco de dados.

33. (FUNDATEC/ANC (PROCERGS)/PROCERGS/Desenvolvimento/Oracle PL SQL/2023) VIEW é um recurso da linguagem SQL que permite uma alternativa de acesso aos dados das tabelas do banco de dados. Sobre isso, analise assertivas abaixo e assinale a alternativa correta.

I. Depois de criar uma VIEW, ela pode ser usada em um comando SELECT.

II. Não se pode executar comandos de alteração (UPDATE) ou exclusão (DELETE) em uma VIEW.

III. É possível fazer uma inserção (INSERT) de dados em uma tabela através de qualquer VIEW criada com base em uma consulta dessa tabela, desde que tenha no mínimo um de seus campos.

IV. A cláusula WITH READ ONLY indica que a VIEW somente permite que se consulte a sua estrutura, mas não pode ser executada.

a) Todas estão corretas.

b) Todas estão incorretas.

c) Apenas I está correta.

d) Apenas II e III estão corretas.

e) Apenas III e IV estão corretas.

34. (FUNDATEC/Ana (GHC)/GHC/Sistemas/2023) Uma forma de otimização de consultas em bancos de dados é a utilização de visões (views) cujo conteúdo é calculado e armazenado em forma de tabelas. Esse tipo de visão é conhecido como:

a) View temporária.

b) View materializada.

c) View auxiliar.

d) View armazenada.

e) View abstrata.

35. (VUNESP/Ana TI (Pref Campinas)/Pref Campinas/2023) Considere a seguinte tabela de um banco de dados relacional:

Hotel (ID, Nome, Cidade, Estado)

O comando SQL para criar uma visão denominada Local, que contenha apenas os atributos ID, Cidade e Estado é:

a) SELECT ID, Cidade, Estado

FROM Hotel

AS CREATE VIEW Local;

b) CREATE VIEW Local AS

SELECT ID, Cidade, Estado

FROM Hotel;

c) SELECT ID, Cidade, Estado

FROM Hotel

AS VIEW Local (ID, Cidade, Estado);

d) CREATE Local AS VIEW

FROM Hotel (ID, Cidade, Estado);

e) CREATE VIEW Local (ID, Cidade, Estado)

FROM Hotel;

36. (VUNESP/Ana (Fernandópolis)/CM Fernandópolis/Tecnologia da Informação/2022) Considere a tabela Cliente de um banco de dados relacional.

Cliente (CPF, Nome, Cidade, Salario)

O comando SQL para criar uma visão de nome Vis1, contendo as colunas Nome e CPF é:

a) CREATE VIEW FROM Cliente AS Vis1 SELECT Nome, CPF;

b) CREATE VIEW Vis1 HAVING Nome, CPF FROM Cliente;

c) CREATE VIEW Vis1 FROM Cliente (Nome, CPF);

d) CREATE VIEW Vis1 USING Cliente (Nome, CPF);

e) CREATE VIEW Vis1 AS SELECT Nome, CPF FROM Cliente;

37. (VUNESP/Prog (Pres Prudente)/Pref Pres Prudente/Pleno/2022) Sistemas gerenciadores de bancos de dados admitem a criação de visões em bancos de dados, sendo correto afirmar que

a) uma visão não admite a utilização de atributos do tipo chave primária.

b) visões têm sua estrutura e definição armazenadas fisicamente sob o controle do sistema gerenciador.

c) cada banco de dados criado não admite mais do que duas visões simultaneamente.

d) cada visão deve possuir exatamente os mesmos atributos de uma tabela já existente no banco de dados.

e) cada visão somente pode ser composta por dados de uma única tabela.

38. (ESAF/AFRFB/SRFB/Política e Administração Tributária/2002) Os triggers são um tipo especial de procedimento armazenado. Em vez de serem executados pelo usuário, eles são executados pelo servidor do banco de dados quando certas operações são realizadas numa tabela.

39. (ADAPTADA - ADM&TEC/Tec (CM Toritama)/CM Toritama/Informática/2023) Uma stored procedure é um grupo de comandos SQL, que executa uma determinada tarefa. Assim como uma trigger, uma procedure é executada automaticamente, dispensando ser chamada a partir de um programa ou manualmente pelo usuário.

GABARITO

1. Certo	14. C	27. E
2. Errado	15. A	28. E
3. Certo	16. C	29. B
4. Certo	17. A	30. B
5. C	18. E	31. C
6. Certo	19. E	32. A
7. Errado	20. C	33. C
8. B	21. D	34. B
9. C	22. B	35. B
10. B	23. D	36. E
11. A	24. C	37. B
12. B	25. A	38. Certa
13. A	26. D	39. Certa

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.